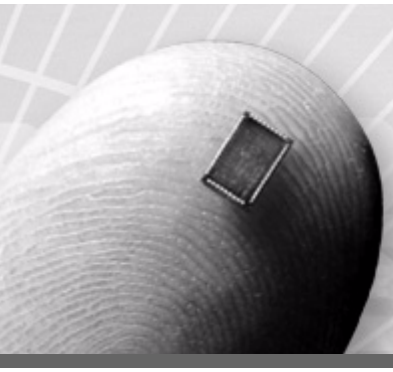
ON Semiconductor®



CFX Toolchain Reference

for BELASIGNA® 300

M-20517-010

ON Semiconductor

September 2011

This document may not be redistributed or transmitted in any form.

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor.

ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). BELASIGNA and SIGNAKLARA are either trademarks or registered trademarks of ON Semiconductor. All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.

ON Semiconductor reserves the right to make changes without further notice to any products herein. ON Semiconductor makes no warranty, representation or guarantee regarding its products including suitability for any particular purpose, nor does ON Semiconductor assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. "Typical" parameters which may be provided in ON Semiconductor data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. ON Semiconductor does not convey any license under its patent rights nor the rights of others. ON Semiconductor products are not designed, intended, or authorized for use as components in systems intended for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the ON Semiconductor product could create a situation where personal injury or death may occur, unless otherwise expressly agreed to in writing by ON Semiconductor and BUYER. Should Buyer purchase or use ON Semiconductor products for any such unintended or unauthorized application, Buyer shall indemnify and hold ON Semiconductor and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that ON Semiconductor was negligent regarding the design or manufacture of the part. ON Semiconductor is an Equal Opportunity/Affirmative Action Employer. Literature is subject to all applicable copyright laws and is not for resale in any manner.

To maintain the quality of our publications, we request your comments on the accuracy, clarity, organization, and value of this document.

Address correspondence to:

Americas and Asia
ON Semiconductor
200-611 Kumpf Drive
Waterloo, Ontario, Canada N2V 1K8
Tel: +1 519 884 9696

Europe, Middle East and Africa
ON Semiconductor
Champs-Montants 12a
2074 Marin, Switzerland
Tel: +41 32 755 74 30

Website
www.onsemi.com

E-mail
dsp.support@onsemi.com

TABLE OF CONTENTS

1. Introduction 1

1.1 Purpose . . . 1
1.2 Building Software . . . 1
1.2.1 Building Software with the IDE . . . 1
1.2.2 Building Software from the Command Line . . . 1
1.3 Intended Audience . . . 2
1.4 Conventions . . . 2
1.5 Manual Organization . . . 2
1.6 Further Reading . . . 3

2. Building Software with the IDE 5

2.1 Introduction . . . 5
2.2 Build Stages . . . 5
2.3 Build Settings . . . 5
2.4 Auto-Build . . . 7

3. Running the Command-Line Tools 9

3.1 Introduction . . . 9
3.2 The Stages of Compilation . . . 9
3.2.1 Preprocessing . . . 9
3.2.2 Assembling . . . 9
3.2.3 Linking . . . 9
3.3 CFCC Usage . . . 10
3.3.1 Syntax . . . 10
3.3.2 CFCC - Toolchain Driver Options . . . 11
3.4 Error Codes . . . 12
3.5 Error Messages . . . 12
3.5.1 “Option error(option): message” . . . 12
3.5.2 “IO Error: message” . . . 13
3.5.3 “No input files” . . . 13
3.5.4 “Unknown target specified in -b: target” . . . 13

3.5.5 "Cannot specify -o with -c and multiple compilations" 13
3.5.6 "Error: argument is not a file" 13
3.5.7 "Don't know how to build file: filename" 14
3.5.8 "System exception when running tool" 14
3.5.9 "Error running tool: message" 14
3.5.10 "Unexpected error running tool" 14
3.5.11 "Error moving filename to destination: message" 14
3.5.12 "Error copying filename to destination: message". 14
3.5.13 "An unexpected error has occurred in tool. Please contact ON Semiconductor customer support". . . . 14

4. Preprocessing Source Code *15*

4.1 Introduction 15
4.2 Running the Preprocessor 15
4.2.1 Preprocessing Options 15
4.2.2 CFINCLUDE Environment Variable 17
4.2.3 Predefined Macros 17
4.3 ON Semiconductor Preprocessing Language 18
4.3.1 Preprocessing Directive Summary. 18
4.3.2 Preprocessing Procedure 19
4.3.3 Lexical Extensions 20
4.3.3.1 Dollar in Names 20
4.3.3.2 Comment Removal 20
4.3.3.3 Escaping the Hash Character 21
4.3.4 Preprocessor Expressions 21
4.3.5 Directive Extensions 22
4.3.5.1 Details of #include File Searching 22
4.3.5.2 Comment Removal 22
4.3.5.3 Generating a Warning with #warning 22
4.3.5.4 Repeating Text Blocks Using #rept 23
4.3.5.5 Defining Multi-Line Macros with #macro. 23
4.3.5.6 Defining Numeric Macros with #set 23
4.3.5.7 Comparison of Macro Types 24

5. Assembling Programs 27

5.1 Introduction 27
5.2 Running the Assembler 27
5.2.1 Assembler Options 28
5.2.2 Instruction Restriction Checks 30
5.2.2.1 Restriction Categories 30
5.2.2.2 Restriction Error Messages 31
5.3 CFX Assembly Language 32
5.3.1 Lexical Conventions 32
5.3.2 Assembly Language Syntax 34
5.3.2.1 Instruction Forms 34
5.3.2.2 Synonyms and Synthesized Instructions 35
5.3.2.2.1 Synonym Instructions and Synthesized Instructions . . . 35
5.3.2.2.2 Alternate Condition Forms 36
5.3.2.2.3 Addressing Mode Synonyms 36
5.3.2.3 Numeric Instruction Operand Syntax 37
5.3.3 Assembler Directives 39
5.3.3.1 Directives for Declaring Symbols 39
5.3.3.2 Directives for Defining Data 40
5.3.3.3 Directives for Changing the Program Counter 42
5.3.3.4 Directives for Partitioning Memory 43
5.3.3.5 Directives for Controlling Checking 45
5.3.3.6 Directives for Preprocessing 47
5.3.3.7 Directives for Controlling the Listing File 48
5.3.3.8 Directives for Defining Special Object File Entries 49
5.4 Generating a Cross-Reference 51
5.4.1 Cross-Reference Options 51
5.4.1.1 Symbol Selection 51
5.4.1.2 Symbol Filtering 51
5.4.1.3 Output Controls 52
5.4.2 Defaults 53
5.4.3 Cross-Reference Output 53
5.5 Generating Debugging Symbols 53
5.5.1 Options 53

6. Using the Object Librarian 55

6.1 Introduction 55
6.2 Librarian Usage 55
6.2.1 Syntax 55
6.2.2 Options 55
6.3 Ordering Object Modules in a Library 56

7. Linking Object Modules 57

7.1 Introduction 57
7.2 Linking Relocatable Object Code 57
7.2.1 Linking Options 57
7.2.2 CFLIB Environment Variable 60
7.3 Ordering Object Module Libraries 60
7.4 Memory Configuration 61
7.4.1 Memory Allocation for SK3 Systems 61
7.5 Special Symbols 62
7.6 Load Map Format 63
7.7 Converting to .cfo Format 64
7.7.1 ABSDUMP Usage 65

8. Creating Configurable Programs 67

8.1 Introduction 67
8.2 Configuration Templates 67
8.2.1 Create a Configuration Template 67
8.2.2 Saving Configuration Template Data 69
8.2.3 Loading a Configuration Template 69
8.3 Program Parameters 69
8.3.1 Parameter Types 71
8.3.2 Adding Parameters 71
8.3.3 Viewing Parameter Properties 74
8.3.4 Editing Parameters 74
8.3.5 Setting Parameter Values 75

8.3.5.1 User Defined Values 75
8.3.5.2 I2C Updates 76
8.3.6 Removing Parameters 76
8.4 General I2C Commands 77
8.5 DSP Implementation Requirements for I2C Runtime Configuration Support 78
8.6 Exporting Configuration Data 80
8.6.1 Exporting Absolute Executable Format 80
8.6.2 Exporting Absolute Executable Bundle Format 81

9. Non-Volatile Memory Management 83

9.1 Introduction 83
9.2 Features 83
9.2.1 Flexible Layout Arrangement 83
9.2.2 Optimization Through Automatic Equivalent Section Detection . . 84
9.2.3 Encryption Support 84
9.2.4 EEPROM Image Upload 84
9.2.5 Graphical Layout View Using NVI Manager 84
9.3 Using NVMM 84
9.3.1 Creating an EEPROM Image Using the Project Wizard 85
9.3.1.1 Creating an EEPROM Project 85
9.3.1.2 BelaSigna 300 EEPROM Layout Defaults 85
9.3.1.3 Editing Build Settings 86
9.3.1.4 Adding an Application 87
9.3.1.5 Securing an EEPROM Image 90
9.3.2 Creating an EEPROM Image by Uploading 92
9.3.3 Reserving a Section of EEPROM 93
9.3.4 Creating an EEPROM Project (with EEPROM Image Output) from an Existing NVI File 94
9.3.5 Viewing an EEPROM Image with NVI Manager 95
9.3.5.1 Virtual File System (VFS) Tree View 96
9.3.5.2 Graphical EEPROM Layout View 96
9.3.5.3 Additional NVI File Views 98
9.3.5.4 NVI Manager Preferences 98

9.4 Options 98
9.4.1 General Options 99
9.4.1.1 Bootloader-Specific Options 100
9.4.1.2 Application-Specific Options 100
9.4.2 Global Field Options 103
9.4.3 BelaSigna 300 (SK3FS) Areas 104
9.4.4 Loading Arbitrary Data Files 104

10. Non-Volatile Image Tools *107*

10.1 Introduction 107
10.2 Non-Volatile Memory Download Tool 107
10.2.1 Using NVMD Within the IDE 107
10.2.1.1 Downloading a .nvi File 107
10.2.1.2 NVI Download Launch Configurations 108
10.2.1.2.1 Download Tab 110
10.2.1.2.2 Connection Tab 110
10.2.2 Usage 111
10.3 Non-Volatile Image Dump Tool 113

A. File Formats *115*

A.1 Introduction 115
A.2 Absolute Executable Format 115
A.2.1 Absolute File Header 115
A.2.2 TARGET Header Record 116
A.2.3 ABSREC Header and Data Records 116
A.2.4 ENTRY Header and Data Records 117
A.2.5 BSSREC Header and Data Records 117
A.2.6 ENTRYS Header and Data Records 118
A.2.7 STRTABLE Header and Data Records 119
A.2.8 SYMTABLE Header and Data Records 119
A.2.9 FILETABLE Header and Data Records 120
A.2.10 LINETABLE Header and Data Records 120
A.2.11 COMMENT Header and Data Records 121
A.2.12 LOADMAP Header and Data Records 121

A.3 Relocatable Object File Format 122
A.3.1 Object File Structure 122
A.3.2 Record Header 122
A.3.3 Name Fields 123
A.3.4 Start Record 123
A.3.5 Segment Record 124
A.3.6 Symbol Record 125
A.3.7 Text Record. 125
A.3.8 Relocation Record 126
A.3.8.1 Simple Relocations 126
A.3.8.2 Extended Relocations 127
A.3.9 Uninitialized Data Record 128
A.3.10 Comment Record 128
A.3.11 Target Record. 128
A.3.12 Debug Symbol Record 128
A.3.12.1 Debug Symbol Table Entries for Symbols 129
A.3.12.2 Debug Symbol Table Entries for Segments 130
A.3.13 File Name Record 130
A.3.14 Line Number Record 131
A.3.15 String Record. 131
A.3.16 End Record 131
A.4 Non-Volatile Memory Image Format (.nvi) 132
A.4.1 NVI File Structure 132
A.4.2 Headers 132
A.4.2.1 Main File Header 132
A.4.2.2 Generic Block Header 134
A.4.3 String Table. 134
A.4.4 VFS Node Chunks 135
A.4.5 Data Chunks 136
A.4.6 CRC Values 136

B. Configuration Template Schema 137

B.1 Introduction 137
B.2 Template Example 137

B.3 Schema 137
B.3.1 <pt:template> Root Element 137
B.3.2 <parameter> Elements 138
B.3.3 <command> Elements 140

1 Introduction

1.1 PURPOSE

This *CFX Toolchain Reference for BELASIGNA®* 300 provides reference information for building software with the Evaluation and Development Kit (EDK). It describes the preprocessing, assembling, and linking processes, and how to chain them together with the Toolchain Driver. It also describes how to create and load non-volatile file system images. Furthermore, the manual provides detailed information on file formats.

In conjunction with the SignaKlara Device Utility (SKDU), you can create and distribute an executable program where parameter values can be changed without recompiling the program. This manual explains how to create the configurable program.

The build process can be performed from a command line or through the Integrated Development Environment (IDE) as described in this introduction.

1.2 BUILDING SOFTWARE

The building process translates source code to machine code that can run directly on the chip. You can build software with the IDE or from the command line.

1.2.1 Building Software with the IDE

The build system in the IDE automatically generates XML build rules based on the contents of the project, and uses an integrated builder (based on the popular Ant build tool) to generate the output (`.abs`, `.lib`, etc.) files. The build settings are configured through a special Build Settings Editor, and the build process can be extended with customized pre- and post-build steps. The IDE also has a feature which allows for the automatic building of projects whenever a file is saved. For more information about these features, see Chapter 2, “Building Software with the IDE” on page 5.

1.2.2 Building Software from the Command Line

The software tools that the IDE uses to build code are also available from the SIGNAKLARA® Tools Command Prompt. The main tool is CFCC, the Toolchain Driver, which you can use to call most of the other tools. The building process includes the following sub-processes:

- Preprocessing
- Assembly
- Linking object modules

For an overview of how you can combine these processes to build software applications, see Chapter 3, “Running the Command-Line Tools” on page 9.

1.3 INTENDED AUDIENCE

This manual was written for readers who have some familiarity with the C language. For more information on concepts such as the C preprocessor, C-style comments, C expression operators, C-style numeric literals, and C string escape sequences, see any of the standard works on the subject, such as *The C Programming Language* by Kernighan and Ritchie.[1]

1.4 CONVENTIONS

In general, numbers are presented in decimal notation. In cases where hexadecimal notation is more convenient, these numbers are identified by the prefix "0x". For example, the decimal number 123456 can also be represented as 0x1E240.

The following special fonts are used in this manual to signify particular types of information:

`monospace font`	Commands and their options, file and path names, error messages, code samples and code snippets.
`mono bold`	A placeholder that represents where you would specify the appropriate information. For example, you would replace **`filename`** with the actual name of the file.
<angle brackets>	In Chapter 5, "Assembling Programs" on page 27, the convention is to use angle brackets (<>) to indicate placeholders. For example, you would replace `<num-expr>` with a numeric expression.
bold	Used for menu names and menu items.

1.5 MANUAL ORGANIZATION

The *CFX Toolchain Reference for BelaSigna 300* contains the following chapters and appendices.

- **Chapter 1: Introduction**, describes the purpose of this manual and how the book is organized, provides system requirements, and gives a list of suggested reading for more information.
- **Chapter 2: Building Software with the IDE**, describes the Integrated Development Environment features for building software.
- **Chapter 3: Running the Command-Line Tools**, provides a high-level overview of the software development process and how to use CFCC.
- **Chapter 4: Preprocessing Source Code**, discusses issues related to the preprocessing stage of software compilation.
- **Chapter 5: Assembling Programs**, provides an overview of assembly language programming with the EDK.
- **Chapter 6: Using the Object Librarian**, provides information on creating and manipulating libraries of object modules.
- **Chapter 7: Linking Object Modules**, discusses issues related to the linking stage of software compilation.
- **Chapter 8: Creating Configurable Programs**, describes how to develop an application that allows the user to customize it without recompiling and linking.

1. Kernighan, Brian W., and Ritchie, Dennis M. The C Programming Language. 2nd ed. Prentice Hall, 1988.

- **Chapter 9: Non-Volatile Memory Management**, explains how to use the Non-Volatile Memory Manager (NVMM) to create loadable, non-volatile images for BelaSigna 300.
- **Chapter 10: Non-Volatile Image Tools**, describes loading non-volatile image files to the BelaSigna 300 non-volatile memory.
- **Appendix A: File Formats**, discusses the file formats used by the assembler toolchain, and the file format for storing non-volatile memory images once the layout has been processed.
- **Appendix B: Configuration Template Schema**, explains the format for the file that specifies the parameter values which can be substituted into a configurable program.

1.6 FURTHER READING

For more information, refer to the following documents:

- *Integrated Development Environment User's Guide*
- *Hardware Reference Manual for BelaSigna 300*
- *CFX DSP Architecture Manual*
- *HEAR Configurable Accelerator Reference Manual*
- *Evaluation and Development Board Manual for BelaSigna 300*
- *Firmware Reference Manual for BelaSigna 300*
- *SignaKlara Device Utility User's Guide*
- *Eclipse Workbench User Guide*
- Kernighan, Brian W., and Ritchie, Dennis M. *The C Programming Language*. 2nd ed. (Prentice Hall, 1988)
- Ant documentation: http://ant.apache.org/manual/
- The Jython Web site: http://www.jython.org/
- The Python Web site: http://python.org/
- Lutz, Mark, and Ascher, David. *Learning Python*. 2nd ed. (O'Reilly, 2003)
- Lutz, Mark. *Programming Python*. 2nd ed. (O'Reilly, 2001)
- Pedroni, Samuele, and Rappin, Noel. *Jython Essentials*. 1st ed. (O'Reilly, 2002)

This page left blank for printing purposes.

2 Building Software with the IDE

2.1 INTRODUCTION

The following sections describe how to build software in the Integrated Development Environment (IDE). Using this interface, you can configure the build settings for your project, add your own custom build steps, and even have the build occur automatically when a source file changes.

The project build system is responsible for building output files from the source code in your project. Depending on the type of project, this output will be an executable, library, or EEPROM image file.

For more detailed information about general build system concepts in the IDE, see the *Integrated Development Environment User's Guide*.

2.2 BUILD STAGES

For most projects the automatic build process is divided into two stages: assembling the source files into object files, and combining the object files to produce the final project output.

The build system uses the HCT command-line tool to transform HCT source files into object files and libraries. For more information on the HCT refer to the *HEAR Configurable Accelerator Reference Manual*.

The build system uses the assembler stage of the CFCC command-line tool to assemble source files into object files. The CFCC assembler stage is described in Chapter 5, "Assembling Programs" on page 27.

For executable projects, the build system uses the linker stage of the CFCC command-line tool to combine the object files into an executable. The CFCC linker stage is described in Chapter 7, "Linking Object Modules" on page 57.

For library projects, the build system uses the `CFLIB` librarian tool to combine the object files into a library. The CFLIB tool is described in Chapter 6, "Using the Object Librarian" on page 55.

To construct EEPROM images, the build system uses the NVMM in the IDE to create loadable non-volatile images for the BelaSigna 300 EEPROM. The NVMM is described in Chapter 9, "Non-Volatile Memory Management" on page 83. Tools for working with EEPROM images are described in Chapter 10, "Non-Volatile Image Tools" on page 107.

2.3 BUILD SETTINGS

The following procedure assumes that you have opened the IDE and created a project containing source code.

You can access your project's build settings by performing any one of the following operations in the Project Explorer View:

- Right-click your project and select **Edit Build Settings...**.
- Right-click the **Build Settings** element in your project and select **Edit Build Settings...**.
- Right-click an existing build configuration and choose **Edit Configuration...**.

If you are using the Navigator View:

- Open the `build.prefs` file found in the `.settings` folder of your project. Double-clicking the file opens the Build Settings Editor.
- If a simple text editor appears, it is possible that you previously instructed Eclipse to open the `build.prefs` file with the default text editor. If this is the case, right-click the `build.prefs` file and select **Open With > Build Settings Editor**. This re-associates the file with the default Build Settings Editor.

This opens the Build Settings Editor as shown in Figure 1.

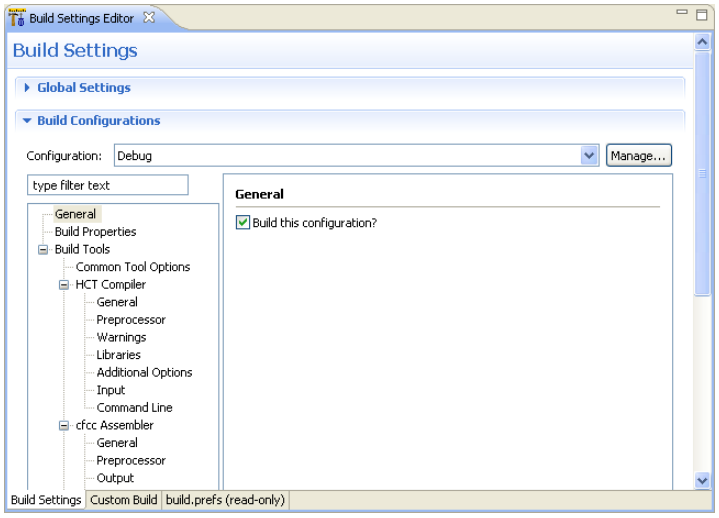


Figure 1: The Build Settings Editor

You can use the Build Settings Editor to configure global build properties and one or more build configurations containing the build settings for your project. For detailed information on how to set up and configure your project's build configurations, refer to the *Integrated Development Environment User's Guide*.

The build settings control the behavior of the various build stages within each build configuration. In addition to general build settings, you can configure specific build settings for the following tools:

HCT Compiler

These settings are passed to the HCT when processing HCT source files into object files. For more information about the command line options for the HCT refer to the *HEAR Configurable Accelerator Reference Manual*.

HCT Librarian

These settings are passed to the HCT when processing HCT source files into HCT libraries. For more information about the command line options for the HCT refer to the *HEAR Configurable Accelerator Reference Manual*.

cfcc Assembler

These settings are passed to CFCC when assembling source files into object files and are configured on the various sub-category pages of the cfcc Assembler tool category. The builder provides access to CFCC's `-I` option (the preferred way of specifying an include path), but also uses the environment in which the IDE is launched. If you want to define a value for the CFINCLUDE environment variable, do so before launching the IDE. For more information about this variable, see Section 4.2.2, “CFINCLUDE Environment Variable” on page 17.

cfcc Linker

This page shows the settings that are passed to CFCC when linking object files into executables. The builder provides access to CFCC's `-L` option (the preferred way of specifying a library path), but also uses the environment in which the IDE is launched. If you want to define a value for the CFLIB environment variable, do so before launching the IDE. For more information about this variable, see Section 7.2.2, “CFLIB Environment Variable” on page 60.

cflib Librarian

This page shows the settings that are passed to CFLIB when combining object files into libraries.

NVMM

These settings are passed to NVMM when constructing SK3 EEPROM images (`.nvi` files). For more information about configuring build settings specific to NVMM refer to Section 9.3.1.3, “Editing Build Settings” on page 86.

The Build Settings Editor provides a convenient graphical interface to most of the build options for the aforementioned tools. For more information on creating and editing build configurations refer to the *Integrated Development Environment User's Guide*. For more information about the command-line options for `CFCC` and `CFLIB`, see:

- Chapter 3, “Running the Command-Line Tools” on page 9
- Chapter 4, “Preprocessing Source Code” on page 15
- Chapter 5, “Assembling Programs” on page 27
- Chapter 6, “Using the Object Librarian” on page 55
- Chapter 7, “Linking Object Modules” on page 57

2.4 AUTO-BUILD

By default, the IDE Workbench automatically rebuilds a project when one of the following occurs:

- The project is created
- A file included in the build (or one of its dependencies) is changed or deleted
- A valid input file is created or added to the project
- The project is cleaned by selecting **Project** > **Clean...**
- A dependent project is built

As part of this project rebuild, the build system performs dependency calculations to determine what needs to be rebuilt, and what is already up-to-date. This allows the build system to rebuild only those files that have changed since the last build. You can continue to edit your source code while the build is running, and any build errors are displayed in the editors and the Project Explorer, Console and Problems views.

It is sometimes helpful to turn off this auto-build feature, especially when making a large number of changes where you know that the source code will not build correctly until you are done. Select **Project** > **Build Automatically** to enable or disable the auto-build feature.

3 Running the Command-Line Tools

3.1 INTRODUCTION

This chapter provides a description of CFCC and its global options. Detailed information regarding specific stages of compilation can be found in later chapters in this manual.

CFCC is used by the builder in the IDE (see Chapter 2, "Building Software with the IDE" on page 5 for details). The default build settings should suffice in most cases, but if you need to modify your project defaults or use options that are not available through the builder preferences, read this chapter.

3.2 THE STAGES OF COMPILATION

CFCC performs three stages of processing when transforming source code into executable machine code:

- Preprocessing
- Assembling
- Linking

CFCC accomplishes this processing by utilizing several lower level tools, and for this reason is often referred to as a *toolchain driver*.

3.2.1 Preprocessing

CFCC preprocesses source files using a standard C preprocessor.[2] Several extensions to the set of directives are also available. Preprocessing is discussed further in Chapter 4, "Preprocessing Source Code" on page 15.

3.2.2 Assembling

The assembler converts preprocessed source code (consisting of instructions, labels, and assembly directives) into binary form, stored in relocatable object files suitable for linking. The assembly language and build options are discussed in Chapter 5, "Assembling Programs" on page 27.

3.2.3 Linking

The relocatable object code produced in the assembling stage is converted into executable absolute code in the linking stage and is written to an executable output file. Collections of previously assembled relocatable object files (called *libraries*) can be created with CFLIB and then linked in this linking step. Linking is discussed further in Chapter 7, "Linking Object Modules" on page 57.

2. ISO 9899-1990 Standard for the C language (also known as C90).

3.3 CFCC USAGE

Which of the three stages of compilation (preprocessing, assembling, and linking) CFCC performs on an input file depends on its filename extension. Table 1 identifies accepted file extensions and the stages of processing performed.

TABLE 1: ACTIONS ON FILE EXTENSIONS

File Extension	Description
`.cfs`	Assembly language source file. CFCC performs preprocessing, assembling, and linking.
`.cft`	Preprocessed assembly language source file. CFCC performs assembling and linking.
`.obj, .lib`	Relocatable object code files and libraries. CFCC performs linking.

Linking is always the last stage unless you use one of the `-c` or `-E` options to avoid it (or unless compilation errors stop the process). For the link stage, all `.obj` files corresponding to source files and `-l` libraries are passed to the linker. The output of linking has a default extension of `.abs`.

3.3.1 Syntax

CFCC accepts the following syntax:

`cfcc` **`options-and-files`**

Where **`options-and-files`** indicates one or more of the CFCC, preprocessing, assembling, or linking options, including input filenames. You can find CFCC options in Table 2 on page 11. Options for the other stages appear in the chapters of this manual devoted to those stages.

Some options can only be bound to one value, such as `-b` to specify a target machine, and `-o` to specify an output file name. In the event that one of these options is specified several times, CFCC always uses the last option specified.

Some examples of CFCC usage:

- To assemble and link the executable foo.abs from foo.cfs:

```
cfcc --strictnop -g -b SK3 foo.cfs
```

- To assemble foo.cfs to foo.obj:

```
cfcc --strictnop -g -c -b SK3 foo.cfs
```

- To preprocess foo.cfs (prints to standard output):

```
cfcc -b SK3 -E foo.cfs
```

- To build app.abs from foo.cfs and bar.cfs:

```
cfcc --strictnop -g -o app.abs -b SK3 foo.cfs bar.cfs
```

- To generate a listing in foo.lst:

```
cfcc --strictnop -g -c --lst -b SK3 foo.cfs
```

The `--strictnop` and `-g` options, for strict CFX instruction matching and debugging information respectively, are recommended for all assembly commands.

3.3.2 CFCC - Toolchain Driver Options

Table 2 describes some of the options you can use to specify the general behavior of CFCC. The following chapters contain specific options for each stage of compilation.

TABLE 2: CFCC OPTIONS

CFCC Option	Description
`filename`	The file or files to be preprocessed, assembled, and linked. The file extension determines which actions are to be performed (see Table 1 on page 10). At least one file name must be specified for CFCC to process.
`-c`	Assemble the source files, but do not link. The compiler outputs an object file for each source file. By default, CFCC makes the object file name for a source file by replacing suffixes such as `.cft`, `.cfs` with `.obj`. You can override this default using the `-o` option. This option also causes CFCC to ignore any unrecognized input files (i.e., files that do not require assembly).
`-E`	Stop after the preprocessing stage; do not run the assembler. The output is preprocessed source code, which is sent to standard output by default. CFCC ignores input files that do not require preprocessing.
`-o` **`file`**	Place output in file **`file`**. This action occurs regardless of the type of output CFCC is producing, whether executable file, object file, or preprocessed source code. Since only one output file can be specified, it does not make sense to use `-o` when compiling more than one input file, unless you are producing an executable as output. If you do not specify `-o`, the default is to put an executable file in `a.abs` (**`source`**`.abs` when compiling a single file, where **`source`** is the file name), the object file for **`source`**`.cfs` in **`source`**`.obj`, and all preprocessed source to standard output.
`-v`	Verbose mode. Prints (on standard error output) the commands executed to run the stages of compilation and runs all stages in verbose mode.
`-w`	Do not generate any warning messages.
`-g`	Produce debugging information.
`-b` **`machine`**	The argument **`machine`** specifies the target machine for compilation. Currently, its value can only be `SK3`. CFCC defines the preprocessor symbol `_TARGET_`**`machine`** to indicate its target.
`-h`	Print a usage message.

TABLE 2: CFCC OPTIONS (CONTINUED)

CFCC Option	Description
`-n`	Dry run. Performs no processing on input files. This is most useful in conjunction with the `-v` option.
`-x` **`file`**	Read options from **`file`**.
`--ansi`	Force strict adherence to the C standard. This option causes the preprocessor to suppress support for `#macro`, `#endm`, `#rept`, `#endr`, `#set`, `#stripc`, `#stripasm`, `#nostrip`, `#nostripc`, and `#warning`. It also turns off support for C++-style comments.
`--version`	Outputs the name and version number of CFCC.

3.4 ERROR CODES

If CFCC runs successfully, it returns an error code of 0. If it fails because of an invalid option, it returns 2. If compilation fails due to input errors, it returns the non-zero code returned by the underlying tool (i.e. CFPP, CFASM, CFBACK, CFPACK, CFASR, CFLINK or MEMCHECK). If it fails for any other reason, it returns an error code of 1.

3.5 ERROR MESSAGES

This section lists and describes error messages you may encounter using the Toolchain Driver.

Note: This catalog of error messages is not comprehensive. It lists the error messages of CFCC proper, but not the messages generated by its underlying tools.

3.5.1 “Option error(option): message”

Printed when there is a problem reading the options from the command line. The `option` is the option in question. The `message` is a message describing the problem. This message is always followed by a usage message.

Example:

```
>cfcc --junk foo.cfs
Option error(junk): option --junk not recognized
Usage: cfcc [options] file...
Options:
    -h           Display this information.
    (Use '-v -h' to display command line options of sub-processes)
    -c           Assemble, but do not link
    -E           Preprocess only; do not assemble or link
    -o <file>    Place the output into <file>
    -v           Verbose output
    -w           Suppress warning messages
    -g           Produce debugging information
    -b <machine> Target architecture (SK3 only)
```

```
-n           Dry run, do not execute (most useful with -v)
-x <file>    Read options from <file>
--ansi       Force strict adherence to the C standard
--version    Display version information
```

3.5.2 "IO Error: message"

Printed when there is a problem reading options from an indirect file.

Example:

```
>cfcc -x foo.ind foo.cfs
IO Error: [Errno 2] No such file or directory: 'foo.ind'
```

3.5.3 "No input files"

Printed when no files are specified.

Example:

```
>cfcc
No input files
```

3.5.4 "Unknown target specified in -b: target"

Printed when an invalid target architecture is specified.

Example:

```
>cfcc -b FROB5000 foo.cfs
Unknown target specified in -b: FROB5000
Only SK3 is valid.
```

3.5.5 "Cannot specify -o with -c and multiple compilations"

Printed when `-o` is specified with `-c` and multiple source files, as `-c` will generate an object file for each source file and `-o` specifies the name for a single output file. A similar error occurs when `-o` is specified with `-E` and multiple source files, as each source is preprocessed separately.

Example:

```
>cfcc -c -o foo.obj foo.cfs bar.cfs bat.cfs
Cannot specify -o with -c and multiple compilations
```

3.5.6 "Error: argument is not a file"

Printed when a specified argument is not a file.

Example:

```
>cfcc non-existent-file.cfs
Error: non-existent-file.cfs is not a file
```

3.5.7 “Don’t know how to build file: filename”

Printed when an argument has an unrecognized extension.

Example:

```
>cfcc foo.xxx
Don't know how to build file: foo.xxx
```

3.5.8 “System exception when running tool”

Printed if an underlying tool (CFPP, CFASM, CFBACK, CFPACK, CFASR, CFLINK or MEMCHECK) signals a system error.

3.5.9 “Error running tool: message”

Printed if there is a problem running an underlying tool.

Example (generated by renaming `cfpp.exe` to `pp.exe`):

```
>cfcc foo.cfs
Error running cfpp.exe: No such file or directory
```

3.5.10 “Unexpected error running tool”

Printed when some other error occurred when running an underlying tool. Users should never see this message.

3.5.11 “Error moving filename to destination: message”

Printed when there is an OS-level problem moving a generated file to a new location.

3.5.12 “Error copying filename to destination: message”

Printed when there is an OS-level problem copying a source file to the temporary working directory.

3.5.13 “An unexpected error has occurred in tool. Please contact ON Semiconductor customer support”

Printed when some unforeseen error occurs and one of the underlying tools crashes.

4 Preprocessing Source Code

4.1 INTRODUCTION

Assembly language programs are often made more readable and maintainable through the use of macro facilities. In the CFX assembly language, these are provided through the use of a standard C preprocessor that is run prior to the assembler itself. In addition to the usual preprocessing constructs, the C preprocessor from ON Semiconductor implements extensions that are especially useful for assembly language programming.

The preprocessor filters the assembly language program provided as input, executing preprocessing directive lines and performing macro replacement on other input lines as they are passed through. In this way, the preprocessor is used to access hardware and system facilities provided by ON Semiconductor with the EDK, to define application-specific constants, to abbreviate repetitive or complicated constructs, and to specify conditional compilation.

This chapter describes how to run the preprocessor using CFCC, and provides detailed documentation of the preprocessing language implemented by the ON Semiconductor preprocessor. Particular focus is placed on the facilities that are extensions to the C standard, and details that are implementation-specific such as the location of `#include` files.

4.2 RUNNING THE PREPROCESSOR

This section describes the CFCC preprocessing options and the CFINCLUDE environment variable. For information on CFCC syntax, see Table 2 on page 11.

4.2.1 Preprocessing Options

CFCC runs the preprocessor on input files that have a `.cfs` extension. Your use of CFCC options (see Table 2 on page 11) affects the behavior of the preprocessor, specifically those listed in Table 3.

TABLE 3: CFCC OPTIONS AFFECTING PREPROCESSOR BEHAVIOR

CFCC Option	Effect on the Preprocessor
`-E`	Stop after the preprocessing stage; do not run the assembler. The output is preprocessed source code, which is sent to standard output by default. CFCC ignores input files that do not require preprocessing.
`-o` **`file`**	Place output in file **`file`**. This action occurs regardless of the type of output CFCC is producing, whether executable file, object file, or preprocessed source code. Since only one output file can be specified, it does not make sense to use `-o` when compiling more than one input file, unless you are producing an executable as output. If you do not specify `-o`, the default is to put an executable file in `a.abs` (**`source`**`.abs` when compiling a single file, where **`source`** is the file name), the object file for **`source`**`.cfs` in **`source`**`.obj`, and all preprocessed source to standard output.
`-b` **`machine`**	The argument **`machine`** specifies the target machine for compilation. Currently, its value can only be `SK3`. CFCC defines the preprocessor symbol `_TARGET_`**`machine`** to indicate its target.
`--ansi`	Force strict adherence to the C standard. This option causes the preprocessor to suppress support for `#macro`, `#endm`, `#rept`, `#endr`, `#set`, `#stripc`, `#stripasm`, `#nostrip`, `#nostripc`, and `#warning`. It also turns off support for C++-style comments.

Options to CFCC that are specific to the preprocessing stage are described in Table 4.

TABLE 4: PREPROCESSING OPTIONS

CFCC Option	Description
`-C`	Preserves C-style (and C++-style) comments in the output.
`-t`	Refrains from outputting control lines which include the source line and file name.
`-S`**`name`**	Defines the identifier **`name`** as if you had included `#set name 1` at the beginning of the input file.
`-D`**`name`**	Defines the identifier **`name`** as if you had included `#define name 1` at the beginning of the input file.
`-D`**`name=str`**	Defines the identifier **`name`** as if you had included `#define name str` at the beginning of the input file.
`-U`**`name`**	Undefines the identifier **`name`** as if you had included `#undef name` at the beginning of the input file.
`-I`**`dir`**	Searches directory **`dir`** for included files if necessary.

TABLE 4: PREPROCESSING OPTIONS (CONTINUED)

CFCC Option	Description
`--ts`	Uses blanks to force explicit preprocessor token separation on output. This behavior allows other tools to implement standard C tokenization, but it is not the default since it is incompatible with the assembly language syntax.
`-Wundef`	Warns about undefined symbols used in `#if`, `#elif`, `#set`, and `#rept` directives. By default, the preprocessor substitutes the value 0 for symbols that are not defined when it is evaluating the expressions for these directives.

4.2.2 CFINCLUDE Environment Variable

To find files included with the `#include` directive, CFCC looks for the `CFINCLUDE` environment variable. `CFINCLUDE` contains the name of the directory to search for included files, after the directories given with `-I` options are searched.

You can provide more than one include file directory in the `CFINCLUDE` environment variable. Use a semicolon (`;`) as the delimiter to separate the names. The convention here is the same as that used for the `PATH` environment variable.

For backwards compatibility, if `CFINCLUDE` is not defined in the environment, CFCC looks for the `MCINCLUDE` environment variable, and uses it in the same way as described above for `CFINCLUDE`. If `MCINCLUDE` is not defined, CFCC looks for the `INCLUDE` environment variable, and uses it instead.

The procedure used to find included files is described in Section 4.3.5.1, “Details of #include File Searching” on page 22.

Note: You can set this environment variable when you start the **EDK SignaKlara Command Prompt** for this EDK version. To view this and other settings, type `set` at the **EDK SignaKlara Command Prompt**.

4.2.3 Predefined Macros

CFCC provides three predefined macros, in addition to those that are provided by any standard C preprocessor. They are:

__cf__

This macro is predefined to the decimal value 1. It is built into the preprocessor, so it should not be undefined or redefined.

TARGET<machine>

As described in Table 3 on page 16, the use of CFCC’s `-b` option determines the specific name of the macro that is predefined. This macro is defined by CFCC using a built-in `-D` option to the preprocessor, so it can be undefined using the `-U` option, if desired.

_CFXASM_VER

This macro is predefined to a value which is a numeric representation of the version of the CFX assembler. The value is an integer equal to

major-version * 100 + *minor-version* * 10 + *revision*, so for version 3.0.0 of the CFX assembler the value of `_CFXASM_VER` is 300. When the old CFX assembler (see Section 5.1, “Introduction” on page 27) is invoked using the `--cfpack` option, the value of `_CFXASM_VER` is less than 300.

This macro is defined by CFCC using a built-in `-D` option to the preprocessor, so it can be undefined using the `-U` option, if desired.

4.3 ON SEMICONDUCTOR PREPROCESSING LANGUAGE

CFCC’s preprocessor provides the standard C preprocessing facilities, plus extensions that enhance its effectiveness as the macro language for CFX assembly language programming.

The following sections describe the enhanced C preprocessing language from ON Semiconductor, with emphasis on detailed descriptions of the facilities that are extensions to the C standard.

4.3.1 Preprocessing Directive Summary

Preprocessor actions are initiated with directives that begin with the hash character ‘#’ as the first non-blank character in a line. (Note that comments are considered blanks.) Table 5 summarizes the directives that are supported by the preprocessor. The directives at the end of the table are extensions to standard C.

TABLE 5: PREPROCESSOR DIRECTIVES

Directive	Description	Extension to C
`#include` **`header-file`**	Process the header file as part of the input.	no
`#define` **`name text`**	Define a one-line macro without arguments.	no
`#define` **`name(args) text`**	Define a one-line macro with arguments.	no
`#undef` **`name`**	Remove the definition of a macro.	no
`#if` **`expression`**	Process following code if **`expression`** is non-zero.	no
`#ifdef` **`name`**	Process following code if **`name`** is defined.	no
`#ifndef` **`name`**	Process following code if **`name`** is not defined.	no
`#elif` **`expression`**	Process `else if` code if previous condition was zero and **`expression`** is non-zero.	no
`#else`	Process `else` code if previous condition was zero.	no
`#endif`	End a conditional compilation section.	no
`#error` **`text`**	Display diagnostic message with **`text`**, and exit.	no
`#line` **`number "file"`**	Set preprocessor’s current line number and file, into `__LINE__` and `__FILE__`.	no
`#pragma` **`text`**	Do nothing. The preprocessor implements no `#pragma` directives, so **`text`** is ignored.	no
`#`	Do nothing. The null directive is ignored.	no
`#macro` **`name`**	Define a multi-line macro without arguments.	yes
`#macro` **`name(args)`**	Define a multi-line macro with arguments.	yes

TABLE 5: PREPROCESSOR DIRECTIVES (CONTINUED)

Directive	Description	Extension to C
`#endm`	End a multi-line macro.	yes
`#set` **`name expression`**	Define a numeric macro.	yes
`#rept` **`expression`**	Start a block that will repeat **`expression`** times.	yes
`#endr`	End a repeated block.	yes
`#warning` **`text`**	Display diagnostic message with **`text`**.	yes
`#stripasm`	Remove assembly-style comments (beginning with semicolon (`;`)). Do not use with the new CFX assembler.	yes
`#nostrip`	Do not remove assembler comments (default).	yes
`#stripc`	Remove C and C++ comments (default).	yes
`#nostripc`	Do not remove C and C++ comments.	yes

4.3.2 Preprocessing Procedure

This section provides a brief, high-level overview of the procedure used by a C preprocessor (including CFCC's extended C preprocessor) for processing an input file.

Preprocessing is divided into the following conceptual phases:

Line-folding

Each instance of a backslash character `\`, without any other characters following it on the source file line, is deleted and the following line is joined to the line containing the backslash character to form a logical source line.

Tokenizing and comment removal

The logical lines of the source file are decomposed into tokens and white-space characters (which includes spaces, tabs, form feed, and comments; the latter are replaced with a single space character). The tokens are standalone lexical units such as names, punctuation characters, operators, numbers, and strings.

Directive execution and macro replacement

Logical lines are processed in order. Each line that is a preprocessing directive is executed.

A `#include` directive causes the specified header file to be preprocessed from phases 1 to 3, recursively.

`#if`, `#ifdef`, and `#ifndef` directives introduce a conditional compilation block, wherein at most one clause is true and therefore fully executed/scanned as described here; the other clauses, until the corresponding `#endif` directive, are scanned only to find corresponding `#elif`, `#else`, or `#endif` directives, or nested conditional compilation blocks.

Non-directive lines are scanned for macros that have been defined, and these are replaced with the text corresponding to the macro's value. After this replacement, the new text is re-scanned for further macros.

These preprocessing phases occur in order. Each preprocessor directive must occur on a single logical line, but line-folding allows this logical line to correspond to multiple physical lines (if the definition of the macro is long, for example). Similarly, since comment removal precedes directive execution, you can intersperse comments within directives, if necessary.

4.3.3 Lexical Extensions

The preprocessor from ON Semiconductor provides some extensions to the C standard at the lexical level. These improve the flexibility of the preprocessor for use as the CFX assembly language macro processor.

4.3.3.1 Dollar in Names

The dollar (`$`) character is allowed in any position (including the first) of a name token. Thus, you can use `$` in the same way as the letters and `_` in macro names. This usage is compatible with the assembler, which also treats `$` as an alphabetic character in names. See Section 5.3.1, "Lexical Conventions" on page 32.

4.3.3.2 Comment Removal

By default, the preprocessor removes C-style and C++-style comments (delimited with `/*` and `*/`, and `//` and newline, respectively) when translating the input to the output. If CFCC's `--ansi` option is used, C++-style comments are not removed or recognized, and the `#stripc` and `#nostripc` directives (see below) affect the handling of C-style comments only.

If the `#nostripc` directive is used, the preprocessor does not remove C-style and C++-style comments when translating the input to the output. These comments are still recognized—their contents do not have macro replacement performed on them and they are not treated as tokens (which could cause syntax errors for comments in directives)—but they are copied through from the input to the output. In some cases they are not treated as space characters (as preprocessing phase 2 requires; see Section 4.3.2, "Preprocessing Procedure" on page 19), so the behavior of the preprocessor can change when comments are interspersed with directives. It is recommended that comments not be interspersed with directives when `#nostripc` is in effect.

The `#stripc` directive causes the preprocessor to remove C-style and C++-style comments, restoring the default behavior.

The `-C` preprocessor option is equivalent to using the `#nostripc` directive at the start of the input file.

If the `#stripasm` directive is used, the preprocessor recognizes assembly-style comments beginning with the semicolon (`;`) and ending with the next newline character. These comments are removed. The `#nostrip` directive restores the default behavior: semicolon comments are not recognized, nor removed. They are treated as tokens and macro replacement occurs in them. Do not use the `#stripasm` directive with the new CFX assembler (see Section 5.1, "Introduction" on page 27) because its syntax uses the semicolon for separating instructions (when there are two or more on a line) so removing everything following a semicolon might remove some of the program.

4.3.3.3 Escaping the Hash Character

In some uses of the preprocessor, it is desirable for the output file to contain hash (#) characters. For example, you might use the preprocessor to automatically generate a source code file from a template, with the output source code file containing preprocessor directives.

This usage is supported by the preprocessor. Hash characters can be "escaped" by preceding them with the backslash (\) character. When preceded in this way, the hash character is not interpreted as the directive lead-in character, so it is passed through from the input to the output.

4.3.4 Preprocessor Expressions

The preprocessor evaluates numeric expressions that are the operands of the `#if`, `#elif`, `#set`, and `#rept` directives. These expressions involve integer constants (which may be generated by macro replacement), operators, and parentheses for grouping. The operators available are the usual C expression operators, as shown in Table 6.

TABLE 6: OPERATORS, BY GROUP

Operator	Description
`defined name` `defined(name)`	Unary "is-macro" predicate
`+ - ~ !`	Unary operators
`* / %`	Multiplicative operators
`+ -`	Additive operators (binary)
`<< >>`	Shift operators
`< > <= >=`	Relational operators
`== !=`	Equality operators
`&`	Bitwise AND operator
`^`	Bitwise exclusive OR operator
`|`	Bitwise inclusive OR operator
`&&`	Logical AND operator
`||`	Logical OR operator
`? :`	Conditional operator (ternary)

Expressions are evaluated using C precedence rules, which are indicated by the rows in Table 6. Operators with higher precedence are at the top of the table, and operators with equal precedence appear in the same row.

The `defined` predicate returns 1 if the name has been defined as a macro using `#define`, `#macro`, or `#set`, without an intervening `#undef`. Otherwise, it returns 0.

Arithmetic operations use 64 bits; signed numbers are represented using two's complement. The right-shift operator (`>>`) is "arithmetic" when applied to a signed, negative value. That is, high-order bit positions are filled with copies of the sign bit. The division operator (`/`) and remainder operator (`%`) perform integer division that truncates towards zero, for negative and positive operands.

4.3.5 Directive Extensions

The following sections describe how this particular preprocessor implements the directives and any non-standard directives.

4.3.5.1 Details of #include File Searching

There are two forms of the #include directive:

```
#include <filename>
#include "filename"
```

These forms may also be generated through macro replacement:

```
#if WIDGET_PRODUCT
    #define PRODUCT_INCLUDE "widget.cfi"
#else
    #define PRODUCT_INCLUDE <standard_product.cfi>
#endif

#include PRODUCT_INCLUDE
```

The preprocessor uses the following method to search for **filename**:

- If **filename** specifies an absolute path, then the preprocessor includes the specified file, and the search is done. An absolute path is one that begins with a drive specification such as C:, a backslash (\), or a forward slash (/).
- Otherwise, **filename** specifies no path or a relative path. If **filename** is surrounded by double quotes, the preprocessor appends **filename** to the directory path of the file that contains the #include directive, and attempts to open the resulting file. If the file can be opened, then it is included, and the search is complete.
- Otherwise, **filename** cannot be found relative to the directory of the file containing the #include, or **filename** is surrounded by angle brackets. The preprocessor then appends **filename** to each specified include directory, in order, and attempts to open the resulting file. If the file can be opened, then it is included and the search is complete; otherwise the preprocessor tries the next directory in the list of include directories.

  The list of include directories is formed from the -I options on the command line (in order from left to right) followed by the directories given in the CFINCLUDE environment variable, from left to right. See Section 4.2.2, “CFINCLUDE Environment Variable” on page 17 for information about setting the CFINCLUDE environment variable.

4.3.5.2 Comment Removal

The preprocessor provides the #stripasm, #nostrip, #stripc, and #nostripc directives to control the removal of semicolon comments and C/C++-style comments. Full details regarding these four directives may be found in Section 4.3.3, “Lexical Extensions” on page 20.

4.3.5.3 Generating a Warning with #warning

The #warning directive produces a similar diagnostic message to that produced by the #error directive. However, the preprocessor continues processing the input and generating output; it does not terminate due to the #warning directive, as it does with the #error directive.

4.3.5.4 Repeating Text Blocks Using #rept

The `#rept` and `#endr` directives enclose a block of text lines that are repeated in the output. The number of repetitions is given by the **`expression`** operand to `#rept`.

The repeat count expression is evaluated as described in Section 4.3.4, "Preprocessor Expressions" on page 21. The result is truncated to 16 bits (the high 48 bits are simply ignored) and treated as a signed 16-bit number. This number is the repeat count; it must be greater than or equal to zero.

The lines between `#rept` and `#endr` are only processed through phases 1 and 2 of the preprocessing procedure (see Section 4.3.2, "Preprocessing Procedure" on page 19) as they are collected. Preprocessing phase 3 is then performed on the lines each time they are repeated in the output. This means that execution of preprocessing directives and macro replacement are performed on each line, each time it is output.

The `#set` directive is particularly useful within a repeated section, as a loop index for example.

4.3.5.5 Defining Multi-Line Macros with #macro

The `#macro` and `#endm` directives are used to define multi-line macros.

The `#macro` directive follows the same rules as the `#define` directive for specifying arguments: multi-line macros may be defined with no arguments (and no parentheses for an argument list), or with empty parentheses (which defines a function-like macro with no arguments), or with one or more argument names in parentheses.

Unlike `#define`, however, the replacement text for a multi-line macro does not appear on the same line as the directive. Instead, the lines between `#macro` and `#endm` are the replacement text. More precisely, the replacement text begins at the start of the line following the `#macro` directive, and ends immediately before the newline at the end of the last line before the `#endm` directive.

The lines between `#macro` and `#endm` are only processed through phases 1 and 2 of the preprocessing procedure (see Section 4.3.2, "Preprocessing Procedure" on page 19) as they are collected. Preprocessing phase 3 is performed on the lines each time the macro is invoked. In other words, an invocation of the macro is replaced by its replacement text (also known as the macro's "body"). This means that execution of preprocessing directives and macro replacement are performed on each line of the macro, each time it is invoked.

Like single-line macros defined with `#define`, multi-line macros may not be redefined unless the new definition is preceded by `#undef` to remove the existing definition.

4.3.5.6 Defining Numeric Macros with #set

The `#set` directive defines a numeric macro. The **`expression`** operand to `#set` must be a correctly formed expression, since `#set` evaluates the **`expression`** as described in Section 4.3.4, "Preprocessor Expressions" on page 21 and stores the result as the replacement value for the numeric macro.

When a numeric macro is invoked, the replacement text for the macro is the replacement value set in the most recent `#set` directive for the macro, printed in signed decimal.

You can redefine numeric macros with a new expression, or redefine them as single-line or multi-line macros, without any warning occurring.

4.3.5.7 Comparison of Macro Types

The ON Semiconductor preprocessing language provides three different methods with which to define a symbolic name for some sort of value:

- The standard `#define` directive
- The extension directive `#macro`
- The extension directive `#set`

This section compares these three macro-defining directives, presenting the pros, cons, and intended usages of each.

The `#macro` directive is quite similar to `#define`, in that it defines a macro with a textual replacement, optionally including arguments that are supplied at the time the macro is invoked and replaced.

The obvious difference between `#macro` and `#define` is that `#macro` allows multiple lines in the replacement text. Since the CFX assembly language is fairly line-oriented (unlike most high-level programming languages like C, in which spacing and line distinctions are matters of style, not necessity), and preprocessor directives themselves are line-oriented, multi-line macros are necessary for typical assembly language macro tasks where a macro often contains many instructions.

The second key distinction between `#macro` and `#define` is that preprocessor directives can be embedded in a multi-line macro defined with `#macro`, and these directives are executed at the time that the macro is invoked and replaced. You can use the conditional compilation directives to test for features, or particular arguments or combinations of arguments (which may require special code sequences, or may be erroneous, for example). You can use the `#define` and `#set` directives to build up values that are used later in the macro body, such as in an immediate load instruction. They are also helpful for creating unique labels for each invocation of a macro. Here is an example of using `#set` to increment a counter that is used to generate unique label names:

```
#macro Sys_SPI_WaitDone

    // Generate unique loop label
    #ifndef LABEL_NAME
        #set COUNTER 0
    #endif
    #set COUNTER COUNTER+1
    #undef LABEL_NAME
    #define LABEL_NAME paste( _SPI_WaitDoneLoop, COUNTER )

    add sp, 1           >>> store x[sp - 1], x1    // Store X1 on stack
    LABEL_NAME:
        load x1, p[D_SPI_CTRL]
        bittst x1, SPI_CTRL_BUSY_POS ||| nop
    goto.0d LABEL_NAME, carry
    load x1, x[sp - 1]  >>> add sp, -1            // Restore X1 from stack

#endm
```

Note: Some of the names in the example above were modified to fit on the page. To see the full code, look at `sk3_sys_spi.cfi`.

Due to their multi-line nature, however, macros created with `#macro` are most often "procedure-like" macros, in that they expand to a number of instructions, and are typically invoked on their own in source code, as in:

```
SendByteOutUART(0)
```

Single-line macros created with `#define` can often be invoked in a "function-like" manner. For example:

```
#define POW2(n)  ( 1 << (n) )

load a0.h, POW2(12)
```

In fact, since a multi-line macro containing only a single line of text will contain no newline characters (see the precise definition of the body of a macro created with `#macro`, in Section 4.3.5.5, "Defining Multi-Line Macros with #macro" on page 23), it is actually possible to create the `POW2()` macro with either `#define` or `#macro`. However, if the body of a multi-line macro is more than one line, because it uses multiple calculation steps, or `#rept` blocks, or conditional compilation with `#if`, then it is not possible to invoke the macro in a function-like manner. You can work around this limitation by defining a procedure-like macro that generates a `#define` or `#set` directive with the desired result value; you can then use the macro created with the `#define` or `#set` directive as an operand in an instruction, for example.

Numeric macros created with `#set` are useful for computation. The macros created by `#define` and `#macro` are purely textual. They do not evaluate their replacement text even if it has the form of a numeric expression. Thus it is difficult or impossible to compute a value that has an iterative nature (so would naturally be "built up" using `#rept`, for example), or to create a value using several conditional steps where bitfields are ORed in, one at a time, conditionally.

The `#set` directive works well for iterative tasks, as the macro can be redefined at any time, typically in terms of itself:

```
#set index 0
#rept 42
    <do something>

    #set index index + 1
#endr
```

The limitation with numeric macros is that they can only be numeric. There is no way to repeatedly append to a macro that has a textual value, such as a string.

This page left blank for printing purposes.

5 Assembling Programs

5.1 INTRODUCTION

The CFX assembler converts CFX assembly language programs into relocatable object files, which can be linked together to form full executables to run on the CFX DSP. The details of the relocatable object file format are described in Section A.3, “Relocatable Object File Format” on page 122.

CFX assembly language is a typical “old-style”, mnemonic-based assembly language, with syntax to support the CFX DSP’s parallel instruction set. Prior to assembling, assembly language programs are usually preprocessed to provide macro facilities.

Two versions of the CFX assembler are provided:

- The new assembler, version 3 or greater, is used by default. It checks for CFX instruction restrictions (see Section 5.2.2, “Instruction Restriction Checks” on page 30) and provides new synonyms for instructions, addressing modes, and conditions to improve program readability (as described in Section 5.3.2.2, “Synonyms and Synthesized Instructions” on page 35). We strongly recommend using this assembler.

  When you run CFCC in verbose mode (using the `-v` option), you will notice from its output that it uses two programs from the EDK’s `bin` folder to assemble programs with the new assembler: `CFASM.EXE` and `CFBACK.EXE`.

- The old assembler, up to version 2.x and sometimes referred to as *CFPACK*, is provided for compatibility with earlier versions of the EDK; it is invoked by CFCC when the `--cfpack` option is given (see Section 5.2.1, “Assembler Options” on page 28). The programs `CFPACK.EXE` and `CFASR.EXE` in the EDK’s bin folder comprise the old assembler.

The two assemblers are compatible both at the source level—the new assembler extends the syntax supported by the old assembler—and at the binary level: both generate the same relocatable object file format. You can link programs together from a mix of object files and libraries assembled with the two assemblers.

5.2 RUNNING THE ASSEMBLER

In most cases, CFCC needs to preprocess assembly language source files. Preprocessing is needed to expand macros within the files, and to obtain information required to properly assemble the files, including hardware and operating software definitions.

CFCC automatically preprocesses assembly language input files with a `.cfs` extension. When CFCC encounters files with a `.cft` extension, it treats them as assembly language source code that does not require preprocessing, and assembles them without preprocessing.

Your use of CFCC options (see Table 2 on page 11) affects the behavior of the assembler, specifically those listed in Table 7. Also, since most assembly language files are preprocessed prior to assembly, you will often use CFCC’s preprocessor options as well (see Table 4 on page 16).

TABLE 7: CFCC OPTIONS AFFECTING ASSEMBLER BEHAVIOR

CFCC Option	Effect on the Assembler
`-c`	Assembles the source files, but does not link them. The assembler outputs an object file for each source file. This option also causes CFCC to ignore any unrecognized input files (i.e., files that do not require assembly).
`-o` **`file`**	Place output in the file specified as a parameter. This action occurs regardless of the type of output CFCC is producing, whether executable file, object file, or preprocessed source code. Since only one output file can be specified, it does not make sense to use `-o` when compiling more than one input file, unless you are producing an executable as output. If you do not specify `-o`, the default is to put the object file for **`source`**`.cfs` in **`source`**`.obj`, where **`source`** is the file name.
`-g`	Produce debugging information.
`-b` **`machine`**	The argument **`machine`** specifies the target machine for compilation. Currently, its value can only be `SK3`. CFCC defines the preprocessor symbol `_TARGET_`**`machine`** to indicate its target.

5.2.1 Assembler Options

In addition to the common CFCC options listed in Table 7, CFCC accepts assembler-specific options. Available options are listed and described in Table 8.

TABLE 8: CFCC ASSEMBLER COMMAND LINE OPTIONS

CFCC Option	Description
`--cfpack`	Uses the older *CFPACK* parser to assemble programs, rather than the restriction-checking CFX assembler, version 3. CFCC defines the preprocessor symbol `_CFXASM_VER` to a value in the 200 range (indicating version 2.x.y) when you give this option; otherwise, `_CFXASM_VER`'s value is 300 or more. We do not recommend using the `--cfpack` option since CFPACK does not check any CFX instruction restrictions. If some of the assembler's restriction checks are too stringent (i.e., the assembler produces error messages for instruction combinations that you know are valid), it is better to disable those specific restriction checks than to turn off all restriction checking by using CFPACK. For more information see Section 5.2.2, "Instruction Restriction Checks" on page 30.
`--lst`	Produces a program listing, showing the values generated for each instruction. Places this listing in the file `name.lst`.
`-p`	Produces a program listing, showing the values generated for each instruction. Places this listing in the file `name.lst`.

TABLE 8: CFCC ASSEMBLER COMMAND LINE OPTIONS (CONTINUED)

CFCC Option	Description
`--lpl` **`lines`**	Sets listing page length. This option determines the number of lines per page in a program listing. A header appears at the top of each listing page. If **`lines`** is zero, the listing is not paginated, and the header appears only once at the top of the listing. This is the default when the `--lst` option is given. If a listing is requested with the `-p` option, the default page length is 50 lines.
`--lb`	No blanks. Omit blank lines from the program listing, if one is requested. If the input has made extensive use of conditional preprocessing directives, you may want to use the `--lb` option to reduce the size and improve the readability of the listing file.
`--strictnop`	Arithmetic-Move instructions must contain both the Arithmetic instruction part and the Move instruction part. If an instruction part is not used, NOP must be used to complete the Arithmetic-Move instruction. By default, the CFX assembler accepts Arithmetic and Move instructions as if they are full Arithmetic-Move instructions. The assembler provides the missing NOP.
`--strict`	Disables support for synthesized and synonym instructions. By default, the assembler accepts alternate forms for some machine instructions, as described in Section 5.3.2.2, “Synonyms and Synthesized Instructions” on page 35. This option is only available with the CFX assembler version 3 or greater (that is, when `--cfpack` is *not* given). Only instruction synonyms are affected by this option; the assembler’s addressing-mode and condition synonyms are still accepted when `--strict` is given.
`--no-check`	Disables checks for restricted instruction combinations. By default, the assembler produces error messages and fails if it finds any incorrect or dangerous code combinations, as described in Section 5.2.2, “Instruction Restriction Checks” on page 30. This option is only available with the CFX assembler version 3 or greater (that is, when `--cfpack` is *not* given). We do not recommend using `--no-check` to disable *all* restriction checking. If some of the restriction checks are too stringent (i.e., the assembler produces error messages for instruction combinations that you know are valid), it is better to disable those specific restriction checks using the `.check` directive (see Section 5.3.3.5, “Directives for Controlling Checking” on page 45).
`--dotsegment`	Enables processing of the `.segment` directive by the CFX assembler. By default, the assembler does not accept the `.segment` directive because the `.pseg`, `.xseg`, `.yseg` and `.xyseg` directives are simpler to use, and they prevent errors with mismatched memory types and word sizes.
`--xref`	Produces a cross-reference listing, using the default options. For more information, see Section 5.4, “Generating a Cross-Reference” on page 51.
`--xrefopts` **`opts`**	Produces a cross-reference listing, using user-specified options. For more information, see Section 5.4, “Generating a Cross-Reference” on page 51.

TABLE 8: CFCC ASSEMBLER COMMAND LINE OPTIONS (CONTINUED)

CFCC Option	Description
`--dbgopts` **`opts`**	In conjunction with the CFCC `-g` option, produces assembler-level debug tables, using the specified options. For details, see Section 5.5, "Generating Debugging Symbols" on page 53.
`--line` **`levels`**	Generates file name and line number table entries only for the specified **`levels`**, which is a comma-separated list of file level numbers. By default, all levels are enabled. For more information, see the description of the `.file` directive in Section 5.3.3.8, "Directives for Defining Special Object File Entries" on page 49.

5.2.2 Instruction Restriction Checks

The CFX DSP's parallel instruction set and "visible" pipeline are designed for high performance, with minimal hardware restrictions. Pipeline integrity is the responsibility of software, so CFX programs must avoid certain instruction combinations and concurrent operations to ensure that the DSP executes instructions with their documented behavior.

The CFX assembler version 3 or greater, which CFCC uses to assemble programs unless the `--cfpack` option is given, checks all of the pipeline- and parallelism-related instruction restrictions that are documented in the *CFX DSP Architecture Manual*. The assembler takes a cautious approach to restriction checking. It produces an error message for code that *might* be restricted, if it cannot verify that no restrictions are violated. The following sections briefly describe the checks that the assembler performs, and how to interpret or disable the error messages that are produced.

5.2.2.1 Restriction Categories

The *CFX DSP Architecture Manual* provides the precise details regarding instruction restrictions. The general categories, with some notes to help understand the restriction checks, are as follows:

Multiplication Restrictions

All of the CFX DSP's multiplication instructions take two cycles to complete, with the second (ALU) cycle executing in parallel with the following instruction. The parallelism of that ALU operation—which occurs for both *multiply-only* (`MUL`, `MULNEG` and related mnemonics) and *multiply-accumulate* (`MULADD`, `MULSUB` and related mnemonics) instructions—introduces several potential conflicts related to timing and to resources such as registers and ALUs.

The "following instruction" is often the instruction at the next address in P memory, but when the multiplication is in the last delay slot of a control-flow instruction, the following instruction is at the target address of the control-flow instruction. For multiplications in the last delay slot of a conditional branch or as the last instruction of a hardware loop, there are two potential following instructions.

Delay Slot Restrictions

Delay slots allow the DSP's pipeline to read ahead, to avoid stalling execution when program flow changes. Since program flow is involved, further program-flow instructions cannot be used in delay slots. To simplify

the decoding that is done while reading ahead, the CFX also forbids the use of multi-cycle *Long* instructions in delay slots.

The instructions in the delay slots of the `RETURNI.2D` instruction have several other restrictions because `RETURNI.2D` is restoring the status register `SR` during its delay slots.

Hardware Loop Restrictions

The CFX DSP's zero-overhead hardware loops use the pipeline to "execute ahead" in a similar fashion to the read-ahead that the CFX does for program-flow delay slots. The program's flow changes at the end of the hardware loop, so there are similar restrictions at the end of hardware loops: control-flow instructions are forbidden. Furthermore, nested hardware loops cannot end on the same instruction.

Concurrent Register Updates

With the exception of the status register `SR`, multiple concurrent updates to a CFX register do not work. Such updates could occur within an instruction since the *Arithmetic* and *Move* parts of an *Arithmetic-Move* instruction execute in parallel, as do the *Dual* forms. Concurrent updates could also occur due to the two-cycle nature of multiplication instructions.

5.2.2.2 Restriction Error Messages

Restrictions are most often an interaction between two instructions:

- A *containing* instruction, such as a multiplication, a `LOOP`, or a control-flow instruction that has delay slots.
- An *erroneous* instruction; the instruction which—due to the restriction—cannot be used where it is, relative to the containing instruction.

When a restriction is violated, the assembler produces an error message that refers to the erroneous instruction. The message has the following format:

```
"<file>.cfs", line <L>/<C>: error: <message>
```

L is the line number of the erroneous instruction, and *C* is the character position within the line at which the problem was detected. *C* is often very useful for pinpointing which register or operand is at fault; even when *C* refers to a whole instruction, it can be useful for determining which sub-instruction of an *Arithmetic-Move*, *Move-Move* or *Dual* instruction is violating the named restriction.

The CFX assembler takes a fairly cautious approach to restriction checking, producing errors for dangerous constructs that might be valid in certain conditions—conditions that the programmer can verify but the assembler cannot. All violations (or potential violations) of restrictions cause the assembler to produce errors—not warnings—so CFCC fails if any restriction is violated.

To disable a restriction check, use the `.check` directive. If you use `.check` to turn off checking on a containing instruction, *all* checks related to that containing instruction and instructions it "contains" are disabled. For example:

```
goto.2d handle_negative if ltz
nop ||| load y0, y[yp0++]          // [delay slot 1]
muladd a0, x0, y0 ||| nop          // [delay slot 2]
```

```
    // The following STORE is valid -- storing the value
    // of a0 before adding x0 * y0 -- because interrupts
    // are disabled in this interrupt-handler code.
    .check off
    store x[X_PREVIOUS_PRODUCT], a0
    .check on

    ...

handle_negative:
    load xp0, X_PREVIOUS_PRODUCT
```

The assembler does not know that interrupts are disabled because your code is part of an interrupt handler and consequently, that the `STORE` instruction is not a problem. (If an interrupt did occur after the `MULADD` was initiated, the `STORE` would store the new value of `a0`, *after* adding `x0 * y0`.) Therefore, without the **`.check off`** directive, the assembler would generate an error message.

If the `handle_negative` label was external to this file (declared with `.extern` or `.declare`), so the assembler could not check the multiplication restrictions at that label (it must do this because the `MULADD` is in the last delay slot of the `GOTO.2D`), a further message would be produced. This could be suppressed—probably along with a comment explaining why the code at the `GOTO.2D`'s target address does not violate multiplication restrictions—using **`.check off`** prior to the `MULADD` instruction. Turning off restriction checking on that single instruction would also suppress the error message for the `STORE`, because the `MULADD` is the containing instruction for the `STORE` (the `STORE` is one of the two instructions that executes during the second cycle of the `MULADD`).

Note: Using **`.check off`** to suppress restriction checks based on knowledge about the code is dangerous because no automated checks ensure that the code continues to work properly when it is changed. Use **`.check off`** sparingly, and monitor dependencies within the code carefully as the code evolves.

See Section 5.3.3.5, “Directives for Controlling Checking” on page 45 for more information about the `.check` directive.

5.3 CFX ASSEMBLY LANGUAGE

The CFX assembly language is a typical mnemonic-based assembly language, with facilities for controlling memory segments, generating data tables, and referencing symbols defined externally, in addition to assembling CFX instructions. Details about the CFX instruction set may be found in the *CFX DSP Architecture Manual*.

5.3.1 Lexical Conventions

A name must begin with an alphabetic character, dot (.), underscore (_), or dollar sign ($). The rest of the name may consist of alphanumerics, underscore, or dollar sign, but no dots. The name “.” is replaced by the current program counter value. A name can be up to 256 characters long, and all characters are significant for both internal and external linkage.

The assembler is case-sensitive for names defined in your program, such as labels and segments. You can give CFX instruction mnemonics, instruction options, register names and assembler

directives in mixed case because they are case-insensitive. For example, you can spell the `XP1` register `XP1`, `xp1`, or `Xp1`.

Blanks and tabs are only needed to delimit tokens, where they are not already delimited by punctuation. For example, an instruction mnemonic is only recognized as such when you separate it from the first operand by at least one space or tab, since there is no punctuation between the mnemonic and the first operand.

When tokens are already delimited by punctuation, any number of spaces or tabs may appear between the tokens.

Continue very long lines by placing a backslash character at the end of the line, just before the newline character. CFCC implements this in the preprocessing stage; only use it in `.cfs` files, since these are preprocessed by CFCC. The CFX assembler version 3 or greater allows lines to be split—without a backslash—when it is obvious that an instruction is unfinished: after operators in expressions and after punctuation (such as the commas between operands or the sub-instruction separators `|||`, `>>>` or `!!`). This is implemented in the assembler parser, so you can use it in both `.cfs` and `.cft` files. For example:

```
muladd a0, x0, y0 !!
        muladd b0, y1, x1  |||
                load x0, x[xp0+!] !!
                        load y1, y[yp0++]
```

Furthermore, the version 3+ assembler allows you to write multiple instructions on a line, separated by semicolons. For example:

```
call.2d xp2
store y[Flag], 1;  add sp, 200  // [2 delay slots]
```

The preprocessor removes comments, and thus uses the C and C++ language conventions: between `/*` and `*/`, or from `//` to the end of the line. The old (CFPACK) assembler also removes comments that begin with ; (semicolon) and continue to the end of the line. However, since such comments are incompatible with the CFX assembler version 3, which uses the semicolon as an instruction separator, we do not recommend using semicolon comments with the old assembler except in the form of a terminator at the end of an instruction with nothing following except an optional `//` comment. For example:

```
returni.0d ;  // Works with both assemblers.
returni.0d    // Works with both assemblers.
returni.0d ;     Only works with old assembler.
```

The C/C++ comment style is also preferred because the preprocessor does not recognize the assembler comments, so it substitutes macros within them (causing trouble with multi-line macros) and tries to treat the comment text as C tokens (causing trouble with words such as “can’t”, which does not have a matching single quote).

Numbers are C-style and may be binary, hexadecimal, octal or decimal, as shown in Table 9.

TABLE 9: C-STYLE NUMBERS

Radix	Starting Characters	Permissible Characters
binary	`0b` or `0B`	01
hexadecimal	`0x` or `0X`	0123456789abcdefABCDEF
octal	`0`	01234567
decimal	non-zero digit	0123456789

The CFX assembler version 3 or greater recognizes boolean constants: the keywords `FALSE` and `TRUE` (which are case-insensitive like other keywords) are numbers with the values 0 and 1, respectively.

5.3.2 Assembly Language Syntax

The CFX assembly language is line-oriented. After preprocessing and removal of comments, each line in an assembly language file consists of zero or more labels, followed by an optional instruction or directive. As mentioned in Section 5.3.1, “Lexical Conventions” on page 32, the CFX assembler version 3 or greater automatically joins lines that are split after separators, and allows the semicolon to terminate or separate instructions on one line.

A label is a name, as defined in Section 5.3.1, “Lexical Conventions” on page 32, followed by a colon (`:`).

The syntax for directives is detailed in Section 5.3.3, “Assembler Directives” on page 39.

CFX instructions are written using the syntax shown in the *CFX DSP Architecture Manual*. Each instruction begins with a mnemonic, consisting of alphanumeric and dot (“.”) characters, which, depending on the instruction, may be followed by a comma-separated list of operands. The instruction mnemonic is separated from the operand list with spaces or tabs.

Typical instruction operands are labels, registers, conditions, constant expressions, and memory references.

5.3.2.1 Instruction Forms

CFX instructions come in three general forms, with the following syntax:

```
Long
Arithmetic  |||  Move
Move  >>>  Move
```

Both the **Arithmetic** and **Move** instruction classes have single and dual forms. **Dual Arithmetic** instructions are composed of an **X Arithmetic** instruction and a **Y Arithmetic** instruction:

```
Single Arith        |||  Move
X Arith !! Y Arith  |||  Move
```

Dual Move instructions are composed of an **X Move** instruction and a **Y Move** instruction:

```
Arithmetic   |||  Single Move
Arithmetic   |||  X Move !! Y Move
Single Move  >>>  Single Move
Single Move  >>>  X Move !! Y Move
```

All combinations of the different types are valid:

```
X Arith !! Y Arith  |||  X Move !! Y Move
X Move !! Y Move    >>>  Single Move
X Move !! Y Move    >>>  X Move !! Y Move
```

In the assembly language, each **Long** instruction and each instruction in the **Arithmetic-Move** and **Move-Move** instructions is written in the form of a typical assembly language instruction, with a mnemonic instruction name and operands (if any). The *CFX DSP Architecture Manual* contains a comprehensive list of all the instructions and their forms (**Long**, **Single Arithmetic**, **Y Move**, etc.)

Unless the `--strictnop` option is given to CFCC, the assembler recognizes **Arithmetic-Move** instructions that are missing either the **Arithmetic** or **Move** part (and the `|||` connector), and assembles the **Arithmetic** or **Move** instruction along with a `NOP`, into an **Arithmetic-Move** instruction. The CFX assembler version 3 or greater allows you to enable or disable these "optional NOPs" on a per-instruction basis using the `.strictnop` directive. See Section 5.3.3.5, "Directives for Controlling Checking" on page 45 for more information.

5.3.2.2 Synonyms and Synthesized Instructions

The CFX assemblers provide multiple forms for the same instructions or operands in cases where the alternate forms can improve program clarity. The following sections describe these *synonym* forms.

5.3.2.2.1 Synonym Instructions and Synthesized Instructions

Both CFX assemblers provide no-delay-slot forms for the control-flow instructions. These have no **`.nD`** suffix on their mnemonics (where **`n`** is 0, 1 or 2); they are provided to simplify the instruction set for new CFX developers and to speed initial development when optimization is not a concern.

These no-delay-slot instructions have the mnemonics `CALL`, `GOTO`, `LOOP`, `RETURN` and `RETURNI`. Where a zero-delay-slot form exists in the native instruction set (`GOTO.0D` and `RETURNI.0D`), the no-delay-slot form is a straightforward *synonym* for the machine's **`.0D`** instruction. Where the native instruction set only has one-delay-slot or two-delay-slot instructions, the no-delay-slot forms are *synthesized* from the machine instruction followed by one or two `NOP` cycles.

The CFX assembler version 3 or greater provides additional synonym instructions, as follows:

`NOT dst-reg, src-reg`
: The CFX does not provide an instruction for the unary logical complement operation. This *Single Arithmetic* synonym implements that operation by XOR'ing with -1 (which is all 1 bits in two's complement form).

`MOVE dst-reg, src-reg`
: The CFX does not provide an unconditional instruction for copying between full accumulators (copying all 56 bits, as *Arithmetic* instructions do). This *Single Arithmetic* synonym implements that operation by XOR'ing with zero.

SUB <immediate>

The CFX does not provide subtraction instructions with immediate operands. The assembler provides a `SUB` synonym for each `ADD` instruction that has an immediate operand. The synonym's immediate operand is negated. Thus you can say:

```
sub sp, 5
```

instead of:

```
add sp, -5
```

The *CFX DSP Architecture Manual* provides more information about the syntax and implementation of these synonym and synthesized instruction forms.

The CFX assembler version 3 or greater provides the `--strict` option (see Section 5.2.1, "Assembler Options" on page 28) and the `.strict` directive (see Section 5.3.3.5, "Directives for Controlling Checking" on page 45) for disabling and enabling the synonym and synthesized instructions.

5.3.2.2.2 Alternate Condition Forms

Both CFX assemblers provide synonyms for some of the conditions that you can use in conditional `GOTO` instructions and conditional *Arithmetic* instructions. For example, you can test whether the X Zero bit in the `SR` is set using any of the conditions `EQ`, `X_EQ`, `ZERO` or `X_ZERO`. The forms without the **`x_`** prefix are simpler and clearer if it is obvious from the context which ALU's flags you are testing. Similarly, the `EQ` form is clearer if two values (registers, constants) have been subtracted or compared, while the `ZERO` form is more straightforward if a value has been compared to zero.

The CFX assembler version 3 or greater provides additional synonyms:

- Relational comparisons to zero have new synonym keywords: `LTZ`, `GTZ`, `LEZ` and `GEZ` for example.
- You can use the `!` operator to logically invert a condition. For example, `!zero` is the same as `not_zero`, `!oflow` is the same as `no_oflow`, and `!ge` is the same as `lt`.
- You can use the keyword `IF` in place of the comma that separates the condition operand from the previous operand in conditional instructions. For example:

```
goto handle_value if !zero

sub x0, 1 if gtz  |||  nop
```

5.3.2.2.3 Addressing Mode Synonyms

The CFX assembler version 3 or greater recognizes alternate forms for some of the core's indirect addressing modes. Most of these synonym forms have two benefits: they use syntax that is the same as earlier DSP cores, and they are more concise (which can improve code readability when the forms are familiar). The indirect addressing modes are summarized in Table 10, where a memory-reference operand to X memory via `xp0` is used for illustration.

TABLE 10: ADDRESSING MODE SYNONYMS

Base Addressing Mode Syntax	Synonym Addressing Modes
`X[ xp0 ]`	
`X[ xp0++ ]`	`X[ xp0+ ]` `X[ xp0 += 1 ]`
`X[ xp0-- ]`	`X[ xp0- ]` `X[ xp0 -= 1 ]`
`X[ xp0 += 2 ]`	
`X[ xp0 -= 2 ]`	
`X[ xp0 += xstep0 ]`	`X[ xp0+! ]`
`X[ xp0 -= xstep0 ]`	`X[ xp0-! ]`
`X[ xp0 +%= xstep0 ]`	`X[ xp0+%! ]`

5.3.2.3 Numeric Instruction Operand Syntax

Numeric operands occur in many types of instructions. Details regarding the size, signedness, and function of these numeric operands can be found in the *CFX DSP Architecture Manual*, while the following paragraphs describe the syntax and constraints that apply to all numeric operands.

Immediate operands have four different forms, which determine the range of values, as follows:

- *immN* operands are put into *N*-bit fields with no defined signedness. They can therefore have values in the range -2^{N-1} to 2^N-1. This is the union of the ranges of *N*-bit two's complement signed numbers and *N*-bit unsigned numbers.
- *immNs* operands are put into signed *N*-bit fields, and can therefore have values in the range -2^{N-1} to $2^{N-1}-1$.
- *immNu* operands are put into unsigned *N*-bit fields, and can have values in the range 0 to 2^N-1.
- *immNn* operands are put into negative *N*-bit fields. The encoded value is taken to be negative by the CFX; that is, it is extended upwards with 1 bits. These immediate operands can have values in the range -2^N up to -1.

The subtract-immediate instruction synonyms provided by the CFX assembler version 3 or greater (see Section 5.3.2.2, "Synonyms and Synthesized Instructions" on page 35) use two further immediate operand forms:

- *immNsn* operands are negated and put into signed *N*-bit fields, and can therefore have values in the range $-2^{N-1}+1$ to 2^{N-1}.
- *immNp* operands are negated and put into negative *N*-bit fields. The encoded value is taken to be negative by the CFX; that is, it is extended upwards with 1 bits. These immediate operands can have values in the range 1 to 2^N.

The CFX assembler version 3 or greater generally allows more values for numeric operands than the old (CFPACK) assembler. In addition to allowing the range (as described above) that corresponds to the machine instruction's immediate field, the version 3+ assembler uses the destination's size and signedness to determine the valid ranges. Most registers are taken as "signed or unsigned", so if the size of the immediate field matches the size of the register (as

with the 24-bit `LOAD` immediate in the *Long* class), both signed and unsigned values are allowed. If the immediate field is smaller than the destination and is signed, the negative values in the range, sign extended up to the size of the destination, are allowed as unsigned values. Similarly, unsigned registers (such as pointer registers) and operands (for `CMPU.NX`, for example) can only be given unsigned values.

The assembler front end checks the value of an immediate operand if no labels are used in the expression. Out-of-range values trigger an error message. In some cases, such as bit indexes and shift counts, a tighter range is enforced as documented in the *CFX DSP Architecture Manual*.

Numeric operands are arithmetic expressions involving constants, symbols, operators, and parentheses for grouping. The operators available are the usual C expression operators, as shown in the Table 11.

TABLE 11: OPERATORS, BY GROUP

Operator	Description
`+ - ~ !`	Unary operators
`* / %`	Multiplicative operators
`+ -`	Additive operators (binary)
`<< >>`	Shift operators
`< > <= >=`	Relational operators
`== !=`	Equality operators
`&`	Bitwise AND operator
`^`	Bitwise exclusive OR operator
`|`	Bitwise inclusive OR operator
`&&`	Logical AND operator
`||`	Logical OR operator

Expressions are evaluated using C precedence rules, which are indicated by the rows in Table 11. Operators with higher precedence are at the top of the table, and operators with equal precedence appear in the same row. Note, however, that the logical AND and OR operators do NOT short-circuit as they do in C; both operands are evaluated, regardless of the truth value of the first operand.

An expression that does not reference any labels or external symbols (declared with the `.declare` or `.extern` directive) is an absolute expression. There is one exception to this constraint on absolute expressions: any subexpression of the form `label1 - label2`, where both labels are defined in the same segment within the current input file, can be evaluated by the assembler and is therefore absolute. Further operations may be performed on such a subexpression, and it remains absolute, subject to the same constraints regarding labels and external symbols.

The assembler evaluates absolute expressions. They are required in certain situations, such as the operand to the `.align` directive, where the assembler must be able to evaluate the expression.

A numeric expression that is not absolute is said to be relocatable. Relocatable expressions can use the operators in Table 11 on page 38, except for the following:

```
!     &&     ||     ==     !=     < >     <=     >=
```

Note: CFCC evaluates the operators of a relocatable expression during the *linking* phase.

Symbols created using the `.equ` directive follow the same rules, described in the preceding paragraphs, for determining whether their values are absolute or relocatable.

Arithmetic operations use 32 bits; all operations are signed using two's complement representation. The right-shift operator `(>>)` is "arithmetic" when applied to a negative value. That is, high-order bit positions are filled with copies of the sign bit. The division operator `/` and remainder operator `%` perform integer division that truncates towards zero, for negative and positive operands. This behavior applies to both absolute and relocatable expressions.

5.3.3 Assembler Directives

The numerous assembler directives can be classified by functionality:

- Declaring Symbols
- Defining Data
- Changing the Program Counter
- Partitioning Memory
- Controlling Assembler Checking
- Preprocessing
- Controlling the Listing File
- Defining Special Object File Entries

5.3.3.1 Directives for Declaring Symbols

.equ <name>, <num-expr>

This directive gives a symbolic name to an expression. The `<num-expr>` can contain references to values that you define later, as long as all values are fully defined by the time the assembler starts processing the line in the second pass.

.declare <label-list>

This statement identifies labels that are defined externally, allowing subsequent assembly code to reference those labels. The `.declare` directive creates *weak* references to the labels; only those labels that are actually referenced cause the object file in which they are defined to be linked into the program by CFCC's linking phase. The `<label-list>` is a comma-separated list of labels.

.extern <label-list>

This statement identifies labels that are defined externally, allowing the program to reference the labels and forcing them to be linked with the program. The `.extern` directive creates *strong* references to the labels; they are linked into the program by CFCC's linking phase due to the `.extern` directive, regardless of whether they are referenced by the code. The `<label-list>` is a comma-separated list of labels.

.global <label-list>

This statement identifies labels that can be referenced in other source files. The `.global` directive for a label can either precede or follow the definition of the label, but it must appear in the file in which the label is defined for the label to be available in other source files via `.declare` or `.extern`. The `<label-list>` is a comma-separated list of labels.

5.3.3.2 Directives for Defining Data

.data <num-expr-list>

This directive evaluates the numeric expressions contained in the comma-separated `<num-expr-list>` and places the results in consecutive words of memory, with one major exception.

For all segment types except XY data memory segments, each `<num-expr>` generates a data word. Within an XY segment, each pair of `<num-expr>`s generates a 48-bit XY-memory "word", where the first `<num-expr>` in the pair is the high-order 24-bit value that goes into Y memory and the second `<num-expr>` (which defaults to 0 if there are an odd number of `<num-expr>`s) is the low-order 24 bits that go into X data memory.

.fill <num-expr-1>, <num-expr-2>

Emits words with value `<num-expr-2>` until the program counter has `<num-expr-1>` in its least significant bits. If, when the `.fill` directive appears, the program counter has the required least significant bits, then this directive emits no words and does not change the program counter.

`<num-expr-1>` must be an absolute expression evaluating to a positive integer. This integer places an alignment restriction on the current segment: if x is the smallest power of 2 that is greater than `<num-expr-1>`, then the starting address of the current segment is a multiple of x. If the current segment has an absolute address, then this address must also be a multiple of x.

`<num-expr-2>` must be an absolute expression. There are no restrictions on its value, but only the least significant *N* bits will be used, where *N* is the minimum of 32 and the wordsize of the current segment. For example, `.fill 13, 255` forces the current segment to start at an address that is a multiple of 16, and emits words with the value `255` until the least significant binary digits of the program counter are `1101`, which is the binary representation of `13`.

.zero <num-expr>

This directive sets the next `<num-expr>` entries in the current segment to zero. `<num-expr>` must be an absolute expression.

.bss <num-expr>

This directive reserves the next `<num-expr>` entries in the current segment, but does not assign any initial value to them. `<num-expr>` must be an absolute expression. The memory region is reserved but uninitialized in CFCC's output files (see Appendix A, "File Formats" on page 115), but the behavior of program loaders varies for such memory regions; some loaders clear `.bss` regions and some do not modify `.bss` memory regions.

.string <option-list>, "string"

.pack <option-list>, "string"

These directives fill memory with a text string. The string is given in a form similar to a C string constant. It must be surrounded by double quotes, and C-style backslash escape sequences are recognized. Table 12 lists and defines escape sequences.

TABLE 12: ESCAPE SEQUENCES

Sequence	Description
\a	alert (bell)
\b	back space
\f	form feed
\n	newline (line feed)
\r	carriage return
\t	horizontal tab
\v	vertical tab
\\	backslash
\"	double quote
\nn…	octal number
\xnn…	hexadecimal number

The octal and hexadecimal number escapes can contain an arbitrary number of digits (not just 3 or 2, as in C). The assembler does not add a NUL character to the end of the string.

You define the way in which the string characters are placed into words by using an `<option-list>`, which is a comma-separated list of option specifiers.

Table 13 lists and describes the option specifiers. The `.string` and `.pack` directives differ only in the default values for these options.

TABLE 13: .STRING AND .PACK OPTIONS

Option	Description
`bits=<num-expr>`	This option gives the size of each character, in bits. The `<num-expr>` must be an absolute expression, the value of which lies in the range 1 to the wordsize of the current segment, and does not exceed 32. Characters that are too large for the number of bits specified are masked to fit the space. The default number of bits is `8`.
`count=<num-expr>`	This specifies the number of characters to be placed in each word. The `<num-expr>` must be an absolute expression, at least 1, and small enough so that the given number of characters fit in a word. For `.string`, the default count is `1`. For `.pack`, the default count is the number of characters that fit in a word, as given by the current segment's wordsize.
`order=low|high`	This option defines the order in which characters are placed in the word. If the order is `high`, then each character is placed in the highest available bits. That is, characters are packed "left to right". If the order is `low`, each character is placed in the lowest available bits. The default order for `.string` is `low`, and for `.pack` is `high`.
`pad=<num-expr>`	This option provides a value to fill unused character positions. The `<num-expr>` must be an absolute expression. If a word contains fewer characters than provided by the count, then the remaining character positions are filled with the pad value, masked to the number of bits per character. Note that this padding applies to unused character positions, not to any part of the word that never contains characters. The default pad is 0.

By default, the `.string` directive places one 8-bit character in the least significant bits of each word, while the `.pack` directive places *N* 8-bit characters (where N is the wordsize of the current segment divided by 8) in each word, packing them from left to right.

5.3.3.3 Directives for Changing the Program Counter

.org <num-expr>

Use this directive to set the current program counter to `<num-expr>`, the value of which must be either absolute or relative to the current segment. In the expression, you must not refer to a constant name or label that is defined after the `.org` directive. Fully define all values used in the `<num-expr>` in the first pass before the `.org` directive is processed. The new origin is always relative to the start of the current segment.

.align <num-expr>

Sets the current program counter to the next address that is a multiple of `<num-expr>`. For example, `.align 4` moves the program counter to the next four-word boundary, if the program counter is not already at that boundary. The `<num-expr>` must be absolute and a power of 2. The start address of the current segment in the current file is also forced to the desired alignment boundary. If more than one `.align` directive is in a segment in a file, the segment is aligned

according to the maximum .align value found. If the current segment has an absolute address, then this address must be a multiple of <num-expr>.

IMPORTANT: If there is a label on the statement, its address is not aligned as you might expect, because the new alignment does not take effect until the beginning of the next statement.

For example, instead of:

```
label: .align 0x10          // the label will NOT be aligned
```

you should say:

```
    .align 0x10
label:                      // the label will be correctly aligned
```

5.3.3.4 Directives for Partitioning Memory

.pseg [<name>]

.pseg [<name>,] <option-list>

The .pseg directive creates a program-memory segment. The <name> defaults to "program" if it is not given. There are also .xseg, .yseg, and .xyseg directives that have similar syntax but differ in terms of the memory bank of the segment that they define or reference. Table 14 lists the memory-bank parameters and the default segment names for all of these directives.

TABLE 14: SEGMENTING DIRECTIVES

Directive	Memory Bank	memtype	wordsize	Default Name
.pseg	Program memory	0	32	program
.xseg	X data memory	1	24	xdata
.yseg	Y data memory	2	24	ydata
.xyseg	X and Y data memories	3	48	xydata

The .pseg family of directives instruct the assembler to begin assembling code or data into a memory segment with the given or default name, which has characteristics as shown in Table 14, and as given by <option-list> if it is provided.

Memory segments allow the program's code and data to be grouped into logically related sections. These directives allow the programmer to specify which memory bank contains a given section of the program, and where, within that memory bank, the code and data are located.

The memory areas in an SK3 system are described in detail in the *Hardware Reference Manual for BelaSigna 300*. More information can also be found in Section 7.4, "Memory Configuration" on page 61.

You can use these directives with the same <name> multiple times within a module, or in different modules. If you specify the same segment multiple times throughout a program, provide the same option values each time in the <option-list>; the assembler and linker enforce this.

After the first time that a given segment is used in a module, the short-form `.pseg [<name>,] append` can be used in the same module; the options all take the values previously specified. A similar short-form can be used with `.xseg`, `.yseg`, and `.xyseg`.

Segment names are treated as a separate class of names, so it is possible to have a label and a segment with the same name.

If no segment directive is given at the start of an input file, the assembler acts as if the directive `.pseg` precedes the code in the file.

The `<option-list>` is a comma-separated list of options, which are described in Table 15.

TABLE 15: SEGMENTING DIRECTIVE OPTIONS

Option	Description
`base=<num-expr>`	Specifies an absolute base address `<num-expr>` at which a segment is to be linked. If there are several segments with the same `memtype` and `base`, they are arranged in order of link time appearance at successively higher addresses starting at or above the base. If previously linked segments were allocated in space up to and beyond the base, then the segment is allocated at the first available address above the base. If this option is not specified, no base address restriction is assumed.
`abs=<num-expr>`	Specifies that the segment is absolute; it must be linked starting at the address `<num-expr>`. All other segments are relocatable, which means that CFCC can place them anywhere in memory during the linking phase. Space for absolute segments is always allocated first by CFCC, during linking. CFCC generates an error message if an attempt is made to overlap two or more absolute segments. An absolute segment with a given name (or the default name) cannot be used in multiple modules.
`align=<num-expr>`	Specifies that the segment's start address must equal zero modulo the given `<num-expr>`, which must be a power of two. The default alignment value is 1.
`rom`	Specifies that the linker should locate the segment in an area of memory that has been designated as ROM using CFCC's `--romN` or `--nromN` option.
`append`	Specifies the option values provided in a previous definition of the segment. This option cannot be used with any other options. The segment must be previously defined in this module.

We recommend that you use these short-form segment directives, instead of the more general `.segment` directive, so that the memtypes and wordsizes of segments are consistent.

.segment <name> [, <option-list>]

The `.segment` directive is the most general way to declare a new memory segment. However, with `.segment` you can inadvertently create a segment that other SignaKlara development tools cannot recognize (due to an incorrect memory bank number or wordsize). Therefore, we recommend that you use the `.pseg` family of directives instead of `.segment`. CFCC generates an error message when the `.segment` directive is used unless you give the `--dotsegment` option.

The use of segments is described above for the `.pseg` directive. The `append` option cannot be used with `.segment`; the corresponding short-form for `.segment` is simply `.segment <name>`.

The `.segment` directive accepts the options shown in Table 15 on page 44, with the exception of `append`, and additional options shown in Table 16.

TABLE 16: .SEGMENT OPTIONS

Option	Description
`memtype=<num-expr>`	Specifies the memory bank to which the segment corresponds. The valid values for `<num-expr>` are 0 (program memory), 1 (X data memory), 2 (Y data memory), and 3 (XY data memory). The default memory type is 0.
`wordsize=<num-expr>`	Specifies the number of bits in a word, in the segment. Program memory must be specified as 32 bits wide; X and Y data memory as 24; and XY data memory as 48.
`common`	Segments defined with the `common` option are handled specially during the linking phase. If `common` segments with the same name appear in different modules, CFCC assigns the same block of memory to all of them. All `common` segments with the same name must be the same size, and at most one can contain data; the rest must use only `.bss` directives to determine the size of the segment. These restrictions are enforced by the linker. By default, segments are not `common`.
`blank_common`	Segments defined as `blank_common` are similar to those defined as `common`, with two exceptions: • No `blank_common` segment can contain any data. • `blank_common` segments in different modules do not have to be the same size; the linker uses the largest size defined. These restrictions are enforced by the linker. By default, segments are not `blank_common`.

5.3.3.5 Directives for Controlling Checking

These directives are provided by the CFX assembler version 3 or greater for controlling, on a per-instruction basis, modes that enable or disable various kinds of assembler checking. The setting for each mode (associated with each directive) is maintained in a stack, with the top-of-stack value controlling whether the mode is on or off at a given point in the compilation. Each stack is initialized with a single entry, based on CFCC command-line options.

The check-controlling directives take a single keyword parameter that determines the action taken by the directive, as follows:

ON Turns the mode on by replacing the top-of-stack element with **On**.

OFF Turns the mode off by replacing the top-of-stack element with **Off**.

PUSH Pushes a new element onto the top of the mode's stack. The new element has the same value as the previous top-of-stack value.

POP Pops the top element off of the mode's stack, so the next element down becomes the top-of-stack value and the current value for the mode changes to that value.

The checking modes are maintained in stacks so that separately-developed utility modules such as header files and macros can set the modes that their code requires, while maintaining the modes of the surrounding (invoking) code that are determined by another developer or team. The examples in the sections below illustrate the typical usage.

.strictnop <action>

The `.strictnop` directive controls whether `NOP` instructions are required (strict) or optional (not strict) with *Arithmetic-Move* instructions that only have an *Arithmetic* or a *Move* part. See Section 5.3.2.1, "Instruction Forms" on page 34 for more information about optional `NOP`s.

This mode is initialized to **Off** (not strict) by default. If the `--strictnop` option is given on the CFCC command line, this mode is initialized to **On** (strict).

A typical use in a `#macro` might be:

```
.strictnop push
.strictnop off
mul a0, x0, y0  // Arith with no Move NOP
.strictnop pop
```

.strict <action>

The `.strict` directive controls whether instruction synonyms and synthesized instructions are accepted by the CFX assembler version 3 or greater. See Section 5.3.2.2, "Synonyms and Synthesized Instructions" on page 35 for more information about these instructions.

This mode is initialized to **Off** (accept synonyms and synthesized instructions) by default. If the `--strict` option is given on the CFCC command line, this mode is initialized to **On** (strict, do not accept synonyms and synthesized instructions).

.check <action>

The `.check` directive controls whether the assembler checks for restricted (invalid or dangerous) instruction combinations. See Section 5.2.2, "Instruction Restriction Checks" on page 30 for more information about restriction checking.

This mode is initialized to **On** (check restrictions) by default. If the `--no-check` option is given on the CFCC command line, this mode is initialized to **Off** (do not check restrictions).

If restriction checking is turned off for a *containing* instruction—a `LOOP` instruction, a control-flow instruction with delay slots, or an *Arithmetic-Move* instruction involving a multiplication (or two)—checks are disabled for that instruction *and* all instructions that it "contains": the instructions in the loop, the delay slot instructions, or the instructions that execute in the second cycle of the multiplication.

A typical use of `.check` might look like:

```
// This code should be clean - turn checks on
// (Often done at the top of the file, or left out
```

```
    // with the assumption that checking is on by default.)
    .check on
    ...
    cmp x0, y0 ||| nop
    goto.2d same_values if eq
    sub x0, x0, y0 ||| load y1, 5  // [delay slot 1]
    mul a0, x0, y0 ||| nop         // [delay slot 2]

    // This conditional GOTO is okay because Flag Updates
    // are disabled, so MUL does not affect the condition
    .check off
    goto.0d less_than if lt
    .check on
same_values:
    ...
```

The programmer has disabled restriction checks for the `GOTO.0D` instruction to suppress the error that the assembler would otherwise generate; it is safe to turn off checking on that instruction because the programmer knows that the Flag Updates mode is disabled (which the assembler cannot know). If the instruction at the label `same_values` violated a multiplication restriction, the assembler would catch this. If restriction checks were turned off for the `MUL` instruction instead, this would suppress the error message on both the `GOTO.0D` *and* the instruction at `same_values`.

5.3.3.6 Directives for Preprocessing

Note: It is generally preferable to use CFCC preprocessing directives, instead of comparable assembler directives. For more information, see Table 5 on page 18.

.error <text>

This directive causes a fatal error message, containing the given `<text>`, to be written to the standard output. It is preferable to use the CFCC `#error` preprocessing directive. The assembler aborts processing on the second pass, so the instructions and directives following this directive must be valid. No listing file is produced.

.warning <text>

This directive causes a non-fatal warning message, containing the given `<text>`, to be written to the standard output. It is preferable to use the CFCC `#warning` preprocessing directive.

.if <num-expr>

This directive begins a conditional assembly section. The `<num-expr>` cannot be a relocatable expression and may only contain values that have already been defined. If the value of `<num-expr>` is not zero, then the assembly code following this directive is assembled until the corresponding `.else` or `.endif` directive is found. You can nest conditional assembly sections. It is preferable to use the CFCC `#if` preprocessing directive.

.ifa <num-expr>,<min>,<max>

This directive begins a conditional code selection. The `<num-expr>` must be an absolute expression, but it may contain values that you define later in the file. The `<min>` and `<max>` must

be absolute expressions, and may only contain values that have already been defined. If the value of <num-expr> is between <min> and <max> inclusive, the assembler emits the assembled code between this directive and the corresponding .else. If the value is outside the range of <min> to <max>, the assembler emits the assembled code between the .else and the .endif.

There are several important differences between .if and .ifa. The controlling expression of a .if must have a value that is known and fixed at the time the assembler reads it. At that point the assembler selects the appropriate part of the .if — either the .if part or the .else part — once and for all. It processes the instructions and directives in the chosen part, and completely ignores those in the other part. By contrast, the value of the controlling expression of a .ifa need not be known until the end of the file. The assembler reads and processes the instructions and directives in both parts of the .ifa. As a result, no single label or name can be defined in both parts. It also means that it is normally not a good idea to have directives such as .pseg in either part of a .ifa. However, .org and .align directives, as well as directives that define data words (namely .data, .string, .pack, .zero, and .bss), behave just like instructions, in that they are emitted only if the enclosing part is selected.

Once the assembler has read the entire input file, it determines the address of each label under the assumption that the controlling condition of every .ifa will be met — that is, that every <num-expr> will fall within the specified range. It then evaluates the controlling expression of each .ifa. If the controlling condition of some .ifa fails, then the assembler selects its .else part. Since the .else part need not contain the same number of instructions as the .ifa part, this may change the locations of some labels. So, the assembler repeats the process for each remaining .ifa, using the new values of labels.

The .ifa directive is designed to select between short and long versions of operations like branching or memory addressing. We recommend that the .ifa part contain the preferred (short) form, and the controlling expression indicate the condition under which the short form is valid.

.else

This directive begins a conditional assembly “else” section. If the .else corresponds to a .if directive whose <num-expr> is zero, the assembly code following this directive is assembled until the corresponding .endif directive is found. If the .else corresponds to a .ifa directive, the assembly code following this directive is selected if the .ifa condition fails.

.endif

This directive terminates a conditional assembly section.

5.3.3.7 Directives for Controlling the Listing File

.title <text>

The argument <text> consists of everything up to the end of the .title line. If listing is enabled, the <text> is displayed on subsequent page headings.

.eject

If listing is enabled, this directive causes the next line to appear on a new page.

.nolist

This directive inhibits the generation of the listing file.

.list

This directive re-enables the printing of lines to the listing file. You can nest the `.list` and `.nolist` directives, and they simply cancel each other. If more `.nolist` directives are processed than `.list` directives, listing is disabled.

5.3.3.8 Directives for Defining Special Object File Entries

For more information on using these assembler directives, see Section 5.5, "Generating Debugging Symbols" on page 53.

.comment <text>

This directive causes a comment record, containing the given `<text>`, to be written to the relocatable object file.

.file <filename>,<level>

This directive describes a file name table entry for debugging. The `<filename>` can be any double-quoted string, or it can be one of the keywords `.file` or `.list`, which refer to the name of the current input file to `CFBACK.EXE` or `CFASR.EXE` (not to the original assembly language source code file). The `<level>` is a small integer that you can use to distinguish between different types of files. It can also be the keyword `.file`, meaning the level conventionally assigned to assembler input files (14), or `.list`, meaning the level conventionally assigned to assembler listing files (15). You can use any number in the range 0 to 31, but certain numbers have conventional interpretations or are used by particular tools as described in Table 17.

TABLE 17: FILE LEVELS

File Level	Conventional use
0	Unknown file type.
4	Reserved for C programs.
6	Reserved for C programs.
7	Assembler output, line numbers for #macro refer to invocation.
8	Original assembly code.
10	Preprocessed assembly code.
12	CFBACK or CFASR input as created by CFASM or CFPACK.
14	Raw CFBACK or CFASR input.
15	Listing file.

.line <line-number>, <filename>, <level>, <num-expr>

This directive describes a line number table entry, for debugging. Everything but the `<line-number>` is optional. The `<line-number>` can either be an absolute expression, or the keyword `.line`, meaning the current input file line number, or the keyword `.list`, meaning the current listing file line number. The `<filename>` and `<level>` are described under the `.file` directive. If the `<filename>` and `<level>` are omitted, the assembler uses the file most recently given in any `.line` or `.file` directive. Finally, the `<num-expr>` provides a

segment-relative address to be associated with the line number; if omitted, the assembler uses the current address in the current segment.

.dbg <dbsym>, <infotype>, <info> [, <infotype>, <info>]

The .dbg directive defines or adds to an entry in the debug symbol table. Debug symbols are distinct from assembler symbols. Debug symbols may or may not directly correspond to assembler symbols. A debug symbol has a name and associated debug information. Each item of debug information consists of an <infotype> and corresponding <info>. The <infotype> is a positive integer in the range 1 to 65534 that identifies a class of debugging information; for example, address, data type, storage class, etc. The assembler and linker impose no interpretation on <infotype> values in the allowable range, but conventionally, <infotype> 1 is used for a symbol link address. (The linker reserves the <infotype> values 0 and 65535 for the debug symbol name and debug symbol table index, respectively.) Normally, it is a good idea to use small <infotype> values because the default debug symbol table format generated by the linker encodes consecutive <info> values in <infotype> order, from 1 to the highest defined <infotype>.

Several different kinds of <info> can appear in the debug symbol table entry.

1. A relative expression, such as a symbol address can appear. This example records the value of assembler symbol foo (that is, the address of label foo) in <infotype> slot 1 of the debug symbol foo:

```
.dbg foo, 1, foo
```

2. An absolute expression, such as a constant encoding some information about the symbol, can appear. This example records the constant 23 in <infotype> slot 2 of the debug symbol foo:

```
.dbg foo, 2, 23
```

3. The keyword .line, which enters the current input file line number in the symbol table entry, or the keyword .list, which enters the current listing file line number can appear. This example records the current listing file line number in slot 4, and records the current input file line number in slot 6:

```
.dbg foo, 4, .list, 6, .line
```

4. A debug symbol name, enclosed in braces, can appear. The debug symbol is created if it does not already exist. This technique could be used, for example, to associate a debug symbol for a C structure tag with a variable of that type. This example records the debug symbol bar in <infotype> slot 7 of the debug symbol foo:

```
.dbg foo, 7, {bar}
```

   In the linker output, slot 7 of foo will contain the symbol table index of bar.

5. A string, enclosed in double quotes, can appear. This example records the string 23 skidoo in <infotype> slot 8 of the debug symbol foo:

```
.dbg foo, 8, "23 skidoo"
```

   In the linker output, slot 8 will contain the string table index of 23 skidoo.

6. A file name, enclosed in angle brackets can appear. This example records the file name multiply.a in <infotype> slot 3:

```
.dbg foo, 3, <multiply.a>
```

In the linker output, slot 3 will contain the file name table index of `multiply.a`.

7. The keyword `.file`, which enters the current input file. This example records the current input file in `<infotype>` slot 5:

```
.dbg foo, 5, .file
```

In the linker output, slot 5 will contain the file name table index of the current input file.

In the object file, debug symbols appear in the order in which they were created.

5.4 GENERATING A CROSS-REFERENCE

The assembler can produce an optional symbol cross-reference, indicating where symbols are defined and where they are used. The following sections discuss the various cross-reference options (which you can control with the `--xrefopts` option), and discuss the default settings.

5.4.1 Cross-Reference Options

If you specify an `--xrefopts opts` option, you can control all the optional features of the cross-reference generator. In place of `opts`, you provide a string ("the options string") containing one or more of the symbol selection, symbol filtering, and output control characters listed in the following three sections.

5.4.1.1 Symbol Selection

The options shown in Table 18 control which types of symbols are included in the cross-reference listing.

TABLE 18: SYMBOL SELECTION OPTIONS

Option	Description
`n`	Include name symbols. Name symbols are generally labels or defined as constant expressions. If an `n` appears in the options string, the cross-reference includes name symbols.
`s`	Include segment symbols. Segment symbols are created by the `.pseg` family of directives. If an `s` appears in the options string, the cross-reference includes segment symbols.

5.4.1.2 Symbol Filtering

The options shown in Table 19 allow you to include or exclude certain symbols, depending on how they are defined and/or used.

TABLE 19: SYMBOL FILTERING OPTIONS

Option	Description
`b`	Include built-in symbols. If the options string contains a `b`, the cross-reference includes built-in symbols. Normally these symbols are not included in the listing.
`r`	Exclude unreferenced symbols. If the options string contains an `r`, the cross-reference excludes any symbols that are not used.
`u`	Exclude undefined symbols. If the options string contains a `u`, the cross-reference excludes any symbols that are not defined.

5.4.1.3 Output Controls

The options shown in Table 20 control the format of the cross-reference listing.

TABLE 20: OUTPUT CONTROL OPTIONS

Option	Description
`c`	Use compact output format. Without this option, the cross-reference listing contains a blank line before the entry for each symbol. With the `c` option, no blank lines are printed.
`f`	Send the cross-reference to a separate file. If you invoke the assembler with a `-p` or `--lst` option (to generate a program listing), and the options string does not contain an `f`, then both the program listing and the cross-reference appear in the file `name.lst`. If you invoke the assembler with a `-p` or `--lst` option, and the options string contains an `f`, then the program listing appears in a file called `name.lst`, and the cross-reference appears in the file `name.xrf`. If you invoke the assembler with neither the `-p` nor the `--lst` option, then no program listing is created, and the cross-reference appears in the file `name.xrf`, regardless of whether the options string contains an `f`.
`i`	Ignore case when sorting. In the cross-reference, symbols are sorted alphabetically according to their names. By default, this sort is case-sensitive. If you supply the `i` option, the cross-reference ignores the distinction between upper and lower case when sorting its listing. *Note:* This option applies only to the order in which symbols appear in the cross-reference, and not to any other aspect of the assembler.
`t`	Include symbol type info. If you include a `t` in the options string, each cross-reference entry indicates the name and the type of the symbol listed. Without the `t` option, the entry contains the name of the symbol, but no indication of its type.
`v`	Sort primarily by type. By default, symbols appear in the cross-reference according to their names. With this option, the primary sort key is the symbol's type; all symbols of a given type appear together, sorted by name.

5.4.2 Defaults

If you specify `--xrefopts` with an empty options string, the assembler assumes that you mean `--xref` and uses the default options as indicated below.

The command-line option `--xref` provides a default cross-reference, just as if you had specified `--xrefopts nst`.

In other words, the default cross-reference contains entries for names and segments, plus type information for all of the symbols it lists. It excludes predefined and built-in symbols, it heeds the distinction between upper-case and lower-case when sorting, and it uses its normal (not compact) output format. The cross-reference listing goes in `name.lst` with the program listing, if you request one; otherwise, it goes in `name.xrf`. If you want to override any of these defaults, you must use `--xrefopts` instead of or in addition to `--xref`.

If you specify `--xrefopts` where the options string does not contain any of `ns`, then you have requested a cross-reference including no symbol types. In this case, rather than generating an empty cross-reference, the assembler includes the default symbol types (names and segments).

5.4.3 Cross-Reference Output

The listing for each symbol in a cross-reference listing takes the following form:

```
name {type} [d_line] u_line ...
```

where:

- `name` is the name of the symbol.
- `type` is the symbol's type, and is omitted if your options string does not contain a `t`.
- `d_line` is the number of the line on which the symbol was defined.
- `u_line` is the number of a line on which the symbol was used.

  The listing contains as many `u_line` entries as necessary to indicate each reference to the symbol.

5.5 GENERATING DEBUGGING SYMBOLS

The assembler can automatically generate debug table entries for debugging assembly code if you use the `-g` option in CFCC. These entries are placed in the debug symbol, file name, and line number tables of the Absolute-format executable file generated by the linker, in addition to any entries defined by `.dbg`, `.file`, or `.line` directives.

5.5.1 Options

You can use the CFCC `-g` command line option to produce debugging information. Use the `--dbgopts opts` command line option to control which debug table entries are produced. The `opts` consists of one or more of the characters shown in Table 21.

TABLE 21: DEBUG OPTIONS

Character	Description
s	Generate a debug symbol for each assembler segment.
y	Generate a debug symbol for each assembler symbol (that is, each label or equate).
m	Modify the debug symbol entries for assembler symbols so that they do not conflict with user-defined debug symbol entries.
v	Generate line number tables for the assembly language source file. These entries are generated at level 10. (See the `.file` directive for a description of file levels.)
p	Generate line number tables for the listing file. This option is meaningful only if a listing file is generated.

If you give the `-g` option without the `--dbgopts` option, the assembler behaves as if you gave `--dbgopts syv`, producing debug symbol table entries for segments, symbols, and source code line numbers.

6 Using the Object Librarian

6.1 INTRODUCTION

CFLIB, an object librarian, creates and manipulates libraries of object modules produced by CFCC (with the `-c` option). Once these libraries have been created by CFLIB, they can be loaded and searched when linking with CFCC.

6.2 LIBRARIAN USAGE

6.2.1 Syntax

Run CFLIB from the **EDK SignaKlara Command Prompt** for this EDK version using the following syntax:

```
cflib l=filename [option]*
```

where `l=`**`filename`** tells CFLIB the name of the working library to which all operations are applied. If the file name has no suffix, `.obj` is normally appended. This argument must appear exactly once.

6.2.2 Options

Table 22 lists and defines the options you can use with CFLIB to determine its behavior.

The options that can modify the contents of the working library ("`filename`", "`del=module`", and "`move=module1,module2`") are performed in the order in which they appear on the command line. Occasionally the order of these options can be significant. For example, if a library contains modules named "`a`", "`b`", and "`c`", in that order, the option pairs "`move=a,c move=c,b`" and "`move=c,b move=a,c`" produce different results.

TABLE 22: CFLIB COMMAND LINE OPTIONS

Option	Description
`filename`	Gives the name of an object file or library to be edited into the working library. Object modules with the same name as modules already in the library replace those modules. Other modules are added to the end of the working library. If the file name has no suffix, one is appended: `.obj` is the default.
`+clear`	If this option is given, the initial contents of the working library are ignored. Using this option is equivalent to deleting the library file before running CFLIB.

TABLE 22: CFLIB COMMAND LINE OPTIONS (CONTINUED)

Option	Description
`list=[`**`modulename`**`]`	The given module is listed, or if no **`modulename`** is given, all modules in the library are listed. The default module listing contains the name of the module and the time of its creation. Under the `+v` option, the listing also includes information about the segments, symbols, and comments in the module(s).
`del=`**`modulename`**	A module with the given name is removed from the working library. If no such module is present in the library, the option is ignored.
`move=`**`module1`**`,`**`module2`**	The object module named **`module1`** is moved within the library immediately in front of **`module2`**. Both modules must already be present in the library.
`lookup=`**`symbol`**	This option gives the name of a symbol as defined in a `.global`, `.declare` or `.extern` assembly directive. CFLIB lists the name of any module that contains the given symbol.
`tag=`**`.ext`**	The `tag=` option tells CFLIB what suffix to append to file names (given in "`filename`" or "`l=filename`" options) that do not already have a suffix. The default suffix is `.obj`.
`x=`**`filename`**	The `x=` option tells CFLIB to stop processing the current set of options in favor of processing the options contained in the indirect file given following the `x=`. Newlines or blanks can separate options in the indirect file, allowing one or more options per line. When the end of file is reached in the indirect file, option processing resumes at the next option after the `x=`.
`+upd`	Normally, if an object module added under the `filename` option has the same name as a module already in the library, the existing module is replaced by the new module. The `+upd` option causes replacement to occur only if the replacement module is newer than the library module. The age of a module is determined by the creation time in its start record.
`+v`	The `+v` option provides verbose output. Specifically, it causes information about segments and symbols to be displayed under the `list=` option. Repeat this option for additional information.
`+s`	Be silent unless errors occur.

6.3 ORDERING OBJECT MODULES IN A LIBRARY

See Section 7.3, "Ordering Object Module Libraries" on page 60.

7 Linking Object Modules

7.1 INTRODUCTION

By default, CFCC links together relocatable object code into absolute executable code. The executable output file format is called *Absolute*, and is described in Appendix A, “File Formats” on page 115.

7.2 LINKING RELOCATABLE OBJECT CODE

This section describes the various CFCC linking options and the CFLIB environment variable. For information on CFCC syntax, see Table 2 on page 11.

7.2.1 Linking Options

Available options are listed and described in Table 23.

CFCC only performs the linking stage in the absence of the `-E` and `-c` options, which halt processing after the preprocessing and compilation stages, respectively.

Output file names are determined in the following manner:

- In the absence of the `-o` option, the output file is `a.abs`, unless only one input file is given, in which case the output file name is the name of the one input file with its suffix replaced by `.abs`.
- When the `-o` option is given, the output file is given the name provided.

Relocatable object files are linked in the order in which they are found while command options are collected. Memory area options (`--memN`) are sorted in ascending order by start address and checked for address overlap with previous memory area options for the same memory type. All other options are collected and set up before linking begins. If an option (such as `--mapfile` or `--entry`) provides a value, the last option of that type found is the one that is used.

TABLE 23: COMMAND LINE OPTIONS

CFCC Option	Description
filename	Gives the name of a file to be linked, which can be an object file, a source code file, or a library file. A source code file is first compiled, and CFCC links the resulting object module. A library file named directly on the CFCC command line (i.e., *not* as an operand to the `-l` option) is treated simply as a collection of object files; *all* of the object modules are linked into the resulting executable file. To search a library and link only the referenced object modules, see the `-l` option. CFCC assigns addresses in order from left to right for files named on the command line, and from the start to the end of a library file. A file to be linked at a lower address should appear first on the command line.
`--mapfile` **filename**	Writes a load map to **filename**. The format of the load map is discussed in Section 7.6, “Load Map Format” on page 63. If **filename** does not have an extension of `.map`, that extension is appended. Do not use this option if you are using `--map`.
`--map`	Write a load map to `outfile.map`, where `outfile` is the prefix of the name used for CFCC’s executable output. Do not use this option if you are using `--mapfile`.
`--entry` **name**	This option allows you to specify that the symbol **name** is the entry address of the program being linked. To allow for the case in which you need to link together code for more than one processor, you can use several `entry` options. Declare each **name** used (with `.global`) in a different memory. If no `--entry` option is given for a particular memory type, an entry address of zero is assumed.
`-l` **filename**	Gives the name of an object library created by CFLIB or by concatenating object modules. Only those modules that are needed to resolve previous external references are linked. Library files are read in the order they appear on the command line, and each library is searched sequentially; CFCC decides whether or not to link a particular object module based on the symbols it has already seen. Thus the order of libraries on the command line, and the order of object modules within a library file, can be significant. If the **filename** is a relative name (i.e. it does not begin with a backslash or a forward slash or a Windows/DOS drive letter followed by a colon), then CFCC looks for it first in the current working directory. If it cannot find it there, then it looks in the directories (if any) given with the `-L` option, searching the directories in the order they were given on the command line, from left to right. If the library is still not found, and if the CFLIB environment variable is defined, then it looks for the library in the directories named in the CFLIB environment variable, from left to right.

TABLE 23: COMMAND LINE OPTIONS (CONTINUED)

CFCC Option	Description
`-L` **`dir`**	This option adds the directory **`dir`** to the list that will be searched by the linker to find libraries.
`-f` **`name`**	This option creates an external reference to the symbol **`name`**, equivalent to one created by a `.extern` assembler directive, before any object files or libraries are read. You can use this option to force an object module containing a global definition for the symbol to be linked from a library. This option permits linking to take place entirely from libraries when no object files are given on the command line.
`--memN` **`SSSS`**/**`EEEE`** `--romN` **`SSSS`**/**`EEEE`**	This option is used to specify an area of RAM or ROM memory in which the linking process can allocate space. Use `--rom` to create ROM. The `--mem` option creates RAM. `N` must be a number, which is taken as the memory type. Possible values for `N` are 0 (for program memory), 1 (X data memory), 2 (Y data memory), and 3 (XY data memory). **`SSSS`** and **`EEEE`** must be numbers; they are the starting and ending addresses, respectively. All numbers can be C-style decimal, octal, or hexadecimal constants. More than one area of memory can be specified for each memory type. Two areas of a given memory cannot overlap. If you do not specify any memory area options, CFCC assumes that each memory type has an address space of 64 Kwords of RAM available starting at 0.
`--nmemN` **`SSSS`**/**`EEEE`** `--nromN` **`SSSS`**/**`EEEE`**	These options produce the same results as the `--memN` or `--romN` option, respectively, except that CFCC first clears any previous memory specifications for the memory type. This feature allows you to override any preconfigured memory configuration information.

TABLE 23: COMMAND LINE OPTIONS (CONTINUED)

CFCC Option	Description
`--overlap` **`seg1`**`/`**`seg2`**	Allows the segments named **`seg1`** and **`seg2`** to use the same physical memory locations. By default the double use (or *overlapping*) of one or more physical memory locations—possibly through the use of XY memory or the HEAR Configurable Accelerator's interleaved memories that are mapped into X memory—is detected by the linker's memory-checking phase, causing the link stage to fail. The `--overlap` option tells the memory checker to allow overlaps between two named memory segments, which will presumably be used at different times during run-time so that their use of the same physical memory locations will not cause run-time problems.
`--no-memcheck`	Disables checking for overlapped use of physical memory locations; CFCC does not run the linker's memory-checking phase. By default, the memory checker ensures that XY memory segments do not use the same locations as X or Y memory and that segments using the HEAR Configurable Accelerator's interleaved memories do not use the same physical locations in X memory twice.

7.2.2 CFLIB Environment Variable

To find the names of directories containing libraries, CFCC looks for the CFLIB environment variable. CFLIB contains the name of the directory to search for libraries.

You can provide more than one library directory in the CFLIB environment variable. Use a semicolon (`;`) as the delimiter to separate the names. The convention here is the same as that used for the PATH environment variable.

For backwards compatibility, if CFLIB is not defined in the environment, CFCC looks for the MCLIB environment variable and uses it in the same way as CFLIB.

Note: You can set this environment variable when you start the **EDK SignaKlara Command Prompt** for this EDK version. To view this and other settings, type `set` at the **EDK SignaKlara Command Prompt**.

7.3 ORDERING OBJECT MODULE LIBRARIES

The order in which object modules are placed in a library can be very important, because CFCC searches modules in the order that they appear in the library. The order in which libraries are specified on the command line can also be very important, because the linking process searches the libraries in the order that they appear on the command line. As a result, if a symbol is defined multiple times, CFCC always uses the first definition encountered, rather than any subsequent definition(s).

7.4 MEMORY CONFIGURATION

Use the `--memN` and `--romN` options to establish any limits on the addressing ranges allowed in the memories in your system. If you do not provide any such options, then CFCC assumes that all memory types have a 64 Kword address range (from `0` to `0xFFFF`). If your memory address ranges fall outside of the default, you will have to use `--memN` options.

If your memory address ranges fall within the default limits, no memory configuration is necessary. However, it is a good idea to configure your exact limits to give CFCC the ability to detect when it has run out of memory to allocate.

Note: It is assumed that memory type `0` (`--mem0`) is used for program memory, memory type `1` (`--mem1`) is used for X data memory, memory type `2` (`--mem2`) is used for Y data memory, and memory type `3` (`--mem3`) is used for XY data memory.

7.4.1 Memory Allocation for SK3 Systems

The flags that are used to tell CFCC about the memory layout of the SK3 system are as follows:

```
--mem0 0x1000/0x3FFF
--mem0 0xE000/0xE2FF
--mem0 0xFFE0/0xFFFF
--mem1 0x0000/0x17FF
--mem1 0x8000/0x93FF
--mem1 0x9F00/0xFFFF
--mem2 0x0000/0x07FF
--rom0 0x0000/0x07FF
--rom0 0x8000/0x87FF
--rom1 0x7800/0x787F
--rom1 0x9400/0x97FF
```

In this example, note that:

- Memory space 0 is program memory or PRAM; the `mem0` specifier designates 12 Kwords of PRAM starting at address 0x1000, memory-mapped control registers, and the interrupt vector table.
- Memory space 1 is X data memory or XRAM; the `mem1` specifiers designate the 6 Kword regular data memory area, and the data memory shared with the HEAR.
- Memory space 2 is Y data memory or YRAM; the `mem2` specifiers designate the 2 Kword regular data memory area.
- The rom0 specifiers are for the read-only Program ROM and the HEAR program memory, which are mapped into the Program memory space. The HEAR program memory is RAM, but is treated as ROM by the tools to separate it from the CFX PRAM, and to indicate that it contains HEAR microcode that the CFX cannot execute. For more information about the HEAR and related software tools, see the *HEAR Configurable Accelerator Reference Manual*.
- The `rom1` specifiers describe the areas of X memory space where ROM lookup tables reside.

However, in actual usage, it is not a good idea to give the flags exactly as shown above, to describe the whole system memory layout to CFCC, because:

1. Most programs never use many of the above sections of memory. There is no practical use for accessing the Program ROM section, and the usual method for accessing the control and

configuration memory is by loading pointer registers with the address constants that are defined in <sk3_hw.cfi>.

2. Part of each memory space is usually reserved for ON Semiconductor utility software, such as the Bootloader, the EEPROM library, the debugger, and CTK or LLCOM. ON Semiconductor reserves the low 32 words of X and Y memory, and the high 1 Kword of Program memory, for these purposes.
3. The CFX supports reading and writing 48-bit values from XY memory, consisting of a word from each of X and Y memory. Many programs will set aside a portion of X and Y memory to treat as XY memory. In the example below, 256 words of X and Y memory (from 0x0020 to 0x011F) are set aside as XY memory using the mem3 specifier, but you are free to change this partitioning. Take care that the XY memory region does not overlap with either the X or Y memory regions.

For these reasons, the recommended options for CFCC with the hardware are as follows:

```
--mem0 0x1000/0x3BFF
--mem0 0xFFE0/0xFFFF
--mem1 0x0120/0x17FF
--mem1 0x8000/0x93FF
--mem1 0x9F00/0xFFFF
--mem2 0x0120/0x07FF
--mem3 0x0020/0x011F
--rom0 0x8000/0x87FF
```

7.5 SPECIAL SYMBOLS

The segment called __mem_next_free_locs__ is a special segment in which CFCC keeps the symbols you can use in your code to determine the highest next free location, and the minimum and maximum memory address in any memory type. The special symbols are:

```
__endN_
__maxN_
__min_ramN_
__max_ramN_
__end_ramN_
__min_romN_
__max_romN_
__end_romN_
```

where N is 0 to 31. CFCC only generates these special symbols for memory types that have been configured using --memN options. Since SK3 programs only use memory types 0 to 3, you will only see the corresponding special symbols. CFCC uses memory type 7 to store debug symbols, so special symbols for memory type 7 are also generated.

The number in the name indicates the memory type with which the symbol is associated. The address of the symbol __end1_ is the highest next free location for memory type one. The addresses of the symbols __end_ram1_ and __end_rom1_ are the highest next free locations of RAM and ROM respectively for memory type one. The highest address for memory type one is the address of __max1_, and the addresses of __max_ram1_ and __max_rom1_ are the highest addresses for RAM and ROM respectively for memory type one. The addresses of __min_ram1_and __min_rom1_ specify the lowest addresses for RAM and ROM respectively for memory type one.

CFCC also defines two symbols for each segment. For a segment `x`, the symbol `__sb_x` has the address of the start of the segment, and the symbol `__se_x` has the address just past the end of the segment (that is, just past the last occupied address).

7.6 LOAD MAP FORMAT

The load map is a text file, each line of which describes one segment, symbol, or comment. The entries are displayed in ascending order of address within memory type, with each segment entry immediately followed by the entries for the symbols within that segment.

The segment line has the format:

```
SG <name> <type> <start> <end>
```

where:

- `SG` identifies the line as a segment line
- `<name>` is a sequence of non-blank characters giving the name of the segment
- `<type>` is a constant giving the memory type of the segment (i.e., 0 for program memory, 1 for X data memory, 2 for Y data memory, and 3 for XY data memory)
- `<start>` is a C-style hexadecimal constant giving the absolute starting address of the segment
- `<end>` is a C-style hexadecimal constant giving the absolute ending address of the segment

Blanks are used as delimiters.

A symbol line consists of:

```
SY <name> <address>
```

where:

- `SY` identifies the line as a symbol line
- `<name>` is a sequence of non-blank characters that is the name of the symbol
- `<address>` is a C-style hexadecimal constant giving the absolute address of the symbol.

You can compute the address relative to the start of the segment by subtracting the most recent segment start address.

Following the segment and symbol lines are comment lines. A comment line appears as follows:

```
; <text>
```

where `<text>` appears in an assembler `.comment` directive.

Here is a sample load map:

```
SG main 0 0x1000 0x1001
SY main 0x1000
SG xmem 1 0x120 0x124
SY testdata 0x120
SG __mem_next_free_locs__ 31 0x0 0x0
SY __min_ram0_ 0x1000
```

```
SY __max_ram0_ 0xffff
SY __max0_ 0xffff
SY __end_ram0_ 0x1002
SY __end0_ 0x1002
SY __min_ram1_ 0x120
SY __max_ram1_ 0xffff
SY __max1_ 0xffff
SY __end_ram1_ 0x125
SY __end1_ 0x125
SY __min_ram2_ 0x120
SY __max_ram2_ 0x7ff
SY __max2_ 0x7ff
SY __end_ram2_ 0x0
SY __end2_ 0x120
SY __min_ram3_ 0x20
SY __max_ram3_ 0x11f
SY __max3_ 0x11f
SY __end_ram3_ 0x0
SY __end3_ 0x20
SY __min_ram7_ 0x24
SY __max_ram7_ 0xffffffff
SY __max7_ 0xffffffff
SY __end_ram7_ 0x0
SY __end7_ 0x24
```

The symbol records for all symbols defined within that segment follow each segment record. The last segment in the list contains the special symbols generated by the linking process to allow you to get the next highest address that is available for use.

7.7 CONVERTING TO .CFO FORMAT

CFCC produces executable files in the binary *Absolute* format (see Section A.2, “Absolute Executable Format” on page 115 for more information). Since this is a binary format, it is not easy for humans to read nor readily usable in simple data manipulation scripts. For times when a textual representation of data is necessary, use the `.cfo` file format. For information on how to convert to the `.cfo` format by using the IDE, see the *Integrated Development Environment User’s Guide*.

The `.cfo` format consists of one or more data lines in the following form:

```
M  AAAA DDDDDDDD
```

where:

- `M` is the memory space, specified as one of the following:
  - `P` for program memory
  - `X` for X data memory
  - `Y` for Y data memory
  - `R` for non-volatile storage, such as EEPROM.
- `AAAA` is the address specified in hexadecimal.
- `DDDDDDDD` is the data, also specified in hexadecimal. If the memory space is narrower than 32 bits, the values are padded on the left with spaces.

Here is an example of a program consisting of a single NOP instruction in `.cfo` format:

```
P  1000 00000000
```

The ABSDUMP utility is provided to convert CFCC's binary executable format into `.cfo` format. ABSDUMP ignores uninitialized memory regions created with the `.bss` directive; nothing is written to the `.cfo` file for such regions.

7.7.1 ABSDUMP Usage

ABSDUMP can be run from the **EDK SignaKlara Command Prompt** for this EDK version using the following syntax:

```
absdump [option]... file...
```

where:

- `file` specifies the file to read as input. It must be an executable file produced by CFCC's linking stage.

Table 24 describes the command line options for ABSDUMP.

TABLE 24: ABSDUMP COMMAND LINE OPTIONS

Option	Description
`-o` **`file`**	Writes output to **`file`**. If this option is not given, output is written to standard output.
`-i`	Dumps more information from the executable file.
`-c`	Dumps more information from the executable file in a compact format.
`-d`	Does not show the `.cfo` format data. When used with `-i` or `-c`, this displays information from the executable file but not the data from code or data segments.

This page left blank for printing purposes.

8 Creating Configurable Programs

8.1 INTRODUCTION

Programs in the absolute executable format (see Appendix A, “Absolute Executable Format” on page 115 for the details about the format) can be configured using parameter definitions to enable data substitution without the need for recompilation of the `.abs` output file.

Parameters are defined by an EDK developer (someone accustomed to programming in the EDK environment) as part of the program development phase. The parameters are based on symbols in the program and are stored in a configuration template file. A configuration template is associated with a particular program by loading it or creating it in the context of the **Absolute File Viewer**. This feature is available in the IDE.

A SignaKlara Device Utility (SKDU) user can open a program and its associated configuration template, set parameter values, and export the program to make the required data substitution. The SKDU is designed to allow users to customize and distribute the program without knowing how to program. For more information on how to use the File Parameterization feature of the SKDU, refer to the *SignaKlara Device Utility User’s Guide*.

8.2 CONFIGURATION TEMPLATES

Configuration templates for `.abs` files have a `.abst` file extension. A given template file in XML format specifies a list of parameters contained in a particular `.abs` file (the target) and optional utility commands for use with supported devices. Refer to Appendix B, “Configuration Template Schema” on page 137 for details regarding the schema definition.

Templates are created by an EDK developer in the IDE via the **Absolute File Viewer.**

8.2.1 Create a Configuration Template

To create a new configuration template in the IDE:

1. Open your `.abs` file in the **Absolute File Viewer** by double-clicking the file in the **Project Explorer**. You will see a view similar to Figure 2.

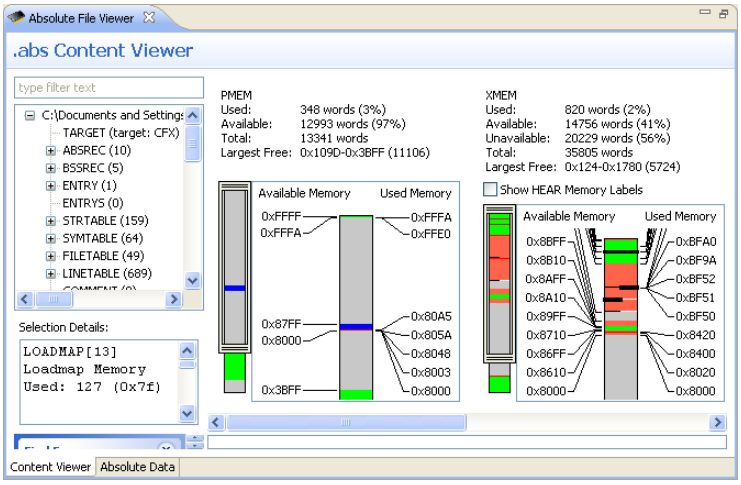


Figure 2: Absolute File Viewer

2. In the menu at the top of the workbench window, select **Configurations > New Configuration Template...**
3. Enter a name for the new configuration template when prompted and click **OK**. The Configuration Template Editor opens as shown in Figure 3.

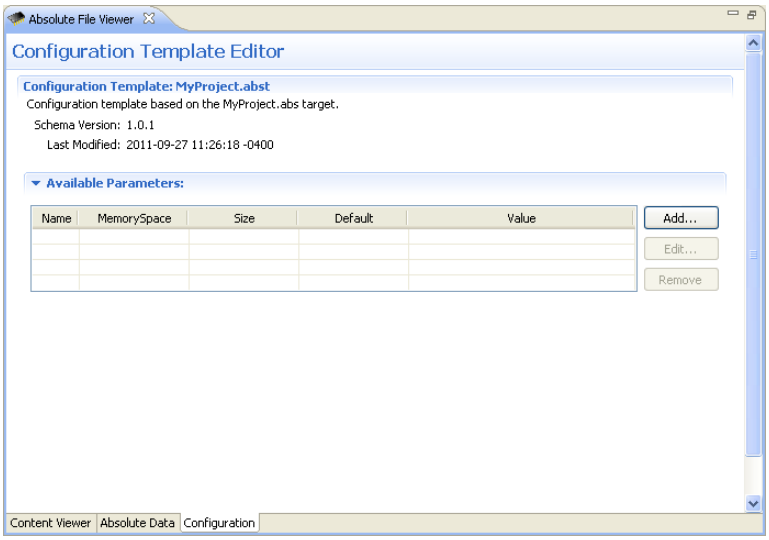


Figure 3: The Configuration Template Editor

The configuration template is created in the same folder as the `.abs` file that is associated with it.

8.2.2 Saving Configuration Template Data

When changes are made to the configuration template, the **Absolute File Viewer** shows an asterisk in the tab at the top of the view indicating that changes need to be saved. To save changes to your configuration template in the original `.abst` file, select **File > Save**.

Alternately, to save your changes to a new `.abst` file, leaving the original file unchanged:

1. Select **File > Save As**.
2. Specify the new `.abst` file to save your changes and click **Save**.
   Note: If you select a file that already exists, you are prompted to overwrite the file with the current changes.

8.2.3 Loading a Configuration Template

To load an existing configuration template in the IDE:

1. Open your `.abs` file in the **Absolute File Viewer** by double-clicking the file in the **Project Explorer**.
2. In the menu at the top of the workbench window, select **Configurations > Load Configuration Template...**
3. Use the file dialog to select the template and click **Open**.

The configuration template is applied to the open `.abs` file.

IMPORTANT: Configuration templates are `.abs` file specific. Upon loading, the template is validated against the open `.abs` file to ensure consistency between the parameters defined in the template and the symbols present in the `.abs` file. Opening a template in the context of the wrong `.abs` file results in warnings, and exporting data from the template yields unpredictable results.

8.3 PROGRAM PARAMETERS

Program parameters are used to define regions within a program that can support data substitution. Parameters are defined based on corresponding symbols in the load map records of the associated `.abs` file.

Parameters have the following attributes:

name — An optional name that can be assigned to the parameter to provide a more user friendly reference. By default, this is set to the value *symbol*.

symbol — The name of the symbol in the `.abs` file load map that the parameter is based on.

group — An optional category that can be assigned to the parameter for use in filtering parameters within the Configuration Template Editor. Parameters that share the same group are displayed together in a list when the group filter is selected; other parameters are hidden.

wordcount — The size (in words) of the parameterized data. By default, this corresponds to the size of the data defined by the symbol in the `.abs` file load map.

wordsize The size (in bits) of a single word of data in the parameter. By default, this corresponds to the wordsize defined by the symbol in the `.abs` file load map.

In addition, parameters can be configured as follows:

Read Only No value settings are available. The default value is used in data substitution.

Hidden Not displayed with the list of available attributes. Since they cannot be selected from a list, the default value is used in data substitution.

Parameter definitions include two values:

default The fixed, type-specific, default value for the parameter. By default, this value is the initialized data defined by the symbol in the load map record of the `.abs` file on which the parameter is based.

Note: If a parameter is based on a symbol that represents an area of uninitialized data, you must manually provide a default value for the parameter because no symbol data is available.

value The optional, type-specific, user-defined value for the parameter. If no user value is defined for a parameter, the default is used in the event of data substitution. The process of setting user-defined values is described in Section 8.3.5, “Setting Parameter Values” on page 75.

Note: If a parameter is configured as *read-only*, setting a user-defined value is prohibited, and the default value is always used for data substitution.

Parameters support the following methods of data substitution:

Memory Reference Substitution by memory reference is supported for all parameters in a `.abst` file. This method uses the memory space and address derived from the symbol in the `.abs` file load map that the parameter is based on to define the area for substitute data to be written. The memory reference substitution method is used to export parameterized data in the configuration template, along with the original data from the `.abs` file as described in Section 8.6.1, “Exporting Absolute Executable Format” on page 80.

I^2C Reference Substitution by I^2C is configurable as an option on all parameters in a `.abst` file. This method requires the assignment of a virtual register index for the parameter along with the definition of both a read and write command (specified as hexadecimal command numbers) to update the program parameter at runtime on the device via the I^2C protocol as described in Section 8.3.5.2, “I2C Updates” on page 76.

Note: To use the I^2C update method for runtime parameter configuration, your application must support the I^2C runtime configuration application programming

interface (API). For more information, refer to Section 8.5, “DSP Implementation Requirements for I2C Runtime Configuration Support” on page 78.

8.3.1 Parameter Types

A number of parameter types are supported to provide flexibility in the definition of parameter values.

Note: The same parameter type applies to both the default and the user-defined value in a given parameter definition.

The following is a list of the supported parameter types:

Numeric Value (no restrictions)

Unrestricted numeric parameters can accept any of the typical numeric formats, including decimal, octal and C-style hexadecimal, as integer values only. No decimal points nor exponents are allowed. Numeric values are represented as signed word values corresponding to the parameter’s word size. For multi-word parameter values, separate words with commas.

Numeric Value (restricted with min/max bounds)

Restricted numeric parameters can accept any numeric value between a specified minimum and maximum value. Specify values in any of the typical numeric formats, including decimal, octal and C-style hexadecimal. Numeric values are represented as signed word values corresponding to the parameter’s word size. For multi-word parameter values, separate words with commas. The min/max restriction on user defined values is enforced on a per word basis.

Enumerated Value

Enumerated parameters use pre-defined options to restrict user specified values. Options are specified by an arbitrary name and an associated numeric value. Specify numeric values using any of the typical numeric formats, including decimal, octal and C-style hexadecimal. Numeric values are represented as signed word values corresponding to the parameter’s word size. For multi-word parameter values, separate individual words in the enumerated value with commas.

External Data

File parameters use an external data file to provide the data to be used for substitution. The data file must use the relative data format (*.d) with a single column of data words specified in hexadecimal format. The value for the parameter is the path to the data file relative to the configuration template file.

8.3.2 Adding Parameters

EDK developers can use the Add Parameter Wizard in the Configuration Template Editor to add parameters to a configuration template. To open the wizard, click on the button labeled **Add...** beside the list of available parameters (see Figure 3 on page 68). The first wizard dialog box will look similar to Figure 4.

Note: The Add Parameter Wizard in the SKDU cannot add parameters to a configuration template.

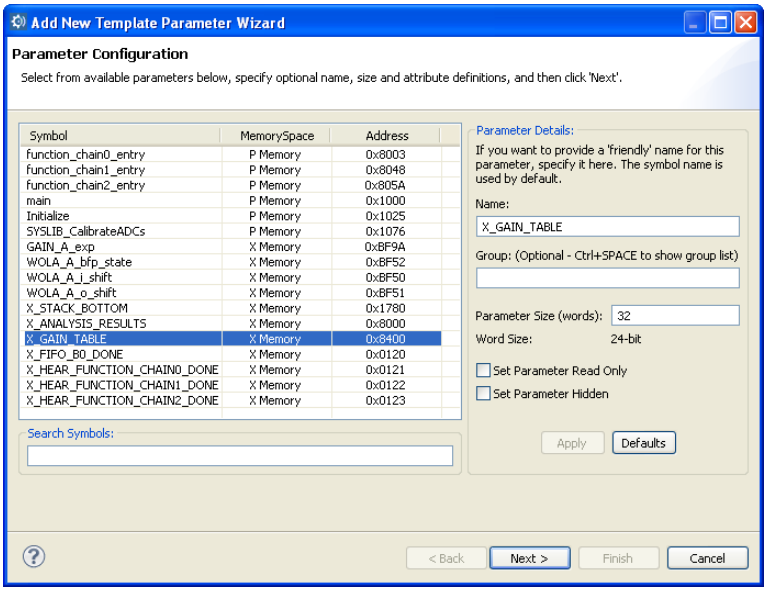


Figure 4: Add Parameter Wizard

With the wizard open, you can configure a parameter by following these steps:

1. Select the symbol that defines the data you want to parameterize from the table on the left. If the table contains many symbols, you can use the **Search Symbols** field below the table to help you find the one you're looking for. Start typing the name of the symbol and the table will display only the filtered results that match your search by case-sensitive comparison. When a symbol is selected, the default parameter details for the selected symbol are displayed beside the table.
   Note: Symbols are obtained from the load map records within the current `.abs` file. Refer to Section A.2.12, "LOADMAP Header and Data Records" on page 121 for details on load map records in the absolute executable format.

2. The name of the parameter is set to the symbol name by default. If you would like to change this to something more meaningful, you can enter a new name in the **Name** field.
3. Use the **Group** field to provide an optional group label that can be used to filter parameters in the table of available parameters in the Configuration Template Editor. Applying the same group label to multiple parameters causes them to be displayed together when the filter corresponding to their group label is applied. All parameters without a group label, or with a group label that does not match the selected filter, will be hidden from view.
4. Optionally, change the default parameter size. The default parameter size corresponds to the actual size of the data defined by the selected symbol. If the area is uninitialized but allocated, the default parameter size is set to the full size of the allocated area. If the actual

data length cannot be determined, such as for an unallocated area of uninitialized data, the default parameter size is set to 1.

IMPORTANT: The parameter size can be set to an arbitrary value, which allows you to parameterize a range of data other than that referenced by the actual symbol. Use caution when changing the parameter size, because it can lead to undesirable results during data substitution.

5. Notice the Word Size, which is derived from the symbol in the `.abs` file. It is provided for reference only.
6. Check the **Set Parameter Read Only** check box to restrict the value of the data used in data substitution to the default value defined by the parameter. In this case, user settings for parameter data values are not permitted.
7. Check the **Set Parameter Hidden** check box to hide the parameter from view in the list of available parameters contained in the Configuration Template Editor. The parameter will still be used in data substitution, where the default value is used in the absence of a user value.
8. Click **Next** to proceed to the parameter definition.
   Note: You must click **Apply** to commit any changes to the parameter details before you can proceed.

9. Select a parameter type from the drop down list on the Parameter Definition page. Refer to Section 8.3.1, “Parameter Types” on page 71 for details regarding supported types.
10. Once you have selected a type, a type-specific default value is provided. This value is derived from the data referenced by the symbol being parameterized. You can use this default value (recommended) or provide your own value conforming to the same type format.

IMPORTANT: If the symbol being parameterized refers to an area of uninitialized data, no default value is provided by the wizard. If no default value is provided automatically, you must provide a value manually before proceeding. The default value must correspond to the size of the data area referenced by the symbol, and must be in the format specified by the selected type.

11. Click **Next** to proceed to the update method configuration.
12. Notice the Target Update Settings that are provided for reference. This section indicates the memory space and corresponding address derived from the symbol in the `.abs` file. This information is embedded in the parameter and will be used to perform substitution via data export.

IMPORTANT: To ensure accurate data substitution, the memory space/address information for each parameter must be in sync with the corresponding `.abs` file. The Configuration Template Editor will respond to any changes in the workspace that affect the target `.abs` file (e.g., recompilation, deletion) by attempting to re-sync all defined parameters with their referenced symbols. If the targeted `.abs` file has moved or no longer exists, you are prompted to select a new target for the template. If a parameter is defined for a symbol that no longer exists, a warning is displayed. If the configuration template is not open when the `.abs` file is changed/removed, it cannot re-sync. The configuration template attempts to re-sync and validate the contained parameters the next time it is opened.

13. Click the **Enable I2C Update Method** check box to configure I²C update settings for the parameter.

IMPORTANT: To use the I²C update method, your application must support the I²C runtime configuration API. For more information, refer to Section 8.5, "DSP Implementation Requirements for I2C Runtime Configuration Support" on page 78.

14. Click **Next** to display a summary of your parameter configuration settings, and **Finish** to complete the parameter configuration, or just click **Finish** if you do not want to review the summary.

Once you have completed the parameter configuration in the wizard, your new parameter is listed in the available parameters section.

8.3.3 Viewing Parameter Properties

You can view the complete parameter configuration details at any time by opening the parameter Properties dialog for the selected parameter as follows:

1. Select the parameter listed in the Available Parameters section.
2. Right click and select **Show Properties**.

The parameter details are displayed in the Properties dialog as shown in Figure 5. Click **Close** to dismiss the dialog.

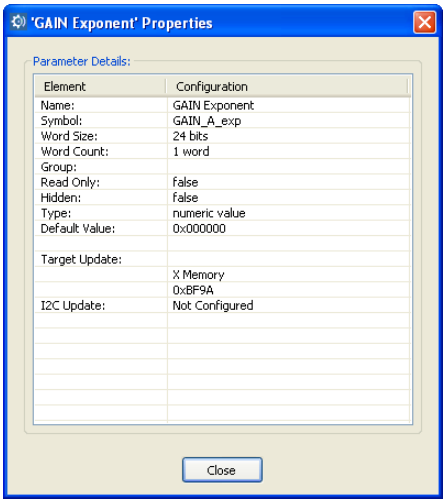


Figure 5: Properties for an Available Parameter

8.3.4 Editing Parameters

EDK developers can use the Edit Parameter Wizard in the Configuration Template Editor to edit existing parameters in a configuration template. To use the wizard:

1. Select the parameter you want to edit from the list of available parameters.

2. Click on the button labeled **Edit...** beside the parameter list (see Figure 3 on page 68). The first wizard dialog box will look similar to Figure 6.

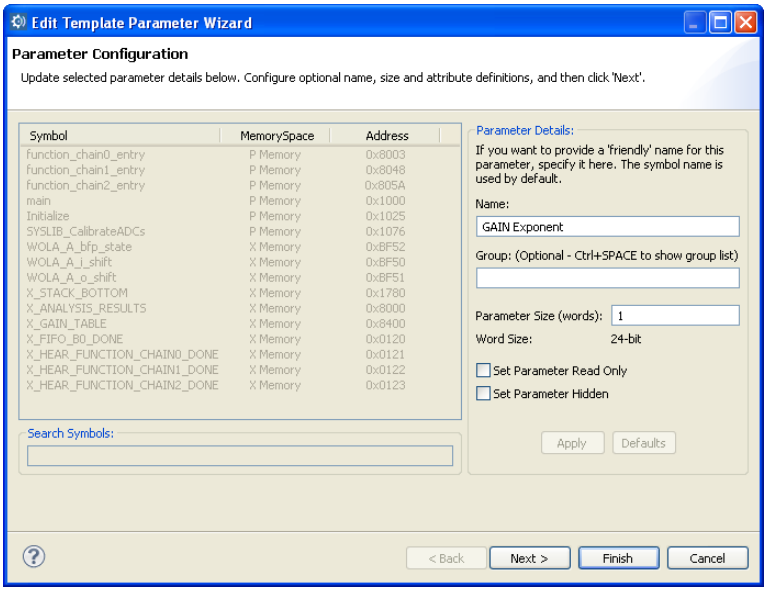


Figure 6: The Edit Template Parameter Wizard

Note: The Edit Parameter Wizard in the SKDU cannot edit parameters in a configuration template.

3. With the wizard open, you can make changes to the original parameter configuration using the same steps that you followed to add the parameter initially (refer to Section 8.3.2, “Adding Parameters” on page 71).
4. Once you have completed your changes to the parameter configuration, click **Finish** to commit your changes and close the wizard. At any time you can click **Cancel** to close the wizard and leave the original parameter configuration unchanged.

8.3.5 Setting Parameter Values

Parameter value settings are configured in the Parameter Details section of the Configuration Template Editor. To view the details section for a given parameter, select the parameter in the available parameters section. The Parameter Details section is displayed at the bottom of the page.

Note: Any errors or warnings generated by invalid parameter values are displayed in three places: the top of the Configuration Template Editor, the Available Parameters list (beside the parameter name), and in the Parameter Details section (if the parameter is selected). To view the details of the error/warning, mouse over the error/warning symbol at the top of the editor or in the Parameter Details section.

8.3.5.1 User Defined Values

In the Parameter Details section, user defined values can be configured for a parameter in the format corresponding to the given parameter type.

To set a user value for the parameter that will override the default value in the event of data substitution:

1. Specify (or select, as appropriate) a new value in the **Value** field.
2. Click **Apply** to commit the parameter value configuration to the template.

You can click **Clear** to remove an existing user value for the parameter (leaving only the default value).

8.3.5.2 I^2C Updates

If a parameter is configured with support for runtime configuration via the I^2C update method, the details section provides controls to run the I^2C commands that read and write the current parameter value.

Note: I^2C update command controls are only enabled in the SKDU. EDK developers cannot run I^2C update commands from the Configuration Template Editor in the IDE.

To update the parameter value in the configuration template with the value obtained from the running application via I^2C on the device, click **I2C Read**. The value is read from the virtual register table at the index defined by the given parameter. (For more information about virtual register tables, see Section 8.5, "DSP Implementation Requirements for I2C Runtime Configuration Support" on page 78.) Multiple word values are read from consecutive indices starting with the index specified. The value is updated in the **Value** field and automatically applied to the template (you do not need to click **Apply**).

Note: I^2C can read read-only parameter values, but they are not applied to the configuration template. In this case, I^2C Read executes the read command. The running application on the device responds with a parameter value, and that parameter value is displayed in the console view.

To write the value of a parameter contained in the configuration template to the running application via I^2C on the device, click **I2C Write**. The value is written to the virtual register table at the index defined by the given parameter. Multiple word values are written to consecutive indices starting with the index specified.

8.3.6 Removing Parameters

EDK developers can remove existing parameters from the list of available parameters in the Configuration Template Editor.

Note: The Configuration Template Editor in the SKDU cannot remove parameters.

To remove a parameter in the Configuration Template Editor:

1. Select the parameter from the table containing the list of available parameters.
2. Click the **Remove** button to the right of the table to remove the parameter.

8.4 GENERAL I2C COMMANDS

The Configuration Template Schema provides a section for the definition of general I2C commands. These commands are typically independent of any specific parameter and are used to execute specific functions on a device that supports I2C runtime configuration. Refer to Section B.3, "Schema" on page 137 for complete details of the commands section.

Note: I2C commands are only enabled in the SKDU. EDK developers cannot execute general I2C commands from the Configuration Template Editor in the IDE.

Commands have the following attributes:

name The name of the command. This attribute is displayed in the list of available general I2C commands and used to select the command to execute.

desc An optional description of the command. If present, this attribute is displayed to provide additional information (e.g., usage, required arguments) for the selected command. You can format the contents of this attribute using tab (\t) and newline (\n) characters as required.

address An optional I2C slave address to be used for command execution. By default, the SKDU uses the I2C address specified in the CTK configuration used by the selected device connection. If you specify this attribute, it overrides the default I2C address for the command.

code The command number as specified by the I2C runtime configuration application programming interface (API). Refer to Section 8.5, "DSP Implementation Requirements for I2C Runtime Configuration Support" on page 78 for details about the command handling.

txbytes The size in bytes of the data arguments for the I2C command.

rxbytes The size in bytes of the data response returned by the I2C command.

formatter An optional attribute that, when specified, can be used to parse the response bytes of an I2C command into a more descriptive, formatted output.

The format string contained in the attribute should use the following syntax. Note that multiple formats are delimited by semicolon:

```
[bm|B|wm]!n!"format"!"name1=value1,name2=value2..."
```

The details of the command formatter syntax are outlined in Table 25.

TABLE 25: I2C COMMAND FORMATTER SYNTAX

Syntax	Description
b**m**	Apply format string to bitfield of length **m** (in bits)
B	Apply format string to data byte
w**m**	Apply format string to data word (wordsize **m** in bits)
n	The 0 relative start point (in bits for type b, otherwise in bytes) for the response data format
format	The string format (use Python print formatting conventions)
name=value	Value pairs (comma delimited) to be used in parenthetical substitution

The following default names are defined:

- *data*—the numeric value of the data contained in the bitfield, byte or word to which the formatting is being applied
- *alt*—if a name/value pair is defined where **name** is equal to **data** (as defined above) then *alt* is defined with the given value. If *alt* is undefined but included in the format string, NA will be displayed.

Examples:

```
b3!7!"Bitfield of length 3 starting at bit 7 as decimal
field : %(data)d"
w16!0!"Entire 16 bit word as 4 digit Hex value:
0x%(data)04X"
b1!0!"First bit is %(data)d. In a word, that would be
%(alt)s"!"0=zero,1=one"
```

chain An optional list of operation codes corresponding to one or more commands that are to execute on the completion of the current command. Specify codes in the same format used by *code*, with multiple codes separated by commas.

Note: Commands corresponding to the specified codes must exist in the list of general I2C commands defined within the template. If no command exists for a specified code, it is skipped in the chain of execution. Only commands requiring no input (attribute txbytes=0) can be included in the chain element.

8.5 DSP IMPLEMENTATION REQUIREMENTS FOR I2C RUNTIME CONFIGURATION SUPPORT

I2C runtime configuration in a DSP application (both parameter updates and general commands) must implement an application programming interface (API) to handle the specific I2C commands.

While the implementation details of this API are not strictly specified, the clients that use this interface (such as the SKDU) make certain assumptions about how I²C runtime configuration support is implemented in a DSP application. This section describes the API requirements.

Implementation of a "Virtual Register Table"

I²C runtime configuration of parameters (reads and writes) assumes that parameter values can be read from and written to an area of memory indexed by a non-negative, 16-bit integer value (specified in little-endian format). For multi-word parameter values, the data is read from and/or written to consecutive indices starting at the specified index.

Command number handling

Each I²C command must contain a unique byte value command number specified as two hexadecimal digits (in the range 0x00 to 0xFF inclusive). The DSP implementation must provide a handling mechanism for all command numbers supported by your application. Additionally, we recommend implementing a command handler to set an error condition for an unrecognized command, and a status handler to update the application status and expose it via the API. These items are not required, but can be helpful when you are debugging.

Command parameter/response handling

I²C commands are sent from the client using the CTK SendReceive method provided via the Protocol interface (refer to the *CTK API Reference Guide* for details on the SendReceive method). This method sends and receives data via the protocol using two data buffers implemented as byte arrays: one for sending and one for receiving. The sizes of the respective byte arrays are provided as additional arguments to the method. Generic commands differ from parameter update (read/write) commands in the structure of the data sent. The typical structures of commands are described in Table 26, Table 27, and Table 28.

TABLE 26: TYPICAL STRUCTURE OF I²C GENERIC COMMAND SEND DATA

Byte Offset	Description	Details
0	Command Number	The unique byte value that identifies the command
1	Command Parameter Data	The arguments expected by the command handler. The size of the data corresponds to the value of `txbytes` specified in the command.

TABLE 27: TYPICAL STRUCTURE OF I²C PARAMETER UPDATE COMMAND SEND DATA

Byte Offset	Description	Details
0	Command Number	The unique byte value that identifies the command

TABLE 27: TYPICAL STRUCTURE OF I2C PARAMETER UPDATE COMMAND SEND DATA (CONTINUED)

Byte Offset	Description	Details
1	Command Parameter Index	The 16-bit index specifying the parameter's location in the virtual register table. This value corresponds to the I2C index in the parameter definition. You must specify the index in the same order that the DSP application expects.
3	Command Parameter Data	The arguments expected by the command handler. For a "write parameter" update command, the size of the data must match the size of the parameter data value, and the order must match what the DSP application expects. This area is empty for "read parameter" update commands.

TABLE 28: TYPICAL STRUCTURE OF I2C COMMAND RESPONSE DATA

Byte Offset	Description	Details
0	Command Response Data	The data returned by the command handler. The size of the data corresponds to the value of `rxbytes` specified in the command. For "read parameter" update commands, this also corresponds to the size of the parameter data value.

I2C slave address specification

Your device probably requires you to use an I2C slave address to handle commands. Include the `address` element with a value corresponding to the slave address for all commands issued from the client. See Table 53 on page 140 for a description of the `address` element.

If no `address` element is specified, the client uses the debug address by default.

8.6 EXPORTING CONFIGURATION DATA

You can export the parameterized data substitutions defined in a configuration template to create one of the following for redistribution:

- A new `.abs` file containing the substituted data in absolute executable format
- A bundled format containing the target `.abs` file and one or more configuration templates

8.6.1 Exporting Absolute Executable Format

To export configuration data in absolute executable format:

1. Make sure that the Absolute File Viewer is open with the configuration template loaded and any changes to parameter values saved.
2. Select the **Configurations > Export Configuration Template...** menu option to open the **Export Configuration Template Wizard**.
3. Select the **Export Absolute Executable Format (.abs) File** export type in the wizard's drop down list. See Figure 7 for an example of the dialog.

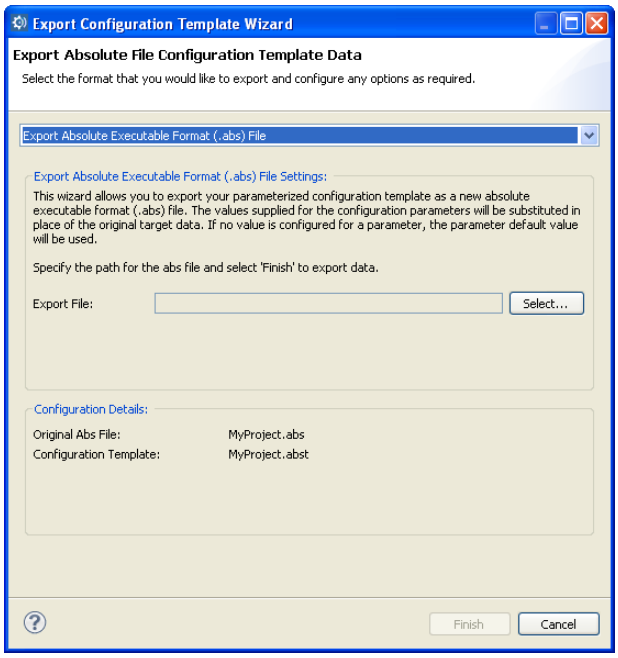


Figure 7: Exporting an Absolute Executable Format File

4. Follow the instructions provided by the wizard to configure the required export settings.
5. Click **Finish** to complete the data export.

Note: If user values have been set for the parameters in the template, those values are substituted in the appropriate locations of the new `.abs` target. Otherwise, the default values are used during exporting to absolute executable format.

8.6.2 Exporting Absolute Executable Bundle Format

To export target and configuration data in a self-contained, Absolute Executable Bundle Format:

1. Make sure that the Absolute File Viewer is open with the configuration template loaded and any changes to parameter values saved.
2. Select the **Configurations > Export Configuration Template...** menu option to open the **Export Configuration Template Wizard**.
3. Select the **Export Absolute Template Bundle Format (.absb) File** export type in the wizard's drop down list. See Figure 8 for an example of the dialog.

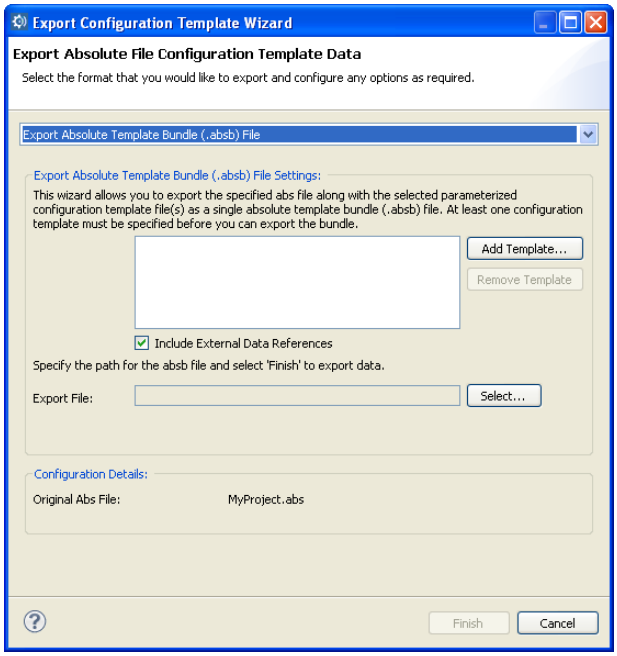


Figure 8: Exporting an Absolute Executable Template Bundle

4. Follow the instructions provided by the wizard to configure the required export settings.
5. Click **Finish** to complete the bundle export.

Note: The bundle file format creates a compressed file with the designated target (`.abs`) file and selected configuration template(s) at the root level. Any external data references are also stored at the root level of the bundle, so names of configuration templates and external data files are unique to avoid collisions. All internal path references within the template file are automatically updated to reflect the bundle structure.

9 Non-Volatile Memory Management

9.1 INTRODUCTION

The Non-Volatile Memory Manager (NVMM), which is integrated with the IDE, supports creating and viewing loadable, non-volatile images for the BelaSigna 300 EEPROM.

You can use the NVMM to generate bootable image files by providing a bootloader and a list of applications to be included in the image. NVMM arranges the sections of your applications for placement in non-volatile memory, and generates the appropriate structures for the BelaSigna 300 EEPROM. You can later load these image files to the BelaSigna 300 non-volatile memory using the download tool, NVMD (`nvmd.exe`).

Note: NVMM ignores applications' uninitialized memory regions (created with the `.bss` directive). Nothing is written to the non-volatile memory image for these sections, so they are not initialized by the bootloader.

Alternately, through the IDE, you can use the NVMM to upload an image contained in your device's non-volatile memory to an EEPROM image file. The EEPROM image file is called a Non-Volatile Image (NVI) file, and the extension is `.nvi`. The NVI file can be used to create an EEPROM project with derived inputs and build preferences.

Files formatted with the standard NVI format can be viewed using the NVI Manager, an IDE component based on the NVMM, which provides a comprehensive, graphical representation of the EEPROM and image layout.

For details on the BelaSigna 300 EEPROM, refer to the *Firmware Reference Manual for BelaSigna 300*. For details on the NVMD application, refer to Section 10.2, "Non-Volatile Memory Download Tool" on page 107.

9.2 FEATURES

NVMM supports the following features:

- Flexible Layout Arrangement
- Optimization through Automatic Equivalent Section Detection
- Image/Application Encryption
- Image Upload through the IDE
- Layout Visualization through the NVI Manager

9.2.1 Flexible Layout Arrangement

NVMM by default automatically arranges the moveable sections of the layout (applications and their sections, and the bootloader). While you are not required to specify addresses for each section, you can override this feature by specifying an explicit address. The automatic arrangement method is designed to make efficient use of the available space in EEPROM by placing sections in contiguous memory wherever possible.

9.2.2 Optimization Through Automatic Equivalent Section Detection

When multiple applications are present in a non-volatile image file, NVMM attempts to identify sections that are equivalent with respect to their data, their destination in memory, and any file system specific attributes/fields they might possess. By identifying equivalent sections, NVMM can include a single instance of the section in the layout and update other references to reuse that instance, thereby reducing the overall size of the resulting non-volatile image.

The implementation of this feature is file system dependent. For the BelaSigna 300 EEPROM file system, NVMM only performs automatic section matching if the section NVMM is trying to match is allowed to be freely placed. As such, NVMM does not attempt to match sections which have a defined address (either directly, by using **`@<address>`**; or indirectly, by defining an address for the application containing the section). However, if you explicitly use the `@auto` address specifier for a section, it enables equivalent section detection even if the containing application was explicitly placed.

9.2.3 Encryption Support

NVMM supports both Level 1 and Level 2 security in the BelaSigna 300 EEPROM:

- Level 2 security applies to applications and their respective program memory sections.
- Level 1 security adds an additional layer of security to the EEPROM image by encrypting the keys employed in Level 2 security as well as the bootloader.

For more detailed information on the differences between the two levels of security and their respective use cases, refer to the *Firmware Reference Manual for BelaSigna 300*.

By default, the non-volatile images created with NVMM are not secure. For more information on creating secure images using NVMM in the IDE, refer to Section 9.3.1.5, “Securing an EEPROM Image” on page 90.

9.2.4 EEPROM Image Upload

NVMM supports uploading an BelaSigna 300 EEPROM image through the IDE. The data in the EEPROM memory is read using a user-specified CTK configuration, and assuming the device contains a valid, bootable EEPROM image, is formatted using the standard NVI file format. You can open the resulting NVI file for viewing using the NVI Manager. For more information on viewing an uploaded EEPROM image, refer to Section 9.3.5, “Viewing an EEPROM Image with NVI Manager” on page 95.

9.2.5 Graphical Layout View Using NVI Manager

The IDE contains the NVI Manager, which is based on the NVMM. In the IDE, `.nvi` files open in the NVI Manager by default.

Note: NVI Manager opens `.nvi` files in read-only mode. For information on how to change the layout or contents of a `.nvi` file, see Section 9.3, “Using NVMM”.

9.3 USING NVMM

In the IDE, use one of the following methods to generate an NVI file that can then be loaded into the non-volatile memory on your BelaSigna 300:

- Create an EEPROM image using the project wizard.
- Create an EEPROM image by uploading it from an EEPROM.
- Create an EEPROM project (with EEPROM image output) from an existing NVI file.

Once you have a `.nvi` file, you can open it for viewing using the NVI Manager.

9.3.1 Creating an EEPROM Image Using the Project Wizard

You can build NVI files using NVMM inside the IDE via a special kind of project: an EEPROM project. To create an empty EEPROM project, use the standard new **SignaKlara Development Project** wizard. It creates a project that uses the NVMM builder to create the properly formatted image file for you, using the applications and options that you specify. For a full list of available options, and an explanation regarding their purpose and proper usage, refer to the Section 9.4, “Options” on page 98.

9.3.1.1 Creating an EEPROM Project

To create a new EEPROM project:

1. Select **File** > **New...** > **SignaKlara Development Project**.
2. Give the new project a valid name and click **Next**.
3. If **BelaSigna 300 Projects** is not already in the **Available Project Types** field, select it from the drop-down list.
4. Choose the **New BelaSigna 300 EEPROM Image Project** from the list of empty project templates.
5. Click **Finish**.

You now have a new EEPROM project in your workspace. If auto-build is enabled (**Project** > **Build Automatically**), NVMM builds your project and issues a warning because you have not yet defined an application.

9.3.1.2 BelaSigna 300 EEPROM Layout Defaults

The default BelaSigna 300 EEPROM layout possesses some characteristics that you must understand before attempting to transfer the layout to your device’s EEPROM:

- A BelaSigna 300 EEPROM layout *requires* a system area, key area, and a bootloader. Applications are optional, but you will be warned if no applications are present. Generally, at least one application is required to create a bootable EEPROM image.
- Layouts are not secure by default. That is, the default build settings for NVMM do not encrypt any of the applications or their respective program memory sections, the key area or the bootloader. Refer to Section 9.3.1.5, “Securing an EEPROM Image” on page 90 for more information on how to create secure images.
- By default, a BelaSigna 300 EEPROM layout does not contain a manufacturing area, but reserves this area of EEPROM to prevent data from being placed in this address range. A

manufacturing area can be created if required by loading the desired information from an arbitrary data file.

CAUTION: The manufacturing area contains device-specific calibration information. If you define your EEPROM layout such that it contains a manufacturing area, and you enable the overwriting of the manufacturing area by NVMD, any existing values in this area of EEPROM will be overwritten and therefore lost when the layout is transferred to the device. Make sure you back up your device-specific calibration values if you intend to supply your own manufacturing area (refer to the *Firmware Reference Manual for BelaSigna 300* for more information).

The IDE provides a read-only editor for viewing the contents of an EEPROM image file (`.nvi`), including a virtual view of the contents of EEPROM. To verify the content of your EEPROM layout, double-click your EEPROM image file in the IDE's Project Explorer view, and it opens with this editor by default. For more information on viewing an image file in the IDE, refer to Section 9.3.5, “Viewing an EEPROM Image with NVI Manager” on page 95.

9.3.1.3 Editing Build Settings

By default, the IDE creates an EEPROM project containing build settings for the NVMM builder including initial values for all required components (except an application to load). The build settings for your EEPROM project are stored in a file named `build.prefs` in the `.settings` folder of your project. To edit the build settings for your project:

1. Right-click your project in the Project Explorer view and select **Edit Build Settings...**.
2. The editor opens to show the current build configuration as displayed in Figure 9.

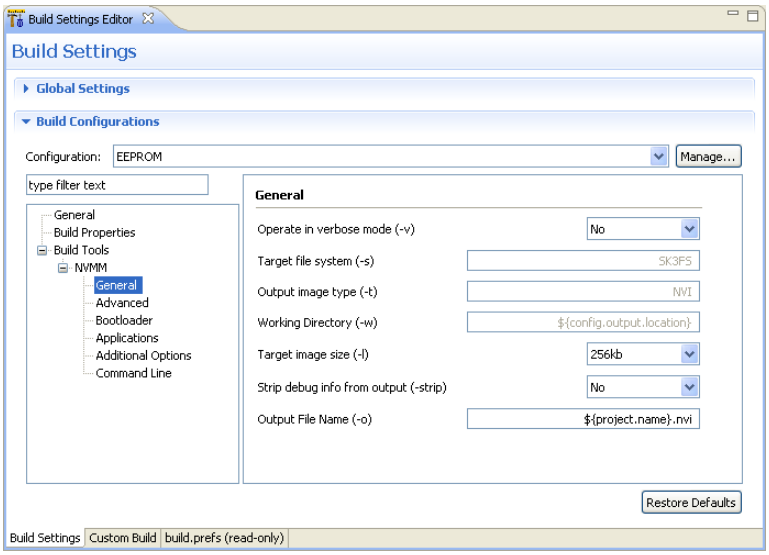


Figure 9: The General Build Settings for an EEPROM Project Build Configuration

3. On the left side, under **Build Tools > NVMM**, you see a list of available build settings grouped into categories that are described in more detail in Table 29. The build settings correspond to the available options for NVMM as described in Section 9.4, “Options” on page 98.

TABLE 29: NVMM BUILD SETTING CATEGORIES

Field Name	Description
General	This category controls many of the settings for NVMM that affect the output (e.g., *Target image size*, *Output file name*). Some of these settings are read-only, as they are determined by the project type (e.g., *Target file system*, *Image output*). For more information about the settings available in this category, refer to Section 9.4.1, “General Options” on page 99.
Advanced	This category contains many of the file-system specific options for NVMM. For BelaSigna 300, these include reserved blocks, field definitions, and loadable areas. Values for these settings can be defined by typing them directly into the provided text field or by using the convenient editors provided via the **Edit...** button located beside each field. For more information on the settings available in the advanced category, refer to Section 9.4.2, “Global Field Options” on page 103, as well as Section 9.4.3, “BelaSigna 300 (SK3FS) Areas” on page 104.
Bootloader	This category deals exclusively with settings for the bootloader. The default settings contained in the controls will place the standard bootloader at the first available address (`0x100`). For more information on the available options and their usage, refer to Section 9.4.1.1, “Bootloader-Specific Options” on page 100.
Applications	This category allows you to specify the applications to be included in your output image. Applications and their options (as well as section-specific options) can be entered directly in the provided text field. Alternately, you can use the **Application Configuration Wizard** accessible via the **Edit Paths** dialog that opens in response to clicking the **Edit...** button. For more information on the options available in this category, refer to Section 9.4.1.2, “Application-Specific Options” on page 100.
Additional Options	This category contains a single text field for entering settings for options that are not available directly through any other build settings. Options and settings entered in this field are passed directly to NVMM.
Command Line	This category displays a single read-only field containing a summary of the current command line for the NVMM. The contents of this field are the collective values of all configured build settings.

When you finish editing the build settings and successfully rebuild the project, NVMM generates an EEPROM image file (`.nvi`) in the default build configuration output folder. The build console displays the status of the output. Refer to Section 10.2, “Non-Volatile Memory Download Tool” on page 107 for more information about transferring this image to your device’s non-volatile memory.

9.3.1.4 Adding an Application

You must edit the build settings for your new project and add a valid application to the EEPROM layout before NVMM can successfully create a bootable image. To add an application, start by following the steps outlined in Section 9.3.1.3, “Editing Build Settings”, then proceed as follows:

1. In the **Build Settings Editor**, select **Build Tools > NVMM > Applications** as shown in Figure 10.

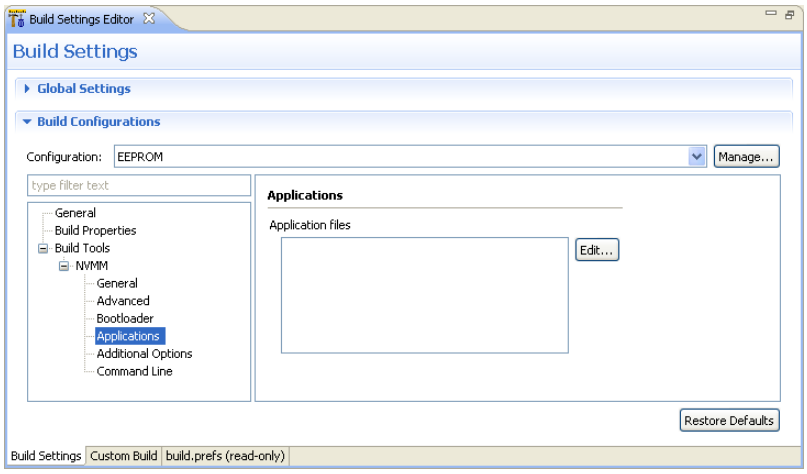


Figure 10: Adding Applications for an EEPROM Project in the Build Settings Editor

2. You can add an application *manually*, or using the *wizard*. The order of the applications determines the order in which they are added to the EEPROM image. You can use the `apptoload` field as described in Table 36 on page 104 to specify the index of the default application that will be loaded and executed by the bootloader. The default value is 1, corresponding to the first application in the list.
3. For the manual method, do the following in the **Application files** text field shown in Figure 10:
   a. Type the application name.
   b. Configure application-specific attributes using the syntax outlined in Section 9.4.1.2, “Application-Specific Options” on page 100.
   c. If the application contains valid data sections, you can also enter section-specific options in the text field using the syntax described in Section 9.4.1.2, “Application-Specific Options” on page 100.
   d. If you are adding more than one application definition, separate the statements with semicolons (;).

4. To use the wizard:
   a. Click the **Edit...** button and use the provided Edit Paths dialog (see Figure 11) to add, edit or delete new and existing applications. The order in which the applications appear in the Edit Paths dialog determines the order in which they are added to the EEPROM image. If you have more than one application, you can use the **Move Up** and **Move Down** icons to change the order.

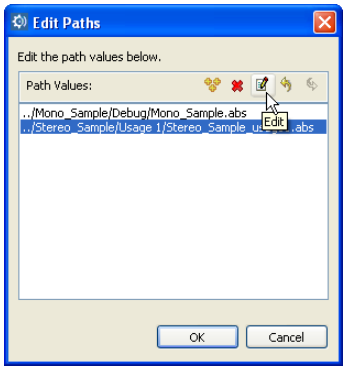


Figure 11: Edit Paths Dialog for Adding, Editing and Ordering Applications

b. Select the **Add** or **Edit** icons from the dialog's toolbar to launch the **NVMM Application Configuration Wizard** shown in Figure 12.

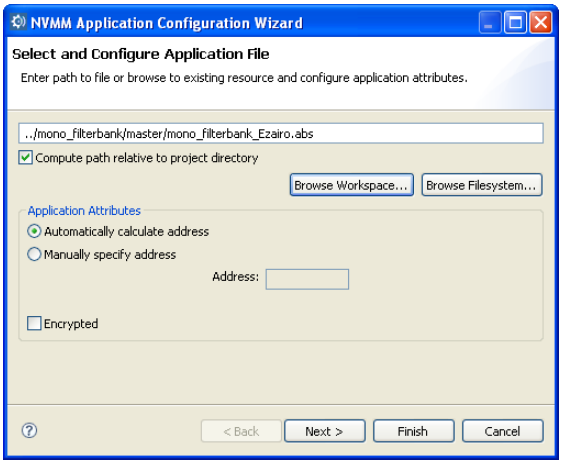


Figure 12: NVMM Application Configuration Wizard

c. Select the application file by browsing either the file system or the workspace with the appropriate buttons. Both absolute and relative paths are accepted; relative paths are calculated in relation to the project directory.
d. Configure application-specific attributes using the **Address** and **Encrypted** options.
e. If the application contains valid data sections, you can also specify section-specific options by clicking the **Next** button. Using the configuration wizard, shown in Figure 13, click the checkbox beside a segment name in the list on the left side to enable the controls for its available options. For more information on advanced, section-specific options for applications, refer to Section 9.4.1.2, "Application-Specific Options" on page 100.

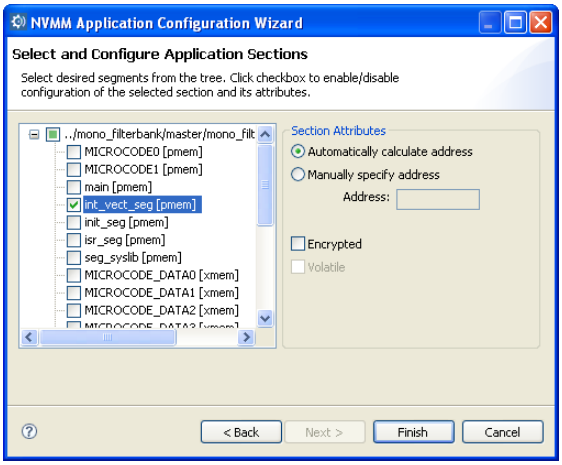


Figure 13: Configuring Application Sections

f. Click **Finish** when you are done using the wizard.
g. Click **OK** on the Edit Paths dialog. The application name and any attributes or options you selected appear in the **Application files** text field.

Note: If the application is part of the output from an existing project in the workspace, add a reference to the EEPROM project to ensure that build dependencies are satisfied. Adding a project reference enforces a strict build ordering, ensuring that the EEPROM image is rebuilt automatically whenever the application is rebuilt. For more information on project references, see the section regarding inter-project dependencies in the *Integrated Development Environment User's Guide*.

When you finish adding an application and successfully rebuild the project, NVMM generates an EEPROM image file (`.nvi`) in the default build configuration output folder. The build console displays the status of the output. Refer to Section 10.2, "Non-Volatile Memory Download Tool" on page 107 for more information on transferring this image to your device's non-volatile memory.

9.3.1.5 Securing an EEPROM Image

The default build settings for NVMM produce unsecured EEPROM images. The implementation of non-volatile image security is file-system specific. To create a secure EEPROM image for BelaSigna 300, start by configuring your applications with Level 2 security:

1. Add your application as described in Section 9.3.1.4, "Adding an Application" on page 87. When configuring your application, enable encryption using one of the following methods:
   - If you are using the **NVMM Application Configuration Wizard**, select the **Encrypted** checkbox for the appropriate application and/or any applicable sections.
   - If you are manually editing options, specify the application and/or section option `#encrypted=true`.

   For more information on the `encrypted` option, refer to Table 34 on page 103.

   Note: By default, program memory sections inherit their encryption state from their parent application. If the application is encrypted, and no section-specific

encryption is set, the section will be encrypted. Likewise, if the application is not encrypted, neither will the section be encrypted. To override the encryption settings for the application (i.e., to create an encrypted application containing one or more sections that are not encrypted or vice-versa), you must use an explicit, section-specific `encrypted` option.

2. Create a user key that NVMM can use to encrypt your application data. The user key is stored in the `user_key` field, in the key area of the EEPROM image. To specify a value for the `user_key` field in the IDE, edit the build settings as described in Section 9.3.1.3, "Editing Build Settings" on page 86, and select the **Advanced** category. Specify the value by using the **Edit...** button or by entering it in the text field.
   Note: If no value for `user_key` is defined, but an application or program section is declared to be encrypted with Level 2 encryption, the key area will be populated with randomly generated keys to ensure a bootable EEPROM image. The random keys are not stored, so the image will not be reproducible. If you want to create a reproducible, encrypted image, you must explicitly specify a value for the `user_key` field definition. This value will be stored in your EEPROM project build settings.

Your application data is now configured with Level 2 security. Level 1 Security builds on Level 2 security by encrypting both the key area used in Level 2 encryption and the bootloader in the non-volatile image output. To enable Level 1 security for the BelaSigna 300 EEPROM:

1. Edit your build settings as described in Section 9.3.1.3, "Editing Build Settings" on page 86, selecting the **Advanced** category.
2. Specify a non-zero value for the `shared_key_index` field by entering the appropriate value, either by using the **Edit...** button or typing directly in the text field. For more information on the `shared_key_index` field and its usage, refer to Table 36 on page 104.
   Note: Level 1 security requires that the shared key corresponding to the selected index be installed on the developer's host PC to build the encrypted image. Keys are installed and managed through the preferences found under **Window > Preferences > SignaKlara Development > EEPROM > Security.** To install the key, click the **Install...** button to open a file browser, and select the keyfile provided by ON Semiconductor.

IMPORTANT: Level 1 security uses a secret, shared encryption key to encrypt data in non-volatile memory. ON Semiconductor assigns unique keys to individual customers. Contact your ON Semiconductor representative to request your key.

3. Save the build settings and build the project. (If auto-build is enabled, saving the changes to the build settings automatically rebuilds the image.) When the build completes successfully, your image is secure.
4. You can verify the image by opening the `.nvi` output with the NVI Manager. Refer to Section 9.3.5, "Viewing an EEPROM Image with NVI Manager" on page 95 for more information.

Encrypted images can be downloaded to a device in exactly the same manner as unencrypted images. Refer to Section 10.2, "Non-Volatile Memory Download Tool" on page 107 for more information on transferring this encrypted image to your device's non-volatile memory.

9.3.2 Creating an EEPROM Image by Uploading

Assuming you have a BelaSigna 300 containing a valid EEPROM image, you can upload the contents of the EEPROM and save the data as an NVI file. The IDE provides a menu command to connect to your BelaSigna 300 device and upload the EEPROM contents. To activate the command:

1. Select **Tools > Upload EEPROM to Image file...** which opens the upload dialog (similar to Figure 14).

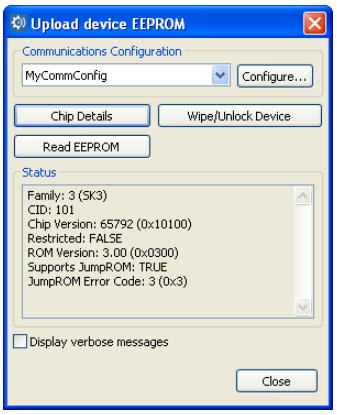


Figure 14: Upload device EEPROM Dialog

2. Select the Communication Toolkit (CTK) configuration to use via the **Communications Configuration** drop-down list. This specifies the communication configuration to use when connecting to your device.
   Note: If you have not already created a configuration, or if you want to modify an existing configuration, click the **Configure...** button to open the CTK Configuration Manager. Refer to the *Integrated Development Environment User's Guide* for details on creating or modifying a communication configuration.

3. If you have a valid communication configuration, you can click the **Chip Details** button to display basic information about the device to which you are connected. The **Status** window will show details such as Chip ID (CID), device family, ROM version, and whether the device is currently in restricted mode.
   Note: If the device is in *restricted mode*, your attempt to read the device memory will fail and an error will be displayed in the status window. You must *unlock* a device operating in restricted mode before attempting to read from it. To unlock the device, click **Wipe/Unlock Device**. *Note that this erases any key data on the device*.

4. To enable verbose messaging in the dialog, click on the checkbox labelled **Display verbose messages**. This displays any communication details for the current session in the scrollable **Status** window, including each byte address and its raw contents (as a hexadecimal value) as read directly from the device.
5. Click the **Read EEPROM** button to connect to the device and upload the contents of the non-volatile memory.
6. Once the data has been successfully uploaded from the device, you are prompted with a Save EEPROM Image File dialog allowing you to select the location in which to save the new file. Navigate to a location on your file system and enter a file name. The default name is

`eeprom_image`. If the file name that you enter exists in the selected location, you are prompted to allow overwriting.

Note: The default file format used to save the image file is NVI; however, you can also choose to save the EEPROM data as a text file (`.txt`) where it is represented as a simple listing of addresses and their corresponding byte values in hexadecimal format.

IMPORTANT: If the device EEPROM contains encrypted data, the encrypted data is uploaded as it exists in non-volatile memory. Currently, decrypting encrypted data contained in images obtained from a device is not supported.

By default, the IDE preferences are set to automatically open a downloaded NVI file in the NVI Manager view. To disable this option, refer to Section 9.3.5.4, “NVI Manager Preferences” on page 98.

9.3.3 Reserving a Section of EEPROM

You can reserve an area of the EEPROM for special data, such as gain calculations, manufacturing data, or a serial number, starting at address 0x100. Depending on whether you want to load data in the reserved block, or leave it blank, choose one of the following procedures.

Leave the reserved area blank to fill with data later

This method creates a single EEPROM image file that you can load onto all EEPROMs, and then you can write the data to the EEPROM in that reserved section separately (either before or after the EEPROM image is written).

1. Open the `build.prefs` file for the NVI project as described in Section 9.3.1.3, “Editing Build Settings” on page 86.
2. Select the **Bootloader** category (see the left panel), and choose **Automatically calculate address**.
3. Select the **Advanced** category, and edit the **Reserve blocks** field to add a new reserved block. For example, to reserve space for a data table containing 20 bytes of data, the block might start at 0x100 and end at 0x120.
   Note: The reserved range does not include the end address; in the example, 0x120 is the next free address following the reserved block.

Once you have saved and built your NVI project, you can open the EEPROM image file as described in Section 9.3.5, “Viewing an EEPROM Image with NVI Manager” on page 95. Notice that the section 0x100 to 0x120 contains no data and the bootloader is placed automatically at the next available address.

Put data in the reserved area at the time that you reserve it

This method requires you to recompile your NVI project whenever you change the data in the reserved section. If you update the data file and rebuild, the new calibration data is included in the image. For details about creating a reserved section of EEPROM with a data file, refer to Section 9.4.4, “Loading Arbitrary Data Files” on page 104.

9.3.4 Creating an EEPROM Project (with EEPROM Image Output) from an Existing NVI File

If you have a valid NVI file in your workspace, you can use a wizard to create a new EEPROM Project based on a selected NVI file. Build settings are automatically generated from the existing layout and inputs are derived from existing data. To activate the wizard:

1. Right-click an NVI file in the Project Explorer view.
2. Select **Create New Project from NVI File** which opens the wizard dialog.
3. Specify the name for the new EEPROM project.
4. Click **Finish**. A new project with this name is created in the workspace.
   Note: If a project with the specified name exists in the workspace, you are asked whether you want to overwrite the existing project and resources.

The new project is created with a default EEPROM build configuration. The inputs for the project are derived from the data in the original NVI file. Table 30 contains a description of the transformations that take place to extract the data from the original EEPROM image and create the inputs for the new EEPROM project.

TABLE 30: DERIVED INPUTS FOR EEPROM PROJECT FROM EXISTING NVI FILE

Original Data	Derived Inputs	Description
System Area	-	The attributes and data from the original System Area are copied, unaltered, to the System Area of the new project by way of field definitions and attributes in the new build settings. Refer to Section 9.4.1, “General Options” on page 99 for more information on available attributes and options.
Key Area	-	If the original image contains a level 2 encryption key, the key value is copied to the `user_key` value in the field definitions of the new build settings.
Manufacturing Area	`ManufacturingArea.d`	If the original image contains a Manufacturing Area, the data in that area is exported to a relative (`.d`) data file and the file is configured in the new build settings as a loadable area. Refer to Section 9.4.3, “BelaSigna 300 (SK3FS) Areas” on page 104 for more information on configuring a Manufacturing Area as a loadable area.
Bootloader	`Bootloader.cfo` `Bootloader.map`	The data in the Bootloader is exported to an absolute (`.cfo`) data file. Section information is captured in a corresponding loadmap (`.map`) file. The Bootloader is configured in the new build settings with explicit addressing options to match those in the original image. Refer to Section 9.4.1.1, “Bootloader-Specific Options” on page 100 for more information on Bootloader specific options.

TABLE 30: DERIVED INPUTS FOR EEPROM PROJECT FROM EXISTING NVI FILE (CONTINUED)

Original Data	Derived Inputs	Description
Applications	`<Application>.cfo` `<Application>.map`	The data in each application is exported to an absolute (`.cfo`) data file. Section information is captured in a corresponding loadmap (`.map`) file. Applications are configured in the new build settings with explicit settings for all section specific addressing options and attributes to match those in the original image. For a complete list of applicable options and attributes, refer to Section 9.4.1.2, "Application-Specific Options" on page 100.
Raw Data	`raw_data.d`	Raw data sections are exported to relative (`.d`) data files, and the files are configured in the new build settings as arbitrary data files. Refer to Section 9.4.4, "Loading Arbitrary Data Files" on page 104 for more information on loading arbitrary data files.

IMPORTANT: If the originating NVI file contains data encrypted with Level 2 encryption, the encrypted data is decrypted with the keys contained in the original image before being exported as unencrypted absolute data. Any padding that was added during the original encryption process is retained in the exported absolute data file. The wizard does not support images that are encrypted with Level 1 encryption.

9.3.5 Viewing an EEPROM Image with NVI Manager

The NVI Manager provides a read-only editor view for files with a `.nvi` extension within the IDE. To open an NVI file, do one of the following:

- In the Project Explorer view, double-click on a `.nvi` file.
- In the Project Explorer view, right-click on a `.nvi` file and select **Open With > NVI Manager**.
- Select **File > Open File...** and navigate to the desired `.nvi` file.

The NVI Manager interprets the NVI file format to render both a tree-view of the EEPROM file system and a graphical representation of the physical EEPROM layout. In addition to the NVI Manager view, additional representations of the NVI file can be selected by using the tabs located on the bottom of the view. Some aspects of the NVI Manager's appearance and behavior can be configured through user preferences.

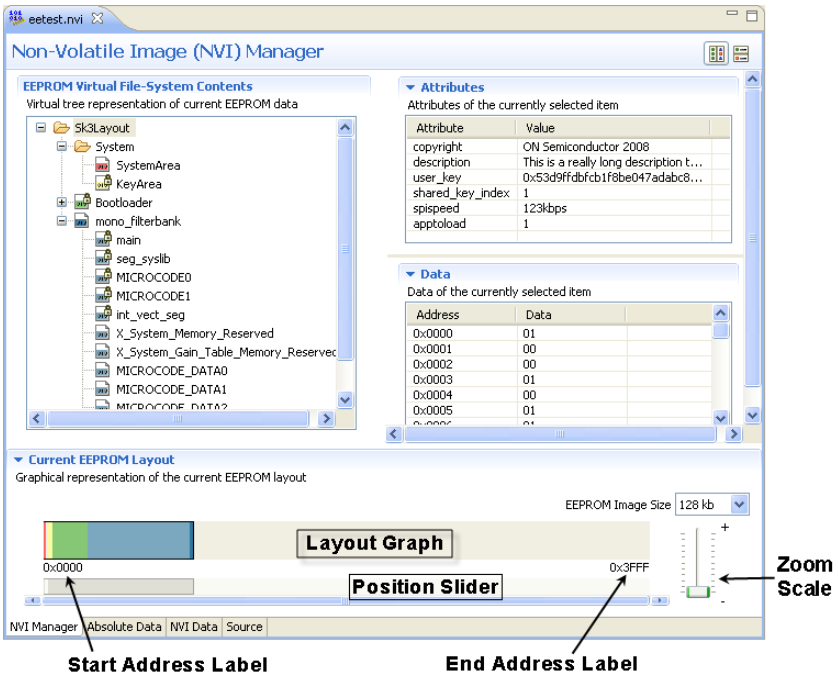


Figure 15: NVI Manager View

9.3.5.1 Virtual File System (VFS) Tree View

The tree view located in the top half of the NVI Manager provides the main interface for user interaction with the rendered layout. Tool tips are provided for all nodes in the tree, displaying the memory type and address where applicable. Selecting a node displays any attributes or data associated with it in the area to the right of the tree in their respective expandable sections. If the node is associated with data, the selected node is also highlighted in the graphical layout at the bottom of the NVI Manager. If a node contains encrypted data, it is displayed with a *locked* icon representation.

The data associated with nodes can be exported to the system clipboard for use outside of the IDE. To export data:

1. Right-click on the node containing the desired data.
2. Select **Copy Data to Clipboard** from the pop-up menu.
3. The data, now on the system clipboard, can be pasted into other applications where supported.

   Note: The **Copy Data to Clipboard** action is only supported by nodes associated with physical data.

9.3.5.2 Graphical EEPROM Layout View

The expandable section in the lower portion of the NVI Manager holds the layout view, which provides a graphical representation of the EEPROM as it would be after loading the current NVI file onto the device. Controls that can help you gain a clearer view of the data layout are labeled on Figure 15 on page 96 and described below:

Layout Graph

This horizontal color-coded bar provides a visualization of the physical EEPROM layout as it would appear on the device. The EEPROM contents are arranged against a blank image; the size of the image is determined by the **EEPROM Image Size** selector. The colored bars represent physical data in areas of memory defined by the file system specification and the NVMM builder options. You can select alternate colors through user preferences.

The start and end address labels define the memory range in view. These labels are automatically updated in response to the actions of the other controls to always reflect the range of memory in the current view.

The image responds to your mouse movements by providing tool tips containing the name, memory type and address of the node associated with the data at the specified point. The boundaries of the node are shown when the tooltip is displayed, and double-clicking on a colored area outlines the associated node. By default, selecting a node in the layout graph centers it (if necessary in a zoomed image), and selects it in the virtual file-system tree view. You can disable the option to snap to the tree selection in the user preferences.

Position Slider

This horizontal slider allows you to change the memory range in view when viewing a zoomed image. By default, this control is disabled because images are not zoomed when first opened. To enable the position slider, increase the zoom factor using the **Zoom Scale**.

The position slider shows the full EEPROM layout in the form of a grey bar that highlights the range of memory covered by the current image relative to the total size of the EEPROM. The slider itself delimits the memory range in view.

When viewing a zoomed image, you can move the slider left or right to show magnified regions of the EEPROM layout.

EEPROM Image Size Selector

This drop-down list allows you to specify the size (in Kbytes) of the EEPROM that you would like to render your image against. This can be useful for determining memory usage on the device. Selecting a different size scales the image against the new layout. The available selections are determined by the file system specified in the NVI file.

If you select a size that is too small for the EEPROM image, a warning symbol appears next to this field, and the graphical view is not refreshed until you select a valid size.

Zoom Scale

This vertical slider allows you to specify the zoom factor used to view the graphical rendering of the EEPROM layout. The range goes from one at the bottom (no magnification) to 100 at the top (highest magnification). You can set the slider incrementally at any point between the two bounds. Increasing the zoom factor reduces the range of memory displayed in the current view, and correspondingly, the relative size of the position slider.

9.3.5.3 Additional NVI File Views

Besides the default NVI Manager view, the multi-page NVI editor features additional representations of the NVI file that can be useful when examining the contents of the file or troubleshooting potential problems with the image. The additional views include:

Absolute Data Represents the NVI file in absolute data format composed of three columns: memory space, address and data. For example:

```
...
R 0101 E5
R 0102 3C
R 0103 00
...
```

NVI Data Represents native NVI data starting with an overview of the VFS node hierarchy (if it exists) followed by a listing of the basic block structure and the data chunks that comprise the actual EEPROM image.

Source Raw binary representation of the NVI file.

9.3.5.4 NVI Manager Preferences

A number of user configurable options allow you to customize the appearance and behavior of the NVI Manager. All preferences are accessed in the IDE through selecting **Window > Preferences...** in the main menu of the window, which opens the Preferences dialog. The preferences associated with the NVI Manager, described below, are located under the path **SignaKlara Development > EEPROM**.

Automatically open uploaded EEPROM images (NVI format)
This option automatically opens any uploaded file with a `.nvi` extension in the IDE with the NVI Manager. It is selected by default.

NVI Viewer Colors
Located under **NVI Manager**, this group of color-field editors enables you to customize the colors used for each component area of the image comprising the graphical representation of the EEPROM layout.

Snap EEPROM layout view to tree selection
Located under **NVI Manager**, this option synchronizes the graphical layout view with the current selection in the tree view. Selecting a new node in the tree moves the slider position to bring the selected area into range (centered, where possible), and highlights it. This can be useful when viewing large, zoomed images with many sections. This option is enabled by default.

9.4 OPTIONS

This section explains the available options for NVMM.

9.4.1 General Options

In its simplest form, NVMM constructs an image by providing it with a bootloader and one or more applications (up to 8 are allowed for BelaSigna 300). NVMM takes general options as well as both bootloader-specific and application-specific options. Table 31 describes the general options available for NVMM.

TABLE 31: GENERAL OPTIONS

Option	Name	Description
`-h`	Help	Passing this option causes NVMM to output the default help information (which is given also when NVMM receives an invalid option, or no parameters) along with file-system specific help. For more file-system specific information refer to the *Firmware Reference Manual for BelaSigna 300*.
`-v`	Verbose Mode	By default, NVMM operates in quiet mode and only displays warnings or errors. To see more information about the image build process, pass `-v` to NVMM. In this case, NVMM operates in verbose mode, and displays information about the progress of the build and how the image is being built.
`-w` **`workdir`**	Working Directory	This option is used to set NVMM's working directory. The working directory is used to calculate the path to a file when a relative path name is given. If this option is not specified, the working directory defaults to the current directory when NVMM is launched.
`-s` **`filesys`**	File System Type	In this release, the only file system supported is `SK3FS`, the file system used with BelaSigna 300, which is currently the default value for this option.
`-t` **`imagetype`**	Output Image Format Type	The only output image format type currently supported by NVMM is the NVI file format, which is the default value for this option.
`-l` **`size`**	Output Image Size	Use this option to specify the maximum image size that NVMM is allowed to generate. The size refers to the actual raw data that will be written to the non-volatile memory of the device, and not to the actual size of the image file. The size of the image can be specified in bits, bytes, Kbits, and Kbytes by using a number followed by `b`, `B`, `kb`, or `kB` respectively. NVMM aborts with an error message if the image size is exceeded.
`-strip`	Strip Debug Info	By default, NVMM uses the debug symbols obtained from any applications specified as input (must be valid `.abs` files) to provide user-readable names for applications and sections in the image output. Use this option to create an image which does *not* contain information obtained from debug symbols. In this case, application and section names are automatically generated using default naming conventions.

TABLE 31: GENERAL OPTIONS (CONTINUED)

Option	Name	Description
`-o` **`output`**	Output File Path	Use this option to specify the path and file name that NVMM should use for generating the output image file. If this option is unspecified, NVMM generates a file with a default name (`i.nvi`) in the working directory.
`-r` **`addr-addr`**	Reserve Address Range	Use this option to prevent NVMM from placing any data in the specified address range. The range specified includes the start address, up to but not including the end address.
`-f` **`field=x`**	Set Global File System Field	Sets the specified global field to the given value **`x`**. For more information on the available global fields and the range of valid values, refer to Section 9.4.2, "Global Field Options" on page 103.
`-d` **`area=f`**	Load File System Area	Loads the given file-system specific area with the contents of the given file **`f`**. The specified file must be a binary file of raw bytes, or a relative data file (ASCII) as described in Section 9.4.4, "Loading Arbitrary Data Files" on page 104. The contents of the file are copied byte for byte into the given area. For more information regarding the list of available areas, refer to Section 9.4.3, "BelaSigna 300 (SK3FS) Areas" on page 104.

9.4.1.1 Bootloader-Specific Options

You can specify bootloader-specific options appended to the bootloader filename. NVMM supports the bootloader-specific options described in Table 32.

TABLE 32: BOOTLOADER-SPECIFIC OPTIONS

Option	Name	Description
`@address`	Address	The address at which the bootloader is placed in non-volatile memory. If unspecified, NVMM calculates an address automatically. See Table 33 for more information on how this address is calculated.

The Build Settings Editor in the IDE allows you to edit the optional address field in a separate text field and takes care of formatting the command line for you. For BelaSigna 300, the default value for the bootloader option already points to the compiled BelaSigna 300 bootloader for the appropriate device version; the compiled bootloader executables are included with the EDK and are found in **`<EDK_HOME>`**`\source\sk3\bootloader\master\sk3_bootload_cid*.abs`.

9.4.1.2 Application-Specific Options

You can also specify application-specific options appended to each application filename. NVMM supports the application-specific options defined in Table 33.

TABLE 33: APPLICATION-SPECIFIC OPTIONS

Option	Name	Description
@address	Address	The application is placed at this address in non-volatile memory. If the address option is unspecified, NVMM automatically places the application and its sections in the layout according to the file system's specification. Specify the address as a numeric value in one of the C-style hex, octal, binary or decimal formats, or using the literal value **auto**. The implementation is somewhat file-system specific, but the general guidelines are: • If an address is defined as a number, the application and its sections are placed sequentially starting at the given address. • If the address is defined as **auto** (or unspecified), the application and its sections are automatically positioned at the first available location in memory (i.e., each section is positioned independently). For application sections (see the **!section** option below), you can also specify the **@address** option. It defines the address at which the specified application section will be placed. If this address is a number, the section is placed at the given address. If this address is defined as **auto**, NVMM automatically places the section regardless of the section's parent application address (which is useful for equivalent section matching). If the **@address** option is not specified for an application section, the section is placed automatically according to its parent application's address.

TABLE 33: APPLICATION-SPECIFIC OPTIONS (CONTINUED)

Option	Name	Description
#**field=x**	Set Application Field(s)	Use the hash character (#) to specify an application-specific field (for a list of application-specific fields refer to Table 34), and set it to value **x**. You can define multiple **field=x** pairs, each starting with the hash character.
!**section**	Define Section-Specific Options	Use the bang character (!) to define section-specific options within an application. Section-specific options take the same syntax as application-specific options, but you must prefix those options with !**section** where **section** is the symbolic name of the section within the application (and remember that it is case-sensitive). For example: `app.abs!main@0x2000#encrypted=true` The snippet above forces the `main` section in application `app.abs` to be located at address `0x2000` in non-volatile memory and forces it to be encrypted. You can define multiple section-specific options simply by repeating the !**section** suffix. Also note that `~apptable` is a special application section specific to the BelaSigna 300 EEPROM file system referring to the application's vector table. See the *Firmware Reference Manual for BelaSigna 300* for more information.

When specifying explicit addresses for program memory sections that are encrypted, ensure that you allow for the additional padding (up to 3 bytes) that might be added to the section during encryption. For more information on encryption, refer to the *Firmware Reference Manual for BelaSigna 300*. NVMM will fail to create the non-volatile image output if sections overlap. When using automatic addressing, the extra padding is automatically taken into account.

Note: By default, NVMM processes the application sections corresponding to the segments in the linker output. Using the !**section** specifier for a named segment within an application file (with or without additional section-specific options) creates a separate EEPROM section for that segment. This can be used to arrange segments independently. There is however, overhead in the form of additional header blocks and calculated CRC values created when sections are split. In the case where the named segment occurs at the beginning or the end of an existing section, splitting results in the creation of two sections: one for the named segment and one for the remaining data. In the case where the named segment occurs in the middle of an existing section, the result of the split is three sections: one section containing the named segment, one section for the data that precedes it, and one for that which follows.

NVMM does not currently define any generic, application-specific fields, but the following application-specific fields are available for BelaSigna 300 (`SK3FS`) images:

TABLE 34: BELASIGNA 300 (SK3FS) APPLICATION-SPECIFIC FIELDS

Field Name	Description
`encrypted=`**`boolean`**	Set the encrypted state of the specified P-section(s). Encryption implies level 2 security (see the *Firmware Reference Manual for BelaSigna 300* for details). User keys, used to encrypt the section data, are defined using global fields as described in Table 36 on page 104. If no user keys are defined, NVMM automatically generates random keys to ensure a bootable EEPROM image is created.
`volatile=`**`boolean`**	Set the volatile state of the specified X- or Y-section. A volatile section type indicates that a non-critical error is generated when the CRC does not match the data, and the boot process continues even though the data in the section might be corrupt.

The Build Settings Editor in the IDE provides a convenient way to add most common, application-specific options described in Table 33. To configure application-specific options:

1. Edit the build settings as described in Section 9.3.1.3, “Editing Build Settings” on page 86, and select the **Advanced** category.
2. To configure common options, click **Edit...** and use the field editors to specify appropriate values. You can also type the options directly into the text fields on the build settings page.

To set options that are not available in the **Advanced** section, type them directly into the text field provided in the **Additional Options** section of the build settings page.

9.4.2 Global Field Options

Table 35 describes the global fields available for setting in NVMM with the `-f` option.

TABLE 35: GENERIC GLOBAL FIELDS

Field Name	Description
`copyright=`**`string`**	Set the copyright information in the output image file.
`description=`**`string`**	Set the description in the output image file.

In addition to the global fields described in Table 35, you can also set the global fields specific to BelaSigna 300 as described in Table 36.

TABLE 36: GLOBAL FIELDS SPECIFIC TO BELASIGNA 300

Field Name	Description
`spispeed=`**`speed`**	Set the SPI port speed flag, where **`speed`** is one of `123kbps` (or `slow`) or `980kbps` (or `fast`).
`shared_key_index=`**`index`**	Set the shared encryption key index for Level 1 Security. The default is `0`, which means no Level 1 Security encryption. For BelaSigna 300, valid ranges for the Level 1 shared key index are 1-127 inclusive.
`apptoload=`**`number`**	Set the application number to load (default: `1`). For BelaSigna 300, the valid range is 1-8 inclusive.
`user_key=`**`key`**	Set the (128-bit) user key to **`key`** for encrypting P-sections (same as `user_key_entry1`). The value entered in this field is stored in the key area EEPROM section of your layout as required by the BelaSigna 300 EEPROM file system (`SK3FS`).

9.4.3 BelaSigna 300 (SK3FS) Areas

Table 37 describes the specific areas of BelaSigna 300 available for loading with the `-d` option.

TABLE 37: BELASIGNA 300 (SK3FS) AREAS

Area Name	Description
`mftrarea`	Load the entire manufacturing area (EEPROM byte addresses `E:0x0020` to `E:0x007D`). If the file contains less than the 94 bytes of data required to fill the manufacturing area, the area is padded to the end with `0x00`. The CRC value is automatically calculated on the entire area once the data is loaded from the file. If the file is larger than 94 bytes, only the first 94 bytes are used. The CRC is calculated on the data contained in those 94 bytes and the rest is truncated.

9.4.4 Loading Arbitrary Data Files

To load arbitrary data into a specific region of EEPROM (e.g., for a fitting table or other set of application-specific values outside of an application), specify your data file as another "application" file in your NVMM settings.

The specifications for loading data files are as follows:

- The data file must have the extension `.d`.
- The file must be formatted as a relative data file containing a single column of data with one byte per line, in hexadecimal format (with or without a leading `0x`). For example:

```
00
01
02
```

- The file will be loaded into EEPROM at an auto-allocated address unless you explicitly specify an address with the `@address` suffix. For a raw data table, always specify the address explicitly.
- The file will be loaded into your layout directly without modification. A CRC *will not* be calculated or appended to the data table.
  Note: If you require a CRC or a checksum for the purposes of data validation, you must calculate it yourself and include it as part of the data table.

- The data file is *not* considered an application and *will not* be referenced in the system area.

  Note: You cannot specify applications from raw data files.

This page left blank for printing purposes.

10 Non-Volatile Image Tools

10.1 INTRODUCTION

Tools that are available for managing non-volatile images are:

- The Non-Volatile Memory Download tool
- The Non-Volatile Image Dump tool

10.2 NON-VOLATILE MEMORY DOWNLOAD TOOL

The Non-Volatile Memory Download (NVMD) tool is a light-weight tool for transferring the data in non-volatile image (NVI) files to the non-volatile memory of your device. NVMD uses the CTK to connect to the device, stop the system's DSP, perform a fast transfer directly to the device's EEPROM, and then initiate a system reset before disconnecting.

10.2.1 Using NVMD Within the IDE

This section describes using NVMD from within the IDE's graphical user interface.

10.2.1.1 Downloading a .nvi File

The IDE provides a graphical interface for transferring NVI files to the non-volatile memory of your device. After you successfully build a BelaSigna 300 EEPROM project, you will have a `.nvi` file in the output folder of your project's default build configuration. This file contains an EEPROM image that can be downloaded to the EEPROM of your BelaSigna 300 device using NVMD. If you need more information about building an NVI file, refer to Chapter 9, "Non-Volatile Memory Management" on page 83.

To download a file to your device's EEPROM:

1. Locate the `.nvi` file in the Project Explorer view, which you can find in the SignaKlara Development perspective. Right-click on it to display the context menu shown in Figure 16.

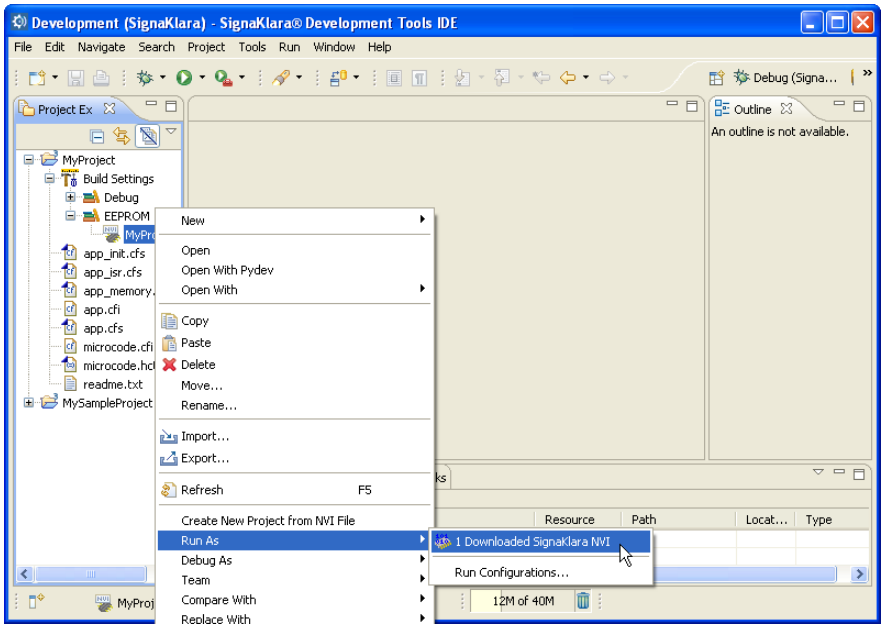


Figure 16: Menu for Downloading an EEPROM Image Using NVMD

2. Select **Run As** > **Downloaded SignaKlara NVI**.
3. If you have not downloaded the image before, you are prompted to select a communications configuration via the Communication Toolkit (CTK) Configuration Manager. If you have not created a communication configuration before, you need to add one now (refer to the *Integrated Development Environment User's Guide* for details on creating a communication configuration).
4. If the device is in restricted mode, you are prompted to wipe and unlock the device to gain access. Note that unlocking the device erases the key data. If you click **Yes**, the device is wiped and unlocked.
5. At this point, the IDE automatically launches NVMD as an external process; any output from the tool is sent to the Console view. When the operation finishes successfully, the image file has been downloaded to the device's EEPROM.

In the process of launching NVMD, the IDE creates and stores a launch configuration using the name of the NVI file by default. A launch configuration contains all of the settings necessary to relaunch NVMD exactly as it was the first time. This launch configuration contains various settings that control the execution of NVMD. More information on these settings is available in Section 10.2.1.2, "NVI Download Launch Configurations" on page 108.

10.2.1.2 NVI Download Launch Configurations

Launch configurations determine the context of an NVMD download operation. They specify which image file to download, and which communication configuration to use. They also store additional settings that control the execution of NVMD. Everything pertinent to an NVMD download operation is stored in its launch configuration.

You can view and edit launch configurations through the launch configuration dialog (shown in Figure 17), which can be displayed by selecting the **Run Configurations...** menu item that appears in several menus:

- Under **Run** in the main menu
- In the context menu when a `.nvi` file is selected in the Navigation view (right-click, choose **Run As**)
- In the toolbar, under the down arrow for the Run icon

In the Run dialog, under **Local SignaKlara NVI** in the left pane, select the launch configuration that was created when you performed the procedure in Section 10.2.1.1, "Downloading a .nvi File" on page 107. The launch configuration has the same name as the NVI file by default. The **Image to download** field is already filled in, and you can change the options as necessary.

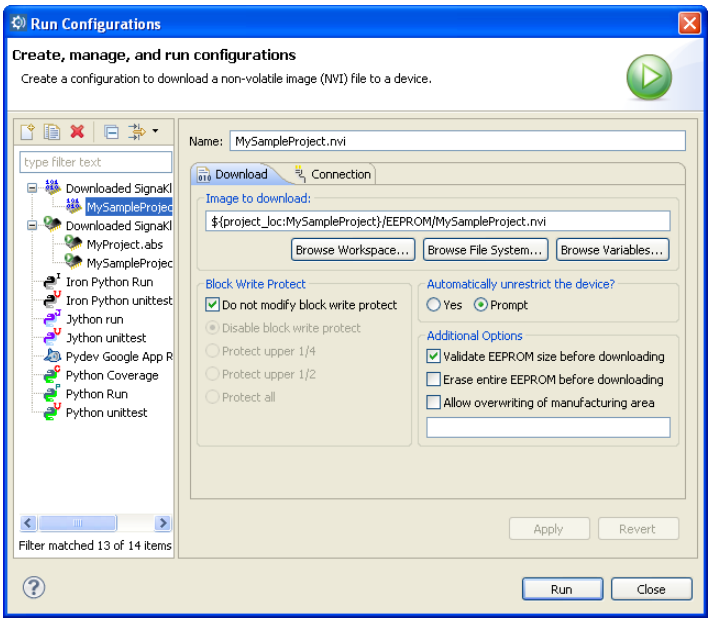


Figure 17: NVMD Launch Configuration Download Tab

The pane on the left side displays all the existing launches grouped by type. Configurations can be added and removed by clicking on the **New** and **Delete** buttons above it.

The right pane consists of two tabbed pages:

Download This tab identifies:

- The image file to download
- Whether to set the block write protect value to a specific value
- Whether to automatically unrestrict the device
- Whether to validate the EEPROM size before downloading
- Whether to wipe the device before downloading
- Whether to allow overwriting the manufacturing area of the device
- Additional command line options as specified in the space provided

Connection The communication configuration to use for the download operation.

10.2.1.2.1 Download Tab

The **Download** tab, shown in Figure 17 on page 109, is used to specify parameters related to downloading image files. The **Image to download** text field specifies which `.nvi` file will be downloaded to the device's EEPROM.

The **Block Write Protect** radio group allows you to specify whether to leave the block write protect value alone, or to set it to a specific value after the download operation completes.

IMPORTANT: If "Do not modify block write protect" is set, NVMD will *not* disable block write protect for download. It might execute without errors, even if this results in an invalid image or no change to the EEPROM.

The **Automatically unrestrict the device** radio group controls what happens if a device is in restricted mode when the download operation begins: the software automatically unrestricts the device, or you are prompted for confirmation. Choosing not to unrestrict a restricted device before downloading causes the download operation to fail.

The **Additional Options** control group provides checkboxes and an editable text field for the following purposes:

- To disable the automatic size validation that takes place prior to downloading an image file to the device

CAUTION: The EEPROM size validation is enabled by default; only disable it if your device does not support the validation operation. The validation step ensures that the attached EEPROM is large enough to hold the image to be downloaded. Downloading an image that exceeds the available EEPROM size results in overflow and might render the device non-bootable.

- To specify whether the entire EEPROM is erased before downloading
- To allow the overwriting the manufacturing area during wipe and download operations

CAUTION: By default, the manufacturing area is not affected by either the wipe operation or the image download operation. Selecting the option to **Allow overwriting of manufacturing area** removes the write protection on the manufacturing area and allows the device-specific information stored there to be overwritten. Use of this option is not recommended in most cases as it results in the loss of important information, such as calibration data written at the time of the production of the device.

- To specify additional command line options that are passed to NVMD for use in the download operation; for a full list of the available options, refer to Table 38 on page 112

10.2.1.2.2 Connection Tab

The IDE is capable of communicating with a variety of development boards through a variety of communication devices. You can specify which protocol and communication device is used for a download operation in the Connection tab, shown in Figure 18.

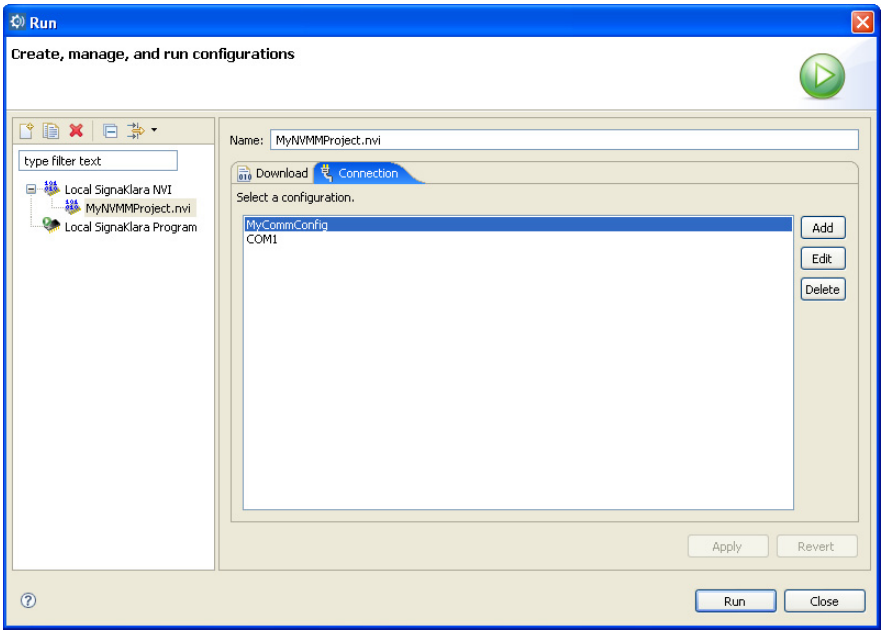


Figure 18: NVMD Launch Configuration Connection Tab

The Connection tab presents a list from which you can select a communication configuration for use by the debug session. You can also add, edit, and remove communication configurations from the list with buttons on the right side. More details on managing communication configurations are given in the *Integrated Development Environment User's Guide*.

IMPORTANT: Communication configurations are global settings. Changing or removing communication configurations affects all of the launch configurations that rely on them.

10.2.2 Usage

NVMD is a command line tool that can also be executed directly from a command prompt. The help (`-h`) option to NVMD provides details on all options and parameters, and they are described in Table 38. A simple command line for using the tool consists of the program name and an image file:

```
nvmd [options] image
```

The image file is a valid, non-volatile image (NVI) file that is the output of NVMM. For details on using NVMM, refer to Chapter 9, "Non-Volatile Memory Management" on page 83.

TABLE 38: NVMD TOOL OPTIONS

Option	Name	Description
`-h`	Help	Passing this option causes NVMD to output a summary of the available options.
`-version`	Version	Passing this option causes NVMD to output the current version number of the executable file.
`-v`	Verbose Mode	By default, NVMD operates in quiet mode and only displays warnings or errors. To see more information about the image download process, `-v` can be passed to NVMD on the command line. In this case, NVMD will operate in verbose mode and will display information about the progress of the download.
`-u`	Unrestrict	If the device that NVMD is attempting to download the image to is in restricted mode, NVMD will abort the download attempt and fail by default. The `-u` option tells NVMD to automatically unrestrict the device and continue the download process instead.
`-f` **`range`**	Fill	Fill the device's non-volatile memory at the specified range with `0xFF` (by default). An alternate byte-value can be specified using the `-x` option. Ranges are specified using the format: **`start`**`:[`**`length`**`]` where **`start`** is a valid address in memory and the optional **`length`** argument is the length in bytes, separated by a colon. Values can be specified in decimal, octal or hexadecimal format. *Note:* if **`length`** is omitted, the area from **`start`** to the end of available memory will be filled. An explicit value for **`length`** *must* be specified if used in conjunction with the `-NoSizeCheck` option.
`-x` **`value`**	Fill Value	Specify the value of the hex byte to be used in conjunction with the `-f` option. This option is ignored if `-f` is not specified.
`-w`	Wipe	Wipe the entire contents of the device's non-volatile memory before downloading by filling all of non-volatile memory with `0xFF`. This option is different from the partial system wipe that occurs as a side-effect of the Unrestrict option. *Note:* This option is *deprecated*. Use `-f 0x0000:` instead.

TABLE 38: NVMD TOOL OPTIONS (CONTINUED)

Option	Name	Description
`-b` **`setting`**	Clear and Set Block Write Protect	This option serves two purposes: first, it disables block write protect on the device if it is enabled (by default NVMD will leave the BWP setting as-is); secondly, it sets a particular block write protect setting upon completion. Valid block write protect values are: `0` or `none` - Disable block write protect `1` or `upper 1/4` - Block write protect upper 1/4 `2` or `upper 1/2` - Block write protect upper 1/2 `3` or `all` - Block write protect all *Note:* If `-b` is not defined, NVMD will *not* disable BWP for download and might execute without errors, even if this results in an invalid image, or no change to the EEPROM.
`-c` **`config`**	CTK Configuration	Tells NVMD to use the given CTK configuration to connect to the device. If this option is not supplied, NVMD will open a CTK configuration select dialog when it is called.
`-ForceOverwriteManufacturingArea`		Tells NVMD to write the block identified as the manufacturing area (node type “MftrArea”) to the device during the download. By default, a manufacturing area is never overwritten. *Note:* *Use extreme caution* when enabling this option as it results in the loss of all existing data in the manufacturing area, including important calibration values written during the production of the device.
`-NoSizeCheck`		Forces NVMD to skip validation of the device's non-volatile memory size prior to downloading the image. By default, the size of the attached non-volatile memory is always verified to ensure that the downloaded image will not exceed it. *Note:* Downloading an image which exceeds the physical size of the non-volatile memory on the device will result in overflow and may render the device non-bootable.

10.3 NON-VOLATILE IMAGE DUMP TOOL

The Non-Volatile Image Dump (NVIDUMP) Tool is a command line tool that can be used to dump the raw data contained in a binary, non-volatile image (NVI) file to various other formats. The help

(`-h`) option to NVIDUMP provides details on all options and parameters, and they are described in Table 39. A simple command line for using the tool consists of the program name and an image file:

```
nvidump [options] image
```

The **`image`** parameter is a valid, non-volatile image (NVI) file. Non-volatile image (NVI) files are created with the NVMM. For details on using NVMM, refer to Chapter 9, "Non-Volatile Memory Management" on page 83. The default output of NVIDUMP is an absolute-formatted dump of the data contained in the non-volatile image file. Additional formats are supported through command line options in Table 39.

TABLE 39: NVIDUMP TOOL OPTIONS

Option	Name	Description
`-h`	Help	NVIDUMP outputs the tool version information and a summary of the available options.
`-v`	Verbose Mode	Use this option in conjunction with the `--vfs` option to include formatted data chunks in the output, along with the VFS information.
`-o` **`file`**	Output File	Capture the output in the given file. If this option is not specified, NVIDUMP displays the output in the console.
`-f` **`start:end`**	Fill Gaps	Fill any gaps in the data contained in the Intel HEX or Motorola SREC output file within the specified range with 0xFF (by default). An alternate byte-value can be specified using the `-x` option. Ranges are specified using the format **`start:end`** where **`start`** is a valid address less than or equal to the lowest address in the data file, and **`end`** is a valid address greater than or equal to the highest address, separated by a colon. Values can be specified in decimal, octal or hexadecimal format.
`-x` **`value`**	Fill Value	Specify the value of the hex byte to be used in conjunction with the `-f` option. This option is ignored if `-f` is not specified.
Options for Output Formats		
`--vfs`	VFS Format	Output the Virtual File System (VFS) data contained in the image file including general file system information and supported image features. By default, absolute data is displayed in the output.
`--intelhex`	Intel HEX Format	Output the data contained in the NVI file in Intel HEX format instead of the default absolute data format.
`--motorolasrec`	Motorola SREC (S19) Format	Output the data contained in the NVI file in Motorola SREC (S19) format instead of the default absolute data format.

A File Formats

A.1 INTRODUCTION

CFCC uses two binary file formats, for the outputs of the assembly and linking stages, respectively. The output of the linking stage is known as an *Absolute* file; its format is described in Section A.2, “Absolute Executable Format” on page 115. The object files produced by the assembler are described in Section A.3, “Relocatable Object File Format” on page 122. The file format for storing non-volatile memory images once the layout has been processed is described in Section A.4, “Non-Volatile Memory Image Format (.nvi)” on page 132.

A.2 ABSOLUTE EXECUTABLE FORMAT

Absolute executable files are designed to be flexible and extensible. An Absolute file consists of a file header followed by various data records.

A.2.1 Absolute File Header

The file header consists of a required part followed by optional parts. The required part has the following structure:

```
#define ABS_MAGIC_SZ 8
struct abs_file_header {
    char magic[ABS_MAGIC_SZ];          /* = "ArABS1\n\0" */
    unsigned long host_endianness;     /* = 0x87654321 */
    unsigned long content_flags;
};
```

You can:

- Use the `magic` constant to identify this file format.
- Use the `host_endianness` field to check that the byte ordering of the machine that reads the file matches the byte ordering of the machine that generated it.
- Use the `content_flags` to determine the form of the optional part of the file header. If a certain flag is set, then its optional header part is present. The optional header parts appear in order according to the flag bit, from least significant to most significant; that is, the optional part corresponding to the flag with the lowest value appears first.

The available content flags are:

```
#define ACF_TARGET        0x00000001 /* target id */
#define ACF_ABSREC        0x00000002 /* object records */
#define ACF_ENTRY         0x00000004 /* entry point */
#define ACF_BSSREC        0x00000008 /* uninitialized data record */
#define ACF_ENTRYS        0x00000010 /* entry point */
#define ACF_STRTABLE      0x00000020 /* string table */
#define ACF_SYMTABLE      0x00000040 /* symbol table */
```

```
#define ACF_FILETABLE     0x00000080 /* file name table */
#define ACF_LINETABLE     0x00000100 /* line number table */
#define ACF_COMMENT       0x00000200 /* comment table */
#define ACF_LOADMAP       0x00000400 /* load map */
```

Table 40 lists and describes the optional header records an Absolute output file header can contain. The following sections describe each optional header record and corresponding data records.

TABLE 40: OPTIONAL HEADER RECORDS

Optional Header Part	Description of Header and Corresponding Data
TARGET	Identifies the target architecture.
ABSREC	Contains code and data.
ENTRY	Provides a table of entry points.
BSSREC	Describes uninitialized data, defined by assembler .bss directives.
ENTRYS	Provides entry points as well, but differs from ENTRY in that it refers to debug symbols, rather than normal (label) symbols.
STRTABLE	The Absolute file's string table, which holds text strings used in other Absolute format tables, such as debug symbol names, file names, and embedded comments.
SYMTABLE	Contains the debug symbol table.
FILETABLE	A table of file names, used for debugging.
LINETABLE	A line number table, for debugging.
COMMENT	Contains embedded comments, as defined by assembler .comment directives.
LOADMAP	Contains a load map equivalent to that generated by the --map command line option.

A.2.2 TARGET Header Record

The optional TARGET header record identifies the target architecture. It has the following form:

```
struct abs_file_target {
    char magic[ABS_MAGIC_SZ];
};
```

The value stored in the magic field always has the value CFX\0\0\0\0\0.

A.2.3 ABSREC Header and Data Records

The optional ABSREC header record and corresponding data records contain code and data. The header record has the following form:

```
struct abs_file_absrec {
    unsigned long absrec_addr;
    unsigned long absrec_count;
};
```

The `absrec_addr` field is the location within the output file of the object records, and `absrec_count` is the number of object records. Each object record begins with an object header, followed by object data. The header has the following form:

```
struct abs_absrec {
    unsigned short numentries;
    unsigned short wordsize;
    unsigned short segid;
    unsigned char locinc;
    unsigned char memtype;
    unsigned long loadaddr;
};
```

The following data contains `numentries` records of `wordsize` bits. Each record is padded to a byte. The whole block of data, in turn, is padded to a 4-byte boundary. The `locinc` is the location counter increment of the data segment, and the `memtype` is its memory type. The `loadaddr` is the memory address at which the data block is to be loaded.

A.2.4 ENTRY Header and Data Records

The optional `ENTRY` header record and corresponding data records provide a table of entry points. The header record has the following form:

```
struct abs_file_entry {
    unsigned long entry_addr;
    unsigned long entry_count;
};
```

The `entry_addr` is the location within the output file of the entry point table, and `entry_count` is the number of entry point records. Each record has the following form:

```
#define ENTRY_NAMESIZE 36
struct abs_entry {
    unsigned short reserved1;
    unsigned char namesize;
    unsigned char memtype;
    unsigned long loadaddr;
    char name[ENTRY_NAMESIZE];
};
```

The `namesize` field gives the size of the (fixed-length) `name` field. The `memtype` and `loadaddr` field are the memory type and address of the entry point, and `name` is the name of the symbol given in the `--entry` command line option.

A.2.5 BSSREC Header and Data Records

The optional `BSSREC` header record and corresponding data records describe uninitialized data, defined by assembler `.bss` directives. Some program loaders process `BSSREC` records and clear

the corresponding memory regions, while others ignore BSSREC records and do not modify their memory regions. The header record has the following form:

```
struct abs_file_bssrec {
    unsigned long bssrec_addr;
    unsigned long bssrec_count;
};
```

The bssrec_addr is the location within the output file of the uninitialized data records, and bssrec_count is the number of uninitialized data records. Each record has the following form:

```
struct abs_bssrec {
    unsigned short numentries;
    unsigned short wordsize;
    unsigned short segid;
    unsigned char locinc;
    unsigned char memtype;
    unsigned long loadaddr;
};
```

The form is identical to that of an ABSREC record header.

A.2.6 ENTRYS Header and Data Records

The optional ENTRYS header record and corresponding data records are similar to the ENTRY table, in that they are intended to identify entry points, but differ from ENTRY in that they refer to debug symbols, rather than normal (that is, label) symbols. The members of the ENTRYS table are the debug symbols named by the entrys= CFLINK command line option. The header record has the following form:

```
struct abs_file_entrys {
    unsigned long entrys_addr;
    unsigned long entrys_count;
};
```

The entrys_addr is the location within the output file of the debug entry records, and entrys_count is the number of debug entry records. Each record has the following form, by default:

```
struct abs_entrys {
    unsigned long symbol;
    unsigned long loadaddr;
};
```

The symbol field is the index into the debug symbol table of the debug symbol given in an entrys= CFLINK command line option. The loadaddr field is actually the value of debug info slot 1, which can (if this convention is followed in defining debug symbols) contain a load address associated with the debug symbol.

A.2.7 STRTABLE Header and Data Records

The optional `STRTABLE` header record and corresponding data records hold text strings used in other Absolute format tables, such as debug symbol names, file names, and embedded comments. The header record has the following form:

```
struct abs_file_strtable {
    unsigned long strtable_addr;
    unsigned long strtable_count;
};
```

The `strtable_addr` is the location within the output file of the string table, and `strtable_count` is the number of strings. The string table has two parts: a "table of contents", and the actual strings. Other Absolute file records that refer to strings do so with an index into the table. The string table of contents appears at the `strtable_addr`, and consists of `strtable_count` consecutive instances of the following structure:

```
struct abs_strtable {
    unsigned long string_addr;
    unsigned long string_length;
};
```

The `string_addr` is the location in the Absolute file of the string itself, and `string_length` is its length, not including the terminating NUL character and padding. The first entry in the table, that for string index 0, is reserved for a zero-length string; a string index of 0 in the Absolute file means "no (known) string".

The string table of contents refers to the actual string text by file address. Normally (but not necessarily), the strings immediately follow the table of contents, in the same order as the table of contents. At least one NUL (0) character, padded to a 4-byte boundary, follows each string.

A.2.8 SYMTABLE Header and Data Records

The optional `SYMTABLE` header record and corresponding data records contain the debug symbol table. Note that debug symbols can be defined arbitrarily by assembler `.dbg` directives, and do not necessarily correspond to assembler/linker symbols (i.e. labels). The header record has the following form:

```
struct abs_file_symtable {
    unsigned long symtable_addr;
    unsigned long symtable_count;
};
```

The `symtable_addr` is the location within the output file of the debug symbol records, and `symtable_count` is the number of debug symbols. Like the string table, the debug symbol table has two parts: a table of contents (with fixed-size entries) followed by the debug symbols themselves (which may be of varying size). The table of contents appears at the `symtable_addr`, and consists of `symtable_count` consecutive instances of the following structure:

```
struct abs_symtable {
    unsigned long symbol_addr;
    unsigned short symbol_interp;
```

```
    unsigned short symbol_size;
};
```

The `symbol_addr` gives the location in the Absolute file of the debug symbol record, and the `symbol_size` gives its length in bytes. The `symbol_interp` is zero, except in custom debug symbol formats. The first entry in the table, that for debug symbol index 0, is reserved; a debug symbol index of 0 in the Absolute file means "no (known) symbol".

The debug symbol table of contents refers to the actual debug symbol by file address. Normally (but not necessarily), the debug symbols themselves immediately follow the table of contents, in the same order.

By default, a debug symbol entry consists of a series of long words. The first (index zero) long word provides the debug symbol name, as a string table index. Subsequent long words contain the "info" values defined by `.dbg` directives, in order by `infotype`, from 1 up to the highest existing "infotype" for that debug symbol.

A.2.9 FILETABLE Header and Data Records

The optional `FILETABLE` header record and corresponding data records contain a table of file names, used for debugging. The header record has the following form:

```
struct abs_file_filetable {
    unsigned long filetable_addr;
    unsigned long filetable_count;
};
```

The `filetable_addr` is the location within the output file of the file table, and `filetable_count` is the number of file table entries. Each record has the following form:

```
struct abs_filetable {
    unsigned short reserved1;
    unsigned char reserved2;
    unsigned char level;
    unsigned long name;
};
```

The `name` field is the file name, as an index into the string table. The `level` field describes the type of file. See the description of the assembler `.file` directive for details. The first file table entry, with index 0, is reserved, and a file table index of 0 means "no (known) file".

A.2.10 LINETABLE Header and Data Records

The optional `LINETABLE` header record and corresponding data records contain a line number table, for debugging. The header record has the following form:

```
struct abs_file_linetable {
    unsigned long linetable_addr;
    unsigned long linetable_count;
};
```

The `linetable_addr` is the location within the output file of the line number table, and `linetable_count` is the number of line number table entries. Each line number record has the following form:

```
struct abs_linetable {
    unsigned short file;
    unsigned char level;
    unsigned char memtype;
    unsigned long line;
    unsigned long address;
};
```

The `line` field is the line number itself, the `file` field is the file containing the source line, expressed as an index into the file name table, and `level` is that file's level. The `address` is the memory location associated with the source line number, and `memtype` is that location's memory type.

Line number table entries are sorted by memory type and by address within each memory type. This sorting allows the table to be scanned for a line number(s) matching or surrounding a particular address.

A.2.11 COMMENT Header and Data Records

The optional `COMMENT` header record and corresponding data records contain embedded comments, as defined by assembler `.comment` directives. The header record has the following form:

```
struct abs_file_comment {
    unsigned long comment_addr;
    unsigned long comment_count;
};
```

The `comment_addr` field is the location within the output file of the comment table, and `comment_count` is the number of comment records. Each record has the following form:

```
struct abs_comment {
    unsigned long comment;
};
```

The `comment` is a string table index to the embedded comment text.

A.2.12 LOADMAP Header and Data Records

The optional `LOADMAP` header record and corresponding data records contain a load map equivalent to that generated by the `--map` command line option. The header record has the following form:

```
struct abs_file_loadmap {
    unsigned long loadmap_addr;
    unsigned long loadmap_count;
};
```

The loadmap_addr is the location within the output file of the load map, and loadmap_count is the number of segment records in the load map. Each load map segment record has the following form:

```
struct abs_loadmap_segment {
    unsigned long name;
    unsigned char reserved1;
    unsigned char memtype;
    unsigned short numsymbols;
    unsigned long startaddr;
    unsigned long endaddr;
};
```

The name is the segment name, as a string table index. The memtype is its memory type, and startaddr and endaddr describe the memory range of the segment. The numsymbols field gives the number of load map symbol records for that segment. These symbol records immediately follow the segment record, and each has the following form:

```
struct abs_loadmap_symbol {
    unsigned long name;
    unsigned long loadaddr;
};
```

The name is the symbol name, and the loadaddr is its address.

These symbols correspond to assembler labels, and should not be confused with the arbitrary debug symbol table entries.

A.3 RELOCATABLE OBJECT FILE FORMAT

The CFX assembler generates relocatable object files in a format referred to as “MC06”, which is described in this appendix.

A.3.1 Object File Structure

An object file consists of one or more object modules. An object module begins with a start record, followed by segment, symbol, text, bss, comment, target, debug symbol, file name, line number, and string records. The object module is terminated by an end record. There is one situation in which a module does not have this form. An object module can be empty, in which case it consists of a single end record. This case is the only one in which an object module can begin with something other than a start record.

There must be at least one segment record. The segment and symbol records are processed by pass 1 of the linker. The text records (if any) are processed by pass 2 of the linker.

A.3.2 Record Header

Each record starts with a record header, the structure of which is:

```
struct inhdr {
    /* # of bytes which follow the record header */
    unsigned short nbytes;
```

```
    unsigned short rectype;
};
```

The `rectype` field can have the following values:

```
REC_SEG     1
REC_SYM     2
REC_TEXT    3
REC_END     4
REC_START   5
REC_COMMENT 6
REC_BSS     7
REC_TARGET  8
REC_DBSYM   9
REC_FILE    10
REC_LINE    11
REC_STR     12
```

A.3.3 Name Fields

All names in object module records can be stored in two different forms, as described by `union name`:

```
union name {
    char str[NAMESIZE];
    struct {
        unsigned long extra;
        unsigned long offset_in_strtab;
    } mc06;
};
```

Names that are 33 characters or less in length are stored in the `str` array. Longer names are stored in the string table, in which case the `extra` field contains zero and the `offset_in_strtab` field is the relative offset of the name, from the start of the first string record in the string table.

There is one case in which a short name is *not* stored in the `str` array: the module name in a start record is *always* stored in the string table, and in this record the `extra` field in the `name` union stores the offset of the start (after the header) of the first string record in the string table, relative to the start (after the header) of the start record.

Thus the seek offset of a string in the string table is the sum of the location of the start record, the string table offset stored in the `extra` field of the start record, and the `offset_in_strtab` field of the `name` union.

A.3.4 Start Record

The object file begins with a start record:

```
struct startrec {
    char magic[4];
    union name modulename_strtaboffset;
```

```
    time_t time;
};
```

The `magic` array is a constant that identifies the object module's format. Its value is "MC06".

The `modulename_strtaboffset` field is used for the module's name, which is derived from the source file name. The module name is *always* stored in the string table, and the `modulename_strtaboffset` field also provides the location of the string table: the `extra` field in the `name` union stores the offset of the start (after the header) of the first string record in the string table, relative to the start (after the header) of the start record.

The `time` field is the time at which the object module was created.

A.3.5 Segment Record

The segment record has the following structure:

```
struct segrec {
    unsigned short segid;
    unsigned short segbits;
    /* segment type flags: abs/rel, common */
    unsigned char segkind;
    unsigned char segtype;
    /* alignment requirement (power of two) */
    unsigned char segalign;
    unsigned char seginc; /* pc increment */
    ADDR segbase; /* base address of the segment */
    ADDR reserved;
    union name segname;
};
```

The `segid` field is the segment number within the current module. This number is referenced in the symbol and text records.

The `segbits` field is the segment's wordsize, from the `wordsize=` attribute of the `.segment` directive.

The `segkind` field consists of flags that indicate whether the segment is absolute or relative, and whether it is a common, blank common, or ordinary segment. This field's value uses the following bit flags:

```
#define SEG_ABS         0x1         /* absolute segment */
#define SEG_COMMON      0x2         /* named common segment */
#define SEG_BLANK       0x4         /* blank common segment */
#define SEG_ORDERED     0x8         /* fixed order segment */
#define SEG_RO          0x10        /* read only segment */
#define SEG_CODE        0x20        /* segment may contain code */
#define SEG_DATA        0x40        /* segment contains no code */
```

The `segtype` field is the segment's memory type, from the `memtype=` attribute of the `.segment` directive.

The segalign field stores the segment's alignment requirement: the maximum power of two of any .align or .fill directive given in the segment. The alignment requirement is stored as the power of two, so if the segment must be aligned on an address which is a multiple of 8, this field stores the value 3.

The seginc field is the segment's "location counter increment", from the locinc= attribute of the .segment directive. It will always be 1 for CFX programs.

The segbase field is the segment's base address, from the base= or abs= attributes of the .segment directive. The type ADDR is unsigned and is 32 bits wide.

The segname field contains the segment's name.

A.3.6 Symbol Record

A symbol record is generated for any name in a .global or .extern directive, or in a .declare directive followed by a reference to the name in the code. The symbol record is laid out as follows:

```
struct symrec {
    short segid;     /* == -1 if sym ref */
    short symid;
    CONST symloc;
    union name symname;
};
```

If the symbol is an external reference, the segid is -1. Otherwise the record is a global definition and segid contains the number of a preceding segment record. The symid is the symbol number within the current file. The number is referenced in text records. The symloc is the location of the symbol relative to the start of the segment in the current file. The type CONST is 32 bits wide.

A.3.7 Text Record

Text records contain the data to be linked along with relocation information. The text record begins with the text record header:

```
struct textrec {
    short segid;
    short numdata;  /* number of text items */
    short datasize; /* item size in bytes */
    short numreloc; /* number of relocation records */
    ADDR loc;
};
```

As with symbol records, the segid refers to one of the segment records.

The loc field is the offset in the segment of the first text item. The type ADDR is unsigned and is 32 bits wide.

The text record header is followed by numdata text items (that is, instructions or data). Each item consists of datasize bytes. The datasize is always a multiple of four, since the assembler uses units of 32 bits. If the width of a text item is less than 32 bits, it is padded on the right to the

next 32-bit boundary. For example, if the text item is 24 bits, then the `datasize` is four and each entry is a 32-bit word with the data in the least significant 24 bits.

If any of the text items contain fields that must be relocated by the linker, the text items are followed by relocation records (see Section A.3.8, “Relocation Record” below).

The total size in bytes of the text record (not including the header common to all records) is:

```
sizeof(struct textrec) + numdata*datasize + numreloc*sizeof(struct reloc)
```

A.3.8 Relocation Record

Relocation records may follow text records or debug symbol records. Each relocation record appears as follows:

```
struct reloc {
    char reloc_type;
    char rel_shift;
    char low_bit;
    char high_bit;
    short dataloc;
    short id;     /* seg or sym id, depending on reloc_type */
    CONST rel_mask;
    CONST rel_val;
};
```

The type `CONST` is 32 bits wide.

There are two kinds of relocations. A simple relocation consists of a single relocation record. An extended relocation encodes an arbitrary expression in a sequence of relocation records.

A.3.8.1 Simple Relocations

A simple relocation encodes a constant relative to a segment or symbol.

The `dataloc` is the text item number (relative to the start of the current text record’s text items). It specifies in which text item a field is to be relocated. The `rel_val` is a constant that is added to the relocated result before placing it in the text item.

The `reloc_type` contains three fields that indicate:

1. The relocation type (bits 0-3)
2. The relocation transformation (bits 4-5)
3. The signedness of the field to be relocated (bits 6-7)

The relocation types are:

```
#define R_ABSREL   0x0
#define R_SEGREL   0x1
#define R_SYMREL   0x2
#define R_EXPREL   0x3
#define R_EXPR     0x4
```

For simple relocation, the type is either relative to a segment (R_SEGREL) or relative to an external symbol (R_SYMREL).

The relocation transformations are:

```
#define R_ALL     0x00
#define R_BITS    0x10
#define R_ML2     0x20
```

The field's signedness is one of:

```
#define RS_NOSIGN   0x00
#define RS_UNSIGNED 0x40
#define RS_SIGNED   0x80
#define RS_NEGATIVE 0xc0
```

The id contains either a segid or a symid, according to the relocation type.

After adding the rel_val to the symbol address or segment base address, the result is modified according to the relocation transformation:

R_ALL	use all bits of the relocated address
R_BITS	use .bits(low_bit, high_bit, address)
R_ML2	use .mul2(address)

The transformed result is checked to verify that it does not overflow the rel_mask, where the valid range is determined according to the field's signedness. It is then shifted left by rel_shift bits, and placed in the text item selected by dataloc. If the result does not fit within the rel_mask, CFCC generates an error message and all bits that do not fit within the mask are ignored.

A.3.8.2 Extended Relocations

An extended relocation consists of at least two relocation records. The last relocation record has a reloc_type of 3 (R_EXPREL), and specifies how the value is to be encoded in the data, using rel_mask, rel_shift, and dataloc in exactly the same way as in a simple relocation. The low_bit, high_bit, id, and rel_val fields are unused.

The value to be encoded is determined by the preceding sequence of relocation records, which all have a reloc_type of 4 (R_EXPR). Each of these relocation records represents one item in the expression, which is written in stack-oriented (Reverse Polish) format, with operands preceding each operator. The rel_shift field indicates the kind of item: expression operator, number, symbol or segment. For an expression operator, all the other fields of the relocation record are unused. For a number, the rel_val field contains the value, and the remaining fields are unused. For a symbol or segment, the id field contains the segid or symid, and the remaining fields are unused.

The relocation operands used in the rel_shift field are:

```
#define RO_NONE         0
#define RO_NUMBER       '#'
#define RO_SEG          'S'
```

```
#define RO_SYM          'Y'
#define RO_ADD          '+'
#define RO_SUB          '-'
#define RO_NEG          'N'
#define RO_MUL          '*'
#define RO_DIV          '/'
#define RO_MOD          '%'
#define RO_NOT          '~'
#define RO_AND          '&'
#define RO_OR           '|'
#define RO_XOR          '^'
#define RO_LSH          'L'
#define RO_RSH          'R'
#define RO_BITS         'B'
```

A.3.9 Uninitialized Data Record

The format of uninitialized data (bss) records, which are produced by the `.bss` directive, is:

```
struct bssrec {
    unsigned short segid;
    unsigned long numdata;
    unsigned short datasize;
    ADDR loc;
};
```

The fields in the bss record are analogous to those in the text record. The bss record is not followed by any text items or relocation records because the data area is uninitialized. The corresponding memory region will be reserved but uninitialized when the object file is linked into an executable, but the behavior of program loaders varies for such memory regions; some loaders clear bss regions and some do not modify them.

A.3.10 Comment Record

Comment records, which are produced by the `.comment` directive, consist of a record header followed by a NUL-terminated string.

A.3.11 Target Record

Target records consist of 8 bytes that identify the target architecture. For CFX programs, these 8 bytes contain the value `CFX\0\0\0\0\0`.

A.3.12 Debug Symbol Record

Debug symbol records appear if you use CFCC's `-g` flag or if you define debug symbols using the `.dbg` directive. They appear in the object file in the order in which the symbol, segment, or `.dbg` directive appears in the input file.

Each debug symbol record begins with a debug record header:

```
struct dbrec {
    unsigned long dbsymid;
    unsigned short numinfo;
```

```
    unsigned short numreloc;
    unsigned short mininfotype;
    unsigned short maxinfotype;
    union name dbname;
};
```

This record is followed by `numinfo` instances of the following:

```
struct dbinforec {
    unsigned short kind;
    unsigned short infotype;
    unsigned long info;
};
```

These instances are in turn followed by `numreloc` relocation records, described in Section A.3.8, "Relocation Record" on page 126. The relocation records are followed by strings, one for each `dbinforec` whose `kind` is `DBSYM_STR`. The string length is held in the `info` field.

The valid values for the `kind` of debug information are:

```
#define DBSYM_NONE     0
#define DBSYM_ABS      1
#define DBSYM_REL      2
#define DBSYM_STR      3
#define DBSYM_FIL      4
#define DBSYM_DBS      5
```

A.3.12.1 Debug Symbol Table Entries for Symbols

When the `y` debug option is in effect, the assembler generates a debug symbol table entry for each assembler symbol (that is, for each label, and for each name defined in a `.equ` directive). The debug symbol has five or six `infotype` entries (not counting its name), described in Table 41. The sixth entry is present only if the `s` debug option is in effect.

TABLE 41: DEBUG INFO FOR ASSEMBLER SYMBOLS

Infotype	Value
1	Symbol's address.
2	Symbol's memory type.
3	Source file (level 12) in which the symbol is defined.
4	Line number at which the symbol was defined.
5	Flags. Bit 16 (`0x00010000`) is set to identify an assembler symbol. Bit 8 (`0x00000100`) is set if the symbol is global. Bit 0 (`0x00000001`) is set for a symbol defined by a label, and bit 1 (`0x00000002`) is set for a symbol defined by a `.equ` directive.
6	The debug symbol table index of the segment in which this symbol is defined.

You can add information items to these automatically generated debug symbol table entries using a `.dbg` directive with `infotype` values from `7` on. You cannot redefine or override the automatically generated values.

If the `m` debug option is in effect, the name of the symbol is prefixed with a colon (`:`) to avoid accidental conflict with debug symbols defined by `.dbg` directives.

A.3.12.2 Debug Symbol Table Entries for Segments

When the `s` debug option is in effect, the assembler generates a debug symbol table entry for each segment. The debug symbol has ten infotype entries, not counting the name. See Table 42.

TABLE 42: DEBUG INFO FOR ASSEMBLER SEGMENTS

Infotype	Value
`1`	Segment's `base=` or `abs=`address, if any.
`2`	Segment's memory type.
`3`	Source file (level 12) in which the segment is first declared.
`4`	Line number at which the segment is first declared.
`5`	Flags. Bit 17 (`0x00020000`) is set to identify an assembler symbol. Bits 0 through 7 duplicate the `segkind` field of the segment record.
`6`	Segment's wordsize.
`7`	Segment's location counter increment.
`8`	Segment's alignment.
`9`	Segment's start address.
`10`	Segment's end address.

The start address and end address include all components of the segment from all object files ultimately linked together. The other fields refer to the values defined by the `.segment` directive in each individual object file.

In the debug symbol table, the segment name is prefixed with a semicolon (;) to avoid conflict with debug symbol entries for assembler symbols and to avoid accidental conflict with debug symbols defined by `.dbg` directives.

You can add additional info items to these automatically generated debug symbol table entries using a `.dbg` directive with "infotype" values from 11 on. For example:

```
    .segment foo
foolbl:
    .dbg {;foo},11,foolbl,12,"My Favorite Segment"
```

You cannot redefine or override the automatically generated values.

A.3.13 File Name Record

The format of file name table entries is:

```
struct filerec {
    unsigned short fileid;
    unsigned short level;
    union name filename;
};
```

The `fileid` field is referenced by line number records.

Valid file levels are:

```
#define FLV_UNKNOWN  0
#define FLV_C        4  /* reserved for C programs */
#define FLV_POSTC    6  /* reserved for C programs */
#define FLV_PM       7  /* #macro line numbers refer to point of call */
#define FLV_V        8  /* pre-VOP */
#define FLV_VOP      10 /* processed VOP (raw CFPACK input: ;VL lines) */
#define FLV_A        12 /* pre-ASM: assembler #lines */
#define FLV_ASR      14 /* raw assembler input */
#define FLV_ALST     15 /* assembler listing file */
```

A.3.14 Line Number Record

The format of line number table entries is:

```
struct linerec {
    ADDR addr;
    short segid;
    unsigned short fileid;
    unsigned long line;
};
```

The `segid` and `addr` fields specify the memory address (relative to the given segment) to which the source code line maps. The type `ADDR` is unsigned and is 32 bits wide.

The `fileid` and `line` fields specify the source file line number that corresponds to the memory address.

A.3.15 String Record

A string record contains a NUL-terminated sequence of bytes.

The assembler writes all string records at the end of the module, just before the end record. The collection of string records constitutes the string table, which includes the module name, plus any names which are too large to fit into the name fields of their respective records.

A.3.16 End Record

The end record marks the end of the module and consists of only the record header with `nbytes` set to zero.

A.4 NON-VOLATILE MEMORY IMAGE FORMAT (.NVI)

The NVI file format is a non-proprietary, binary file format for storing non-volatile memory images once the layout has been processed. It stores data in a format that is quick and easy to extract for production, while maintaining sufficient metadata to read back the layout with no loss of information. The IDE provides a read-only editor, allowing the contents of the NVI file to be viewed, as well as a virtual representation of the values in non-volatile memory. NVMD can also be used with the `-v` option to output file information while the NVI file is being downloaded to the device's EEPROM. You can dump the contents of NVI files to a variety of formats using the NVIDUMP tool; see Section 10.3, "Non-Volatile Image Dump Tool" on page 113 for more information.

NVI files can be created through NVMM in the following ways:

- Using the IDE builder
- Uploading of the EEPROM contents directly from a device through the IDE

Using any of these methods, the file created conforms to the NVI file specification. The following is an outline of the current NVI file specification.

A.4.1 NVI File Structure

NVI files contain binary data encoding the complete layout of the EEPROM image as it will be downloaded to an attached device. The fundamental components of the file structure can be broken down into the following elements:

- Headers
- String Table
- VFS Node Chunks
- Data Chunks
- CRC Values

A.4.2 Headers

There are essentially two types of headers in a `.nvi` file: the main file header, which is a single entry found at the beginning of the file; and block headers, based on a generic template, which serve as the starting point for each block.

A.4.2.1 Main File Header

An NVI file must start with a proper file header. The header determines whether or not the file is a valid NVI file. It also serves as the starting point for any type of traversal or parsing of layout information. The structure of the file header is outlined in Table 43.

TABLE 43: MAIN FILE HEADER STRUCTURE (36 BYTES)

Field Size	Field Name	Definition
byte[8]	MAGIC_NUMBER	Magic identifier for validation: actual value "SORA:SAI".
int32	HEADER_SIZE	Size of the main header in bytes (36).

TABLE 43: MAIN FILE HEADER STRUCTURE (36 BYTES) (CONTINUED)

Field Size	Field Name	Definition
int8	VERSION_MAJOR	Major version for file specification. 0x00 for the current version.
int8	VERSION_MINOR	Minor version for file specification. 0x04 for the current version.
int16	BLOCK_COUNT	The number of blocks in this file.
int32	FEATURES	A set of flags designating features present in the file.
int16	TARGET_FS	Target file system.
int8	TARGET_FS_VER_MAJOR	Major version of the target file system.
int8	TARGET_FS_VER_MINOR	Minor version of the target file system.
int32	DESCRIPTION_ID	Pointer to a description string in string table.
int32	COPYRIGHT_ID	Pointer to a copyright message in string table.
int8	ATOMIC_UNIT	The size in bits of the atomic unit used in the file system of the stored image.
int8	BYTE_ORDER	The byte order for the stored image (0=big-endian, 1=little-endian).
int16	VFS_ROOT	Block pointer to the VFS root node. Null if and only if the VFS_EMBEDDED feature flag is not set.

A list of the currently supported features, which collectively contribute the features present in the file through a bitwise OR of their individual values, can be found in Table 44.

TABLE 44: SUPPORTED FEATURE FLAGS

Value	Feature Name	Definition
0x00000001	COMPLETE_RAW	Denotes a completely raw image containing only a single data block
0x00000002	HAS_KEYS	Specifies an image containing a key section to store shared keys for level 1 security (reserved for future use)
0x00000004	VFS_EMBEDDED	Indicates the presence of an embedded tree structure containing extended layout information in the image

A list of file systems currently supported by the NVI file format is included in Table 45.

TABLE 45: SUPPORTED FILE SYSTEMS

ID	File System Name	Definition
0x0003	SK3FS	The SK3 file system, used in BelaSigna 300.

A.4.2.2 Generic Block Header

While the main file header outlined in the section above is unique in the NVI file structure, it also shares similarities with the generic block header that serves as a template for every block in the file. For example, you can see that the `MAGIC_NUMBER` that begins the main file header is simply a specific application of the generic file header `BLOCK_TYPE` field. The structure of the generic block header is outlined in Table 46.

TABLE 46: GENERIC BLOCK HEADER STRUCTURE (16 BYTES)

Field Size	Field Name	Definition
byte[8]	BLOCK_TYPE	8-character string uniquely identifying the block type
int32	BLOCK_HEADER_SIZE	Total size of the block header (not including body)
int32	BLOCK_SIZE	Total size of the block (including the header and body). *Note:* Pad all blocks to a 32-bit boundary: (BLOCK_SIZE % 4 = 0).

Blocks are placed sequentially in the NVI file, one after another in increasing order, starting at 0. Block pointers, used as references between blocks, hold a 16-bit integer value specifying the unique number of the block to which they point. A block pointer of 0xFFFF represents a null block (i.e., no block). It is possible to iterate through the blocks in the NVI format by reading the generic block header and using the BLOCK_SIZE field to jump to the next block, and so on. The following sections describe some of the specific instances of blocks used in the NVI format.

A.4.3 String Table

The string table holds any human-readable strings referenced within the NVI file structure. For example, the strings can be used to denote names and attributes. This block is unique within the NVI file, occurring only once. In a sequential block listing, it is the first to follow the main file header that contains an actual body. The strings contained in the body of the block are encoded with the `UTF-8` format, terminated by the '`\0`' character. Null characters are used to pad the table to the 32-bit boundary.

Note: The information in the string table is useful when working with your NVI image in the IDE; however, it is not included in the information downloaded to the device.

The string table is preceded by a header based on the generic block header discussed above. The actual structure of the string table header is outlined in Table 47.

TABLE 47: STRING TABLE BLOCK HEADER

Field Size	Field Name	Definition
byte[8]	BLOCK_TYPE	Actual value "StrTable"
int32	BLOCK_HEADER_SIZE	20 bytes
int32	BLOCK_SIZE	BLOCK_HEADER_SIZE + sizeof(string data) + padding
int32	STR_COUNT	The number of strings in the table.

The string pointers found in other blocks in the NVI file point to an offset from the beginning of the string data in the string table. Use the following offsets to calculate the absolute position of a string in the NVI file:

```
offset to start of string table header + BLOCK_HEADER_SIZE of string table
   header + offset to string in table data (string pointer value)
```

The value of 0xFFFFFFFF is a special value that refers to the null string.

A.4.4 VFS Node Chunks

If the main file header includes the VFS_EMBEDDED feature among the set flags encoded in the FEATURES field, then the VFS_ROOT field (also in the main header) will contain a block pointer to the first (root) VFS node chunk. The VFS node chunks encode additional layout information including real and virtual parent-child relationships, attributes, human readable names and node typing, as well as the physical data chunks.

The structure of the VFS nodes is analogous to a tree, where the root node (following with the tree analogy) is the layout itself. Children of the layout can be virtual or real. Real elements contained in the VFS structure are always associated with a data chunk. VFS node chunks are ordered consecutively, following the string table in the NVI file. Each VFS node chunk starts with a header. The structure of the header is outlined in Table 48.

TABLE 48: VFS NODE CHUNK HEADER

Field Size	Field Name	Definition
byte[8]	BLOCK_TYPE	Actual value “VsfNode”
int32	BLOCK_HEADER_SIZE	36 bytes
int32	BLOCK_SIZE	BLOCK_HEADER_SIZE + (8 * ATTRIB_COUNT) + (2 * CHILD_COUNT) + padding
byte[8]	NODE_TYPE	8-character string denoting node type. Implementation is file system specific
int32	NODE_NAME	String pointer to a human-readable name of a block
int16	DATA_CHUNK	Block pointer to an associated data chunk; can be null (0xFFFF) for nodes with no data (virtual)
int16	DATA_SUBCHUNK	Reserved for future use; must be null (0xFFFF) in the current version
int16	ATTRIB_COUNT	The number of attributes stored in this node.
int16	CHILD_COUNT	The number of child nodes associated with this node.

Following the VFS node chunk header, the references to attributes and children are placed consecutively in their respective order. Attributes are stored as 32-bit string pointer to a key in the string table, followed by another 32-bit string pointer to the associated value. Child references take the form of a 16-bit block pointer indicating the associated child node.

Note: Names and attributes associated with the VFS and stored in the string table are generally derived from settings in the NVMM builder (e.g., application and section names from `.abs` source files). When the information is not available,

such as when an NVI image is created from uploading a device EEPROM, default names and attributes are generated.

A.4.5 Data Chunks

The data chunks contain the physical data that is ultimately transferred to the device EEPROM (or in the case of an NVI image created by uploading the contents of a device EEPROM, they hold the data which has been read.) Each data chunk is, as with the other blocks already discussed, preceded by a header based on the generic template. The specific structure of a data chunk header is outlined in Table 49.

TABLE 49: DATA CHUNK HEADER

Field Size	Field Name	Definition
byte[8]	BLOCK_TYPE	Actual value “DatChunk”
int32	BLOCK_HEADER_SIZE	32 bytes
int32	BLOCK_SIZE	BLOCK_HEADER_SIZE + CHUNK_SIZE + padding
int16	NODE	Block pointer to associated VFS node; can be null (0xFFFF)
int16	SUB_CHUNK_COUNT	Reserved for future use. Must be 0x0000 in the current version.
int32	CHUNK_SOURCE	String pointer to source of block (e.g. application filename) in string table.
int32	CHUNK_SIZE	Size in bytes of the raw data contained in body to follow.
int32	DEST_ADDRESS	The destination address (in non-volatile memory) to which this chunk of data is to be copied.

The data that follows the header is stored as raw bytes to be transferred to the device during a download operation.

A.4.6 CRC Values

Note that in the interest of data validation and integrity, the NVI specification includes a CRC value calculated over the file as a whole. The file CRC is calculated as a 32-bit integer and written to the last 4 bytes of the file. Any conflicts detected in the CRC verification are generally interpreted as file errors.

B Configuration Template Schema

B.1 Introduction

The structure of the Configuration Template Format is described by an XML schema. The details of the schema definition and an example of the template structure are outlined below.

B.2 Template Example

The example provides the skeleton structure of a configuration template file:

```
<?xml version="1.0" encoding="UTF-8"?>
<pt:template xmlns:pt="http:///com.signaklara.filesupport.ui/schemas/
   PTemplate.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xsi:schemaLocation="http:///com.signaklara.filesupport.ui/schemas/
   PTemplate.xsd PTemplate.xsd ">

   <target>acoustic_echo_canceller.abs</target>
   <version>1.0.1</version>
   <date>2010-08-04 12:35:37 -0400</date>
   <desc>Configuration template based on the acoustic_echo_canceller.abs
   target.</desc>
   <parameters>
      <!-- ... zero or more parameter elements -->
   </parameters>
   <!-- optional commands element -->
   <commands>
      <!-- ... zero or more command elements -->
   </commands>
</pt:template>
```

B.3 Schema

The configuration template schema is an XML document. The example in Section B.2, “Template Example” makes use of the standard configuration template schema, as referenced in the `<pt:ptemplate>` element. The schema defines the namespaces used by the template, and the required and optional elements, along with rules for their validation. The following sections provide additional details on the key elements/sections contained in the template schema.

B.3.1 <pt:template> Root Element

The root element of the document is designated as `pt:template`. Each `pt:template` element contains the elements described in Table 50, in the specified order, as outlined in Section B.2, “Template Example”.

TABLE 50: CHILD ELEMENTS OF <PT:TEMPLATE>

Order	Name	Min/Max Occurrences	Description
1	target	0/1	Optional string value containing the path (relative to the given template file) to the target containing the symbolic data on which the parameters are defined.
2	version	1/1	Required numeric version code for schema reference. The current version code is 1.0.1.
3	date	1/1	Required last modification date of the template file, specified in this format: **`YYYY-MM-DD hh:mm:ss GMT`**, where GMT is in the form of four digits and a sign (e.g., -0400).
4	desc	0/1	Optional string value containing a description of the current template file.
5	parameters	1/1	Required element group containing zero or more `parameter` element definitions. Refer to Section B.3.2, "<parameter> Elements" for details.
6	commands	0/1	Optional element group containing zero or more `command` element definitions. Refer to Section B.3.3, "<command> Elements" for details.

B.3.2 <parameter> Elements

The schema requires one `parameters` element containing zero or more individual `parameter` elements. An unlimited number of parameters can be defined in a configuration template. The `parameter` element specification includes the elements described in Table 51, in the specified order.

TABLE 51: CHILD ELEMENTS OF <PARAMETER>

Order	Name	Min/Max Occurrences	Description
1	name	1/1	Required string value containing a friendly name for the parameter. By default, this is set to the symbol name.
2	symbol	1/1	Required string value specifying the name of the symbol in the target containing the data to be parameterized.
3	method	1/1	Required complex value specifying the update methods supported by the current parameter. Must always include the `memory` method, followed by an optional `i2c` method.

TABLE 51: CHILD ELEMENTS OF <PARAMETER> (CONTINUED)

Order	Name	Min/Max Occurrences	Description

4	type	1/1	Required enumerated value specifying the designated parameter type. Current supported types include:
			
			`numeric value`
			`numeric range`
			`enum value`
			`file`

5	group	0/1	Optional string value designating a group to which the current parameter belongs. Used for filtering parameter views only. Parameters with the same group value can be filtered and displayed together; other parameters are hidden from view as long as filter is applied.
6	wordcount	1/1	Required integer value greater than zero specifying the size of the parameterized data value in words.
7	wordsize	1/1	Required integer value greater than zero specifying the size of a word in the parameterized data in bits. The value typically corresponds to the wordsize of the memory space designated by the target data.
8	restriction	0/1	Optional complex value used to specify any restrictions to be used in data validation for the parameter values. Can be one of: • `enumRestriction` - used to specify name/value options for enumerated values • `rangeRestriction` - used to specify a min/max value for bounded numeric values
9	default	1/1	Required default parameter value corresponding to the parameter type.
10	value	0/1	Optional user-defined parameter value corresponding to the parameter type.

The parameter element specification also includes the required attributes described in Table 52.

TABLE 52: ATTRIBUTES IN <PARAMETER>

Name	Type	Description
read_only	boolean	Specifies that the parameter is read-only. If read_only="true", the option of setting a user-defined value for the parameter is prohibited and the default value is used for any data substitution operations. The default value is "false".
hidden	boolean	Specifies that the parameter is hidden. If hidden="true", the parameter is not displayed in the available parameter list. The parameter is still included in any data substitution operations, regardless of the value of the attribute. The default value is "false".

B.3.3 <command> Elements

The schema allows one `commands` element containing zero or more individual `command` elements. An unlimited number of commands can be defined in a configuration template. The `command` element specification includes the elements described in Table 53, in the specified order.

TABLE 53: CHILD ELEMENTS OF <COMMAND>

Order	Name	Min/Max Occurrences	Description
1	name	1/1	Required string value specifying the name of the given command.
2	desc	0/1	Optional string value providing a brief description of the command function. For general I^2C commands, this is displayed in the I2C Commands section of the Configuration Template Editor. Formatting options using `\t` for a tab character and `\n` for a newline are supported in the description string.
3	address	0/1	Non-negative integer value specifying an I^2C slave address to use in addressing the current command. Use this element to override the default debug address for the command. Depending on your device, you might have to use this element.
4	code	1/1	Required string value designating a unique command number. The value is specified as a two-character hexadecimal byte value in the range 0x00 to 0xFF inclusive, with or without the preceding 0x.
5	txbytes	1/1	Required non-negative integer specifying the number of bytes of data sent as part of the command payload.
6	rxbytes	1/1	Required non-negative integer specifying the number of bytes of data expected in the command response upon execution.

TABLE 53: CHILD ELEMENTS OF <COMMAND> (CONTINUED)

Order	Name	Min/Max Occurrences	Description
7	formatter	0/1	Optional string value specifying a response parser used to apply formatting to the command response.
8	chain	0/1	Optional string value specifying a command or list of commands (separated by commas) to be executed on completion of this command. Chained commands are specified by their command number using the format described above.

Global Headquarters
ON Semiconductor
5005 East McDowell Road
Phoenix, Arizona 85008 USA
Tel: +1 602 244 6600

Website
www.onsemi.com

Copyright © 2011 Semiconductor Components Industries, LLC d/b/a ON Semiconductor. ON Semiconductor and the ON logo are registered trademarks of Semiconductor Components Industries, LLC (ON Semiconductor). BELASIGNA and SIGNAKLARA are either trademarks or registered trademarks of ON Semiconductor. All other brands, product names and company names mentioned herein may be trademarks or registered trademarks of their respective holders.